컴파일러 과제5 보고서

20221500 양성준

목차

1. 구현 **1**
1-1. 구현 환경 1
1-2. 프로그램 구성 1
1-3. 구현 및 변경 사항 2
1-4. 주요 구현 원리 2
1-5. 구현 범위 및 제한사항 2
2. 검증 **3**
2-1. 테스트 구성 3
2-2. 테스트 결과 3
2-3. AST 구조 검증 4
2-4. 오류 검증 10
2-5. 테스트 케이스 상세 11
3. 개발 과정 **33**
3-1. 오류 및 해결 기록 33

1. 구현

1-1. 구현 환경

UNIX 환경에서 빌드하고 테스트했습니다.

1-2. 프로그램 구성

입력받은 C 코드를 토큰으로 나눈 뒤, 문법 규칙에 따라 구문 트리(AST)와 선언 정보를 생성하도록 구현했습니다. 주요 파일의 역할은 다음과 같습니다.

파일	주요 역할
c.l	토큰 인식, 줄 번호 처리, 어휘 오류 보고
c.y	문법 규칙, AST 생성, 구문 오류 보고
semantic.c / semantic.h	심볼과 타입 정보 관리, 스코프 처리, 선언과 전방 참조 검사
syntax_tree.c / syntax_tree.h	makeNode와 makeNodeList로 AST 노드 생성
type.h	A_NODE, A_ID, A_TYPE 등 구조체와 열거형 정의
print.c	구성된 신택스 트리 출력
error.c / error.h	오류 분류, 오류 번호와 메시지 출력
main.c	initialize(), yyparse(), print_ast(root)를 차례로 호출
parse_value.h	포인터 크기에 맞는 YYSTYPE 선언과 형 변환 매크로

1-3. 구현 및 변경 사항

5장 강의노트와 부록 A1~A4, A9를 바탕으로 하고, 일부 제 스타일로 구을 진행해보았습니다.

- 문법 범위: 초기화식과 저장 클래스 지정자 제외
- YYSTYPE: 포인터 크기에 맞는 자료형 선택과 정적 크기 검사

- **함수 사전 선언**: 정수형이 아닌 반환형의 함수는 호출 전에 프로토타입 선언
- 선언 처리: 함수 원형, 매개변수, 타입 연결 및 전방 참조 검사
- 스코프 처리: 함수 매개변수와 블록의 스코프 구분
- AST 생성: makeNode와 makeNodeList 사용

- 오류 처리: 어휘, 구문, 선언, 내부 오류 분류 및 선언 오류 번호 출력
- 테스트 실행: Makefile에 전체, 그룹별, 개별 테스트 명령 추가

1-4. 주요 구현 원리

- 심볼과 스코프: current_id와 prev로 식별자를 검색하고, current_level로 스코프를 관리하며 종료 시 이전 선언 범위 복원
- 타입 구성: A_TYPE의 element_type을 재귀적으로 연결해 포인터, 배열, 함수의 중첩 타입 표현
- AST 생성: Yacc의 $1, $2와 $$로 의미값을 전달하고, makeNode와 makeNodeList로 노드와 목록 생성

1-5. 구현 범위 및 제한사항

- 구현: 5장의 선언 처리, 스코프 관리, 타입 구성과 기본 의미 검사
- 제외: 초기화식과 저장 클래스 지정자

2. 검증

2-1. 테스트 구성

테스트는 구문 분석, AST 구조, 선언 처리, 오류 분류와 기준 구현 비교로 구성했습니다.

그룹	검증 목적
AST	정상 입력과 문법 오류 입력의 구문 분석 검증
ASTEXP	AST 노드 구조와 연결 검증
SEM	타입과 식별자 처리 함수 검증
SEMERR	선언 오류 검출과 오류 분류 검증
BASELINE	기준 구현과 입력 처리 및 AST 출력 비교

2-2. 테스트 결과

전체 테스트 결과는 다음과 같습니다.

그룹	통과	실패
AST	17	0
ASTEXP	7	0
SEM	17	0
SEMERR	16	0
BASELINE	125	0
합계	**182**	**0**

기능 테스트 **57건**과 기준 구현 비교 **125건**이 모두 통과했습니다. 빌드 경고가 없었으며, **기대 AST 7개**도 실제 출력과 일치했습니다.

2-3. AST 구조 검증

AST 기대 구조가 정의된 7개 사례의 입력 코드, 기대 AST와 검증 내용을 정리했습니다. 실행할 때마다 달라지는 주소값은 비교에서 제외했습니다.

ASTEXP 01. 연산자 우선순위

검증 목적	연산자 우선순위와 정수 상수 노드 검증

소스 / 명령	`tests/ast/cases/03_precedence.c`
	`tests/ast/expected/03_precedence.txt`
	`make test-astexp TEST=1`

입력 코드	`int main()`
	`{`
	`  return 1 + 2 * 3;`
	`}`

기대 AST	`N_PROGRAM`
	`└─ clink → A_ID "main" [ID_FUNC, level=0, specifier=S_NULL]`
	`  └─ type → T_FUNC`
	`  ├─ field → NULL`
	`  ├─ element_type → int`
	`  └─ expr → N_STMT_COMPOUND`
	`  ├─ llink → NULL`
	`  └─ rlink → N_STMT_LIST`
	`  ├─ llink → N_STMT_RETURN`
	`  │  └─ clink → N_EXP_ADD`
	`  │  ├─ llink → N_EXP_INT_CONST(1)`
	`  │  └─ rlink → N_EXP_MUL`
	`  │  ├─ llink → N_EXP_INT_CONST(2)`
	`  │  └─ rlink → N_EXP_INT_CONST(3)`
	`  └─ rlink → N_STMT_LIST_NIL`

검증 내용	• 덧셈 노드의 왼쪽 1, 오른쪽 곱셈 노드 2 * 3 연결 • 괄호 전용 노드 미생성 • main의 다음 선언과 지역 선언 목록 NULL • 의미 분석 전 노드 타입 NULL, 값 0
검증 결과	기대 AST 객체 그래프와 일치, 통과

ASTEXP 02. if-else 구조

검증 목적	조건식과 두 분기 노드의 연결 검증

소스 / 명령	`tests/ast/cases/07_if_else.c`
	`tests/ast/expected/07_if_else.txt`
	`make test-astexp TEST=2`

입력 코드	`int main()`
	`{`
	`  int x;`
	`  x = 1;`
	`  if (x > 0)`
	`  return 1;`
	`  else`
	`  return 0;`
	`}`

기대 AST	`N_PROGRAM`
	`└─ clink → A_ID "main" [ID_FUNC, level=0, specifier=S_NULL]`
	`  └─ type → T_FUNC`
	`  ├─ field → NULL`
	`  ├─ element_type → int`
	`  └─ expr → N_STMT_COMPOUND`
	`  ├─ llink → A_ID "x" [ID_VAR, level=1, specifier=S_NULL, type=int] [X_LOCAL]`
	`  └─ rlink → N_STMT_LIST`
	`  ├─ llink → N_STMT_EXPRESSION`
	`  │ └─ clink → N_EXP_ASSIGN`
	`  │ ├─ llink → N_EXP_IDENT → X_LOCAL`
	`  │ └─ rlink → N_EXP_INT_CONST(1)`
	`  └─ rlink → N_STMT_LIST`
	`  ├─ llink → N_STMT_IF_ELSE`
	`  │ ├─ llink → N_EXP_GTR`
	`  │ │ ├─ llink → N_EXP_IDENT → X_LOCAL`
	`  │ │ └─ rlink → N_EXP_INT_CONST(0)`
	`  │ ├─ clink → N_STMT_RETURN`
	`  │ │ └─ clink → N_EXP_INT_CONST(1)`
	`  │ └─ rlink → N_STMT_RETURN`
	`  │ └─ clink → N_EXP_INT_CONST(0)`
	`  └─ rlink → N_STMT_LIST_NIL`

검증 내용	• 조건식, 참 분기, 거짓 분기의 순서와 연결 • 단일 문장 분기의 불필요한 복합문 노드 미생성 • 대입문과 조건식의 x가 같은 지역 선언 참조
검증 결과	기대 AST 객체 그래프와 일치, 통과

ASTEXP 03. for 구조

검증 목적	for문의 세 수식과 반복 본문 연결 검증
소스 / 명령	tests/ast/cases/09_for.c tests/ast/expected/09_for.txt make test-astexp TEST=3

입력 코드	int main()
	{
	    int i;
	    int sum;
	    i = 0;
	    sum = 0;
	    for (i = 0; i < 10; i++)
	        sum = sum + i;
	    return sum;
	}

기대 AST	N_PROGRAM └─ clink → A_ID "main" [ID_FUNC, level=0, specifier=S_NULL] └─ type → T_FUNC ├─ field → NULL ├─ element_type → int └─ expr → N_STMT_COMPOUND ├─ llink → A_ID "i" [ID_VAR, level=1, type=int] │ └─ link → A_ID "sum" [ID_VAR, level=1, type=int] └─ rlink → N_STMT_LIST ├─ llink → N_STMT_EXPRESSION │ └─ clink → N_EXP_ASSIGN(N_EXP_IDENT(i), N_EXP_INT_CONST(0)) └─ rlink → N_STMT_LIST ├─ llink → N_STMT_EXPRESSION │ └─ clink → N_EXP_ASSIGN(N_EXP_IDENT(sum), N_EXP_INT_CONST(0)) └─ rlink → N_STMT_LIST ├─ llink → N_STMT_FOR │ ├─ llink → N_FOR_EXP │ │ ├─ llink → N_EXP_ASSIGN(N_EXP_IDENT(i), N_EXP_INT_CONST(0)) │ │ ├─ clink → N_EXP_LSS(N_EXP_IDENT(i), N_EXP_INT_CONST(10)) │ │ └─ rlink → N_EXP_POST_INC │ │ └─ clink → N_EXP_IDENT(i) │ └─ rlink → N_STMT_EXPRESSION │ └─ clink → N_EXP_ASSIGN │ ├─ llink → N_EXP_IDENT(sum) │ └─ rlink → N_EXP_ADD │ ├─ llink → N_EXP_IDENT(sum) │ └─ rlink → N_EXP_IDENT(i) └─ rlink → N_STMT_LIST ├─ llink → N_STMT_RETURN │ └─ clink → N_EXP_IDENT(sum) └─ rlink → N_STMT_LIST_NIL
검증 내용	• N_FOR_EXP의 초기식, 조건식, 증감식 순서와 연결 • 반복 본문의 수식문 처리 • 모든 i와 sum 참조가 각각 같은 지역 선언 참조 • N_STMT_FOR와 N_FOR_EXP의 노드 및 링크 구조 비교
검증 결과	기대 AST 객체 그래프와 일치, 통과

ASTEXP 04. 함수 원형

검증 목적	함수 원형과 전역 선언 목록 검증

소스 / 명령	`tests/ast/cases/24_function_prototype.c`
	`tests/ast/expected/24_function_prototype.txt`
	`make test-astexp TEST=4`

입력 코드	`int add(int a, int b);`
	`int main() {`
	`  return 0;`
	`}`

기대 AST	`N_PROGRAM`
	`└─ clink / global A_ID list`
	`  ├─ A_ID "add"  [ID_FUNC, level=0, specifier=S_NULL]`
	`  │  └─ type: T_FUNC`
	`  │  ├─ field / parameters`
	`  │  │  ├─ A_ID "a" [ID_PARM, level=1, type=int]`
	`  │  │  └─ next → A_ID "b" [ID_PARM, level=1, type=int]`
	`  │  ├─ element_type: int`
	`  │  └─ expr: NULL  (prototype: no function body)`
	`  └─ next → A_ID "main" [ID_FUNC, level=0, specifier=S_NULL]`
	`  └─ type: T_FUNC`
	`  ├─ field: NULL`
	`  ├─ element_type: int`
	`  └─ expr: N_STMT_COMPOUND`
	`  ├─ llink: NULL (no local declarations)`
	`  └─ rlink: N_STMT_LIST`
	`  ├─ llink: N_STMT_RETURN`
	`  │  └─ clink: N_EXP_INT_CONST`
	`  │  └─ clink: 0`
	`  └─ rlink: N_STMT_LIST_NIL`

검증 내용	• 전역 선언 목록의 add, main 순서와 종료 • add 매개변수의 전역 선언 목록 유입 방지 • add 본문 NULL, main 본문 복합문 • 문장 목록의 N_STMT_LIST_NIL 종료 • main()의 매개변수 목록 NULL
검증 결과	기대 AST 객체 그래프와 일치, 통과

ASTEXP 05. 함수 매개변수와 스코프

검증 목적	함수 매개변수 선언과 식별자 참조 검증
소스 / 명령	tests/ast/cases/25_function_parameters.c tests/ast/expected/25_function_parameters.txt make test-astexp TEST=5

입력 코드	int add(int a, int b) {
	    return a + b;
	}

기대 AST	N_PROGRAM └─ clink / global A_ID list └─ A_ID "add" [ID_FUNC, level=0, specifier=S_NULL] ├─ type: T_FUNC │ ├─ field / parameters │ │ ├─ A_ID "a" [ID_PARM, level=1, type=int] │ │ └─ next → A_ID "b" [ID_PARM, level=1, type=int] │ ├─ element_type: int │ └─ expr: N_STMT_COMPOUND │ ├─ llink: NULL (no local declarations) │ └─ rlink: N_STMT_LIST │ ├─ llink: N_STMT_RETURN │ │ └─ clink: N_EXP_ADD │ │ ├─ llink: N_EXP_IDENT │ │ │ └─ clink → A_ID "a" (same symbol as parameter a) │ │ └─ rlink: N_EXP_IDENT │ │ └─ clink → A_ID "b" (same symbol as parameter b) │ └─ rlink: N_STMT_LIST_NIL └─ next: NULL
검증 내용	• 반환식의 덧셈 노드와 두 식별자 피연산자 연결 • 식별자가 매개변수 a, b의 원래 선언 참조 • 매개변수의 전역 선언 목록 유입 방지 • 반환식과 N_STMT_RETURN의 clink 연결
검증 결과	기대 AST 객체 그래프와 일치, 통과

ASTEXP 06. 중첩 스코프의 이름 가림

검증 목적	중첩 블록의 이름 가림과 참조 복원 검증
소스 / 명령	tests/ast/cases/44_scope_shadowing.c tests/ast/expected/44_scope_shadowing.txt make test-astexp TEST=6

입력 코드	int x;
	int main() {
	    int x;
	    x = 1;
	    {
	        int x;
	        x = 2;
	    }
	    return x;
	}

기대 AST	N_PROGRAM └─ clink / global A_ID list ├─ A_ID "x" [ID_VAR, level=0, specifier=S_NULL, type=int] [X_GLOBAL] └─ next → A_ID "main" [ID_FUNC, level=0, specifier=S_NULL] └─ type: T_FUNC ├─ field: NULL ├─ element_type: int └─ expr: N_STMT_COMPOUND [outer function block] ├─ llink → A_ID "x" [ID_VAR, level=1, type=int] [X_OUTER] └─ rlink: N_STMT_LIST ├─ llink: N_STMT_EXPRESSION │ └─ clink: N_EXP_ASSIGN │ ├─ llink: N_EXP_IDENT → X_OUTER │ └─ rlink: N_EXP_INT_CONST(1) └─ rlink: N_STMT_LIST ├─ llink: N_STMT_COMPOUND [inner nested block] │ ├─ llink → A_ID "x" [ID_VAR, level=2, type=int] [X_INNER] │ └─ rlink: N_STMT_LIST │ ├─ llink: N_STMT_EXPRESSION │ │ └─ clink: N_EXP_ASSIGN │ │ ├─ llink: N_EXP_IDENT → X_INNER │ │ └─ rlink: N_EXP_INT_CONST(2) │ └─ rlink: N_STMT_LIST_NIL └─ rlink: N_STMT_LIST ├─ llink: N_STMT_RETURN │ └─ clink: N_EXP_IDENT → X_OUTER └─ rlink: N_STMT_LIST_NIL
검증 내용	• 전역, 함수, 내부 블록의 x를 레벨 0, 1, 2로 구분 • x = 1의 함수 지역 x 참조, x = 2의 내부 블록 x 참조 • 내부 블록 종료 후 return x의 함수 지역 x 참조 복원 • main 안에서 전역 x 참조 없음
검증 결과	기대 AST 객체 그래프와 일치, 통과

ASTEXP 07. 함수 선언자

검증 목적	빈 매개변수 목록을 가진 함수 선언자 검증
소스 / 명령	tests/ast/cases/59_function_declarator.c tests/ast/expected/59_function_declarator.txt make test-astexp TEST=7
입력 코드	int f();
기대 AST	N_PROGRAM └─ clink / global A_ID list └─ A_ID "f" [ID_FUNC, level=0, specifier=S_NULL] ├─ type: T_FUNC │ ├─ field / parameters: NULL (empty parameter list) │ ├─ element_type: int │ └─ expr: NULL (prototype / declaration only) └─ next: NULL
검증 내용	• f의 본문 없는 함수 선언 처리 • 반환 타입 int, 매개변수 목록과 본문 NULL • 불필요한 복합문, 반환문, 매개변수 및 전역 선언 미생성
검증 결과	기대 AST 객체 그래프와 일치, 통과

2-4. 오류 검증

SEMERR 16건 중 1~10번은 잘못된 선언을 거부하는지, 11~14번은 정상 선언을 허용하는지 확인하는 테스트입니다. 15~16번은 구문 오류와 어휘 오류가 각각 알맞게 분류되는지 검사합니다.

오류 분류	대표 검증 사례	검증 결과
LEXICAL_ERROR	SEMERR 16, 잘못된 문자	통과
SYNTAX_ERROR	SEMERR 15, 문법 오류	통과
DECLARATION_ERROR	SEMERR 01~10, 중복 선언, 미선언 식별자, 원형 충돌, 미완성 전방 참조 등	통과
INTERNAL_ERROR	메모리 할당 실패 등 내부 오류	구현 확인, 직접 시험 없음

선언 오류에는 강의노트의 오류 번호 11, 12, 13, 14, 21, 22, 23, 26, 32를 사용했습니다. 출력에는 [DECLARATION_ERROR:12]처럼 분류명과 번호를 함께 표시했습니다. 구문 오류와 어휘 오류는 각각 별도의 분류명으로 출력했습니다.

2-5. 테스트 케이스 상세

컴파일러 W4 테스트 케이스

식별자 해제와 상태 초기화는 sem_test_support.c의 공통 함수로 처리했습니다..

1. SEM, 의미 처리 단위 테스트

SEM 01. 타입 생성

의도 / 기준	• 타입 객체 생성과 기본 필드 초기화
소스 / 명령	`tests/sem/sem_type_table.c` `make test-sem TEST=1`

테스트 코드	`void SEM_01_MakeType(void) {`
	`  A_TYPE *t = makeType(T_POINTER);`
	`  assert(t != NULL);`
	`  assert(t->kind == T_POINTER);`
	`  assert(t->element_type == NULL);`
	`  free(t);`
	`}`

실행 결과 **종료 코드 0**	`PASS [SEM:01] SEM_01_MakeType  make type` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 02. 식별자 생성

의도 / 기준	• 식별자 이름 저장과 심볼 체인 연결
소스 / 명령	`tests/sem/sem_symbol_table.c` `make test-sem TEST=2`

테스트 코드	`void SEM_02_MakeIdentifier(void) {`
	`  A_ID *a = makeIdentifier("a");`
	`  A_ID *b = makeIdentifier("b");`
	`  assert(strcmp(a->name, "a") == 0);`
	`  assert(strcmp(b->name, "b") == 0);`
	`  assert(b->prev == a);`
	`  assert(current_id == b);`
	`  assert(a->prev == NULL);`
	`  assert(a->link == NULL);`
	`  semTestFreeIdentifierChain();`
	`}`

실행 결과 **종료 코드 0**	`PASS [SEM:02] SEM_02_MakeIdentifier  make identifier` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 03. 식별자 검색

의도 / 기준	• 식별자 검색과 스코프별 이름 가림
소스 / 명령	`tests/sem/sem_symbol_table.c` `make test-sem TEST=3`

테스트 코드	`void SEM_03_SearchIdentifier(void) {`
	`  current_id = NULL;`
	`  current_level = 0;`
	`  A_ID *global_x = makeIdentifier("x");`
	`  A_ID *global_y = makeIdentifier("y");`
	`  assert(searchIdentifier("x") == global_x);`
	`  assert(searchIdentifier("y") == global_y);`
	`  assert(searchIdentifier("z") == NULL);`
	`  current_level = 1;`
	`  A_ID *local_x = makeIdentifier("x");`
	`  assert(searchIdentifier("x") == local_x);`
	`  assert(searchIdentifierAtCurrentLevel("x", local_x->prev) == NULL);`
	`  A_ID *duplicate_x = makeIdentifier("x");`
	`  assert(searchIdentifierAtCurrentLevel("x", duplicate_x->prev) == local_x);`
	`  semTestFreeIdentifierChain();`
	`  current_level = 0;`
	`}`

실행 결과 **종료 코드 0**	`PASS [SEM:03] SEM_03_SearchIdentifier  search identifier` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 04. 스코프 진입·종료

의도 / 기준	• 스코프 진입, 종료와 검색 범위 복원
소스 / 명령	tests/sem/sem_scope.c make test-sem TEST=4

테스트 코드	void SEM_04_Scope(void) {
	  assert(current_level == 0);
	  assert(current_id == NULL);
	  A_ID *a = setDeclaratorKind(makeIdentifier("a"), ID_VAR);
	  enterScope();
	  assert(current_level == 1);
	  A_ID *b = setDeclaratorKind(makeIdentifier("b"), ID_VAR);
	  assert(current_id == b);
	  assert(searchIdentifier("a") == a);
	  assert(searchIdentifier("b") == b);
	  enterScope();
	  assert(current_level == 2);
	  A_ID *c = setDeclaratorKind(makeIdentifier("c"), ID_VAR);
	  assert(searchIdentifier("c") == c);
	  leaveScope();
	  assert(current_level == 1);
	  assert(current_id == b);
	  assert(searchIdentifier("c") == NULL);
	  assert(searchIdentifier("b") == b);
	  leaveScope();
	  assert(current_level == 0);
	  assert(current_id == a);
	  assert(searchIdentifier("b") == NULL);
	  assert(searchIdentifier("a") == a);
	  free(c->name);
	  free(c);
	  free(b->name);
	  free(b);
	  free(a->name);
	  free(a);
	  current_id = NULL;
	}

실행 결과 종료 코드 0	PASS [SEM:04] SEM_04_Scope scope SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 05. 선언자 목록 연결

의도 / 기준	• 선언자 목록 연결과 심볼 체인 보존
소스 / 명령	tests/sem/sem_symbol_table.c make test-sem TEST=5

테스트 코드	void SEM_05_LinkDeclaratorList(void) {
	  assert(current_id == NULL);
	  assert(current_level == 0);
	  A_ID *a = makeIdentifier("a");
	  A_ID *b = makeIdentifier("b");
	  A_ID *c = makeIdentifier("c");
	  A_ID *list = linkDeclaratorList(a, b);
	  list = linkDeclaratorList(list, c);
	  // 선언자 목록 연결 검증
	  assert(list == a);
	  assert(a->link == b);
	  assert(b->link == c);
	  assert(c->link == NULL);
	  // 심볼 테이블 연결은 변경되지 않아야 함
	  assert(current_id == c);
	  assert(c->prev == b);
	  assert(b->prev == a);
	  assert(a->prev == NULL);
	  // 빈 리스트 처리 검증
	  assert(linkDeclaratorList(NULL, NULL) == NULL);
	  assert(linkDeclaratorList(NULL, a) == a);
	  assert(linkDeclaratorList(a, NULL) == a);
	  semTestFreeIdentifierChain();
	}

실행 결과 종료 코드 0	PASS [SEM:05] SEM_05_LinkDeclaratorList link declarator list SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 06. 타입 체인 연결

의도 / 기준	• 포인터와 배열의 원소 타입 연결
소스 / 명령	`tests/sem/sem_type_table.c` `make test-sem TEST=6`

테스트 코드	`void SEM_06_SetTypeElementType(void) {`
	`  A_TYPE *int_type = makeType(T_ENUM);`
	`  A_TYPE *pointer1 = makeType(T_POINTER);`
	`  A_TYPE *pointer2 = makeType(T_POINTER);`
	`  // T_POINTER -> int_type`
	`  A_TYPE *result = setTypeElementType(pointer1, int_type);`
	`  assert(result == pointer1);`
	`  assert(pointer1->element_type == int_type);`
	`  // T_POINTER -> T_POINTER -> int_type`
	`  result = setTypeElementType(pointer2, pointer1);`
	`  assert(result == pointer2);`
	`  assert(pointer2->element_type == pointer1);`
	`  assert(pointer1->element_type == int_type);`
	`  // 기존 체인의 마지막에 다른 타입 연결`
	`  A_TYPE *array = makeType(T_ARRAY);`
	`  A_TYPE *outer = makeType(T_POINTER);`
	`  setTypeElementType(outer, array);`
	`  setTypeElementType(outer, pointer2);`
	`  assert(outer->element_type == array);`
	`  assert(array->element_type == pointer2);`
	`  assert(pointer2->element_type == pointer1);`
	`  // 만들어진 타입 체인의 메모리 해제`
	`  free(outer);`
	`  free(array);`
	`  free(pointer2);`
	`  free(pointer1);`
	`  free(int_type);`
	`}`

실행 결과 **종료 코드 0**	`PASS [SEM:06] SEM_06_SetTypeElementType  set type element type` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 07. 선언자 타입 연결

의도 / 기준	• 선언자의 타입 설정과 중첩 타입 연결
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=7

테스트 코드	void SEM_07_SetDeclaratorElementType(void) {
	  assert(current_id == NULL);
	  assert(current_level == 0);
	  A_ID *p = makeIdentifier("p");
	  A_TYPE *pointer1 = makeType(T_POINTER);
	  A_TYPE *pointer2 = makeType(T_POINTER);
	  A_TYPE *int_type = makeType(T_ENUM);
	  // 1. 처음 타입 연결
	  A_ID *result = setDeclaratorElementType(p, pointer1);
	  assert(result == p);
	  assert(p->type == pointer1);
	  assert(pointer1->element_type == NULL);
	  // 2. 기존 타입 체인의 마지막에 연결
	  setDeclaratorElementType(p, pointer2);
	  setDeclaratorElementType(p, int_type);
	  assert(p->type == pointer1);
	  assert(pointer1->element_type == pointer2);
	  assert(pointer2->element_type == int_type);
	  assert(int_type->element_type == NULL);
	  // 3. NULL 타입 처리
	  result = setDeclaratorElementType(p, NULL);
	  assert(result == p);
	  assert(p->type == pointer1);
	  assert(pointer2->element_type == int_type);
	  // 메모리 해제
	  free(pointer1);
	  free(pointer2);
	  free(int_type);
	  free(p->name);
	  free(p);
	  current_id = NULL;
	}

실행 결과 종료 코드 0	PASS [SEM:07] SEM_07_SetDeclaratorElementType set declarator element type SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 08. 배열 크기 수식

의도 / 기준	• 배열 크기 수식 연결과 생략 처리
소스 / 명령	`tests/sem/sem_type_table.c` `make test-sem TEST=8`

테스트 코드	`void SEM_08_SetTypeExpr(void) {`
	`  A_TYPE *array = makeType(T_ARRAY);`
	`  A_NODE size_expr = {0};`
	`  size_expr.name = N_EXP_INT_CONST;`
	`  size_expr.clink = (A_NODE *)(long)10;`
	`  // 1. 배열 크기 수식 연결`
	`  A_TYPE *result = setTypeExpr(array, &size_expr);`
	`  assert(result == array);`
	`  assert(array->kind == T_ARRAY);`
	`  assert(array->expr == &size_expr);`
	`  // 2. 원소 타입 연결`
	`  A_TYPE *int_type = makeType(T_ENUM);`
	`  setTypeElementType(array, int_type);`
	`  assert(array->element_type == int_type);`
	`  assert(array->expr == &size_expr);`
	`  // 3. 크기가 생략된 배열`
	`  A_TYPE *unsized_array = makeType(T_ARRAY);`
	`  result = setTypeExpr(unsized_array, NULL);`
	`  assert(result == unsized_array);`
	`  assert(unsized_array->expr == NULL);`
	`  // 메모리 해제`
	`  free(unsized_array);`
	`  free(int_type);`
	`  free(array);`
	`}`

실행 결과 **종료 코드 0**	`PASS [SEM:08] SEM_08_SetTypeExpr  set type expression` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 09. 필드·매개변수 목록

의도 / 기준	• 구조체 필드와 함수 매개변수 목록 연결
소스 / 명령	tests/sem/sem_type_table.c make test-sem TEST=9

테스트 코드	void SEM_09_SetTypeField(void) {
	  assert(current_id == NULL);
	  assert(current_level == 0);
	  // 1. 구조체 타입 생성
	  A_TYPE *struct_type = makeType(T_STRUCT);
	  A_ID *x = makeIdentifier("x");
	  A_ID *y = makeIdentifier("y");
	  A_ID *fields = linkDeclaratorList(x, y);
	  A_TYPE *result = setTypeField(struct_type, fields);
	  assert(result == struct_type);
	  assert(struct_type->kind == T_STRUCT);
	  assert(struct_type->field == x);
	  assert(struct_type->field->link == y);
	  assert(y->link == NULL);
	  // 2. 함수 타입의 파라미터 목록
	  A_TYPE *func_type = makeType(T_FUNC);
	  A_ID *param = makeIdentifier("param");
	  result = setTypeField(func_type, param);
	  assert(result == func_type);
	  assert(func_type->field == param);
	  // 3. 빈 파라미터 목록
	  A_TYPE *empty_func = makeType(T_FUNC);
	  result = setTypeField(empty_func, NULL);
	  assert(result == empty_func);
	  assert(empty_func->field == NULL);
	  // 4. 기존 타입 체인에는 영향이 없는지 확인
	  assert(struct_type->element_type == NULL);
	  assert(func_type->element_type == NULL);
	  // 메모리 해제
	  free(empty_func);
	  free(func_type);
	  free(struct_type);
	  free(param->name);
	  free(param);
	  free(y->name);
	  free(y);
	  free(x->name);
	  free(x);
	  current_id = NULL;
	  current_level = 0;
	}

실행 결과 종료 코드 0	PASS [SEM:09] SEM_09_SetTypeField set type field SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 10. 선언 지정자 생성

의도 / 기준	• 타입을 담는 선언 지정자 생성
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=10

테스트 코드	void SEM_10_MakeSpecifier(void) {
	  A_TYPE *int_type = makeType(T_ENUM);
	  // 1. 타입만 있는 선언: int a;
	  A_SPECIFIER *s1 = makeSpecifier(int_type);
	  assert(s1 != NULL);
	  assert(s1->type == int_type);
	  assert(s1->stor == S_NULL);
	  assert(s1->line == line_no);
	  // 2. 타입이 생략된 선언 지정자
	  A_SPECIFIER *s2 = makeSpecifier(NULL);
	  assert(s2 != NULL);
	  assert(s2->type == NULL);
	  assert(s2->stor == S_NULL);
	  // 3. 서로 독립적인 객체인지 확인
	  assert(s1 != s2);
	  free(s1);
	  free(s2);
	  free(int_type);
	}

실행 결과 종료 코드 0	PASS [SEM:10] SEM_10_MakeSpecifier make specifier SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 11. 선언 지정자 독립성

의도 / 기준	• 선언 지정자 객체와 타입 정보의 독립성
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=11

테스트 코드	void SEM_11_SpecifierIndependence(void) {
	  A_TYPE *first_type = makeType(T_ENUM);
	  A_TYPE *second_type = makeType(T_POINTER);
	  A_SPECIFIER *first = makeSpecifier(first_type);
	  A_SPECIFIER *second = makeSpecifier(second_type);
	  assert(first != second);
	  assert(first->type == first_type);
	  assert(second->type == second_type);
	  assert(first->stor == S_NULL);
	  assert(second->stor == S_NULL);
	  free(first);
	  free(second);
	  free(first_type);
	  free(second_type);
	}

실행 결과 종료 코드 0	PASS [SEM:11] SEM_11_SpecifierIndependence specifier independence SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 12. 기본 타입 설정

의도 / 기준	• 생략한 기본 타입 설정과 기존 타입 보존
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=12

테스트 코드	void SEM_12_SetDefaultSpecifier(void) {
	  // 아직 initialize()를 구현하지 않았으므로
	  // 테스트에서 기본 int 타입을 임시 생성
	  int_type = makeType(T_ENUM);
	  // 1. 타입이 없는 경우 기본 int 적용
	  A_SPECIFIER *s1 = makeSpecifier(NULL);
	  setDefaultSpecifier(s1);
	  assert(s1->type == int_type);
	  assert(s1->stor == S_NULL);
	  // 2. 타입이 이미 지정된 경우 유지
	  A_TYPE *char_type = makeType(T_ENUM);
	  A_SPECIFIER *s2 = makeSpecifier(char_type);
	  setDefaultSpecifier(s2);
	  assert(s2->type == char_type);
	  assert(s2->type != int_type);
	  // 3. 타입이 없는 다른 선언 지정자
	  A_SPECIFIER *s3 = makeSpecifier(NULL);
	  setDefaultSpecifier(s3);
	  assert(s3->type == int_type);
	  assert(s3->stor == S_NULL);
	  // 메모리 해제
	  free(s1);
	  free(s2);
	  free(s3);
	  free(char_type);
	  free(int_type);
	  int_type = NULL;
	}

실행 결과 종료 코드 0	PASS [SEM:12] SEM_12_SetDefaultSpecifier set default specifier SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 13. 의미 분석 환경 초기화

의도 / 기준	• 기본 타입과 내장 함수의 식별자 등록
소스 / 명령	tests/sem/sem_symbol_table.c make test-sem TEST=13

테스트 코드	void SEM_13_Initialize(void) {
	  assert(current_id == NULL);
	  assert(current_level == 0);
	  assert(int_type == NULL);
	  initialize();
	  // 기본 타입 생성 확인
	  assert(int_type != NULL);
	  assert(float_type != NULL);
	  assert(char_type != NULL);
	  assert(void_type != NULL);
	  assert(string_type != NULL);
	  assert(int_type->kind == T_ENUM);
	  assert(float_type->kind == T_ENUM);
	  assert(char_type->kind == T_ENUM);
	  assert(void_type->kind == T_VOID);
	  assert(string_type->kind == T_POINTER);
	  assert(string_type->element_type == char_type);
	  // 기본 타입 명칭 검색
	  A_ID *id = searchIdentifier("int");
	  assert(id != NULL);
	  assert(id->kind == ID_TYPE);
	  assert(id->type == int_type);
	  id = searchIdentifier("float");
	  assert(id != NULL);
	  assert(id->type == float_type);
	  id = searchIdentifier("char");
	  assert(id != NULL);
	  assert(id->type == char_type);
	  id = searchIdentifier("void");
	  assert(id != NULL);
	  assert(id->type == void_type);
	  assert(searchIdentifier("double") == NULL);
	  // 기본 함수 검색
	  id = searchIdentifier("printf");
	  assert(id != NULL);
	  assert(id->kind == ID_FUNC);
	  assert(id->type->kind == T_FUNC);
	  assert(id->type->element_type == void_type);
	  assert(id->type->field != NULL);
	  assert(id->type->field->type == string_type);
	  assert(id->type->field->link != NULL);
	  assert(id->type->field->link->type == NULL);
	  id = searchIdentifier("scanf");
	  assert(id != NULL);
	  assert(id->kind == ID_FUNC);
	  assert(id->type->element_type == void_type);
	  id = searchIdentifier("malloc");
	  assert(id != NULL);
	  assert(id->kind == ID_FUNC);
	  assert(id->type->element_type == string_type);
	  assert(id->type->field != NULL);
	  assert(id->type->field->type == int_type);
	  // 모든 기본 명칭은 전역 레벨에 등록됨
	  assert(id->level == 0);
	  semTestResetInitializedState();
	}

실행 결과 종료 코드 0	PASS [SEM:13] SEM_13_Initialize initialize SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 14. 선언 목록 지정자 적용

의도 / 기준	• 일반 선언과 포인터 선언의 타입 및 스코프 처리

소스 / 명령

```
tests/sem/sem_declarator.c
make test-sem TEST=14
```

테스트 코드

```
void SEM_14_SetDeclaratorListSpecifier(void) {
  assert(current_id == NULL);
  assert(current_level == 0);
  assert(int_type == NULL);
  initialize();
  // 1. int a, *p;
  A_ID *a = makeIdentifier("a");
  A_ID *p = makeIdentifier("p");
  setDeclaratorElementType(p, makeType(T_POINTER));
  A_ID *list = linkDeclaratorList(a, p);
  A_SPECIFIER *specifier = makeSpecifier(int_type);
  A_ID *result = setDeclaratorListSpecifier(list, specifier);
  assert(result == a);
  assert(a->kind == ID_VAR);
  assert(a->type == int_type);
  assert(a->specifier == S_NULL);
  assert(p->kind == ID_VAR);
  assert(p->type->kind == T_POINTER);
  assert(p->type->element_type == int_type);
  assert(p->specifier == S_NULL);
  // 2. int value;
  A_ID *value = makeIdentifier("value");
  A_SPECIFIER *default_spec = makeSpecifier(int_type);
  setDeclaratorListSpecifier(value, default_spec);
  assert(value->kind == ID_VAR);
  assert(value->type == int_type);
  assert(value->specifier == S_NULL);
  // 3. 다른 스코프에서 동일 명칭 선언
  enterScope();
  A_ID *local_a = makeIdentifier("a");
  A_SPECIFIER *local_spec = makeSpecifier(int_type);
  setDeclaratorListSpecifier(local_a, local_spec);
  assert(local_a->kind == ID_VAR);
  assert(local_a->type == int_type);
  assert(local_a->level == 1);
  assert(searchIdentifier("a") == local_a);
  leaveScope();
  assert(searchIdentifier("a") == a);
  // 4. 테스트에서 생성한 객체 해제
  free(specifier);
  free(default_spec);
  free(local_spec);
  free(local_a->name);
  free(local_a);
  // 기본 타입은 공유되므로 여기서 해제하지 않음
  free(p->type);
  semTestResetInitializedState();
}
```

실행 결과
종료 코드 0

```
PASS [SEM:14] SEM_14_SetDeclaratorListSpecifier       set declarator list specifier
SUMMARY [SEM] 1 passed, 0 failed, 1 total
```

SEM 15. 함수 선언 지정자

의도 / 기준	• 함수 원형과 정의의 반환 타입 및 매개변수 연결
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=15

테스트 코드	void SEM_15_SetFunctionDeclaratorSpecifier(void) {
	  assert(current_id == NULL);
	  assert(current_level == 0);
	  assert(int_type == NULL);
	  /* 테스트용 기본 타입 */
	  int_type = makeType(T_ENUM);
	  /* 1. int foo(); 프로토타입 등록 */
	  A_ID *prototype = makeIdentifier("foo");
	  setDeclaratorElementType(prototype, makeType(T_FUNC));
	  A_SPECIFIER *proto_spec = makeSpecifier(int_type);
	  setDeclaratorListSpecifier(prototype, proto_spec);
	  assert(prototype->kind == ID_FUNC);
	  assert(prototype->type->element_type == int_type);
	  /* 2. int foo() { ... } 함수 정의 분석 */
	  A_ID *definition = makeIdentifier("foo");
	  setDeclaratorElementType(definition, makeType(T_FUNC));
	  A_SPECIFIER *def_spec = makeSpecifier(int_type);
	  A_ID *result = setFunctionDeclaratorSpecifier(definition, def_spec);
	  assert(result == definition);
	  assert(definition->kind == ID_FUNC);
	  assert(definition->type->kind == T_FUNC);
	  assert(definition->type->element_type == int_type);
	  /* 3. int bar(int x) { ... } */
	  A_ID *bar = makeIdentifier("bar");
	  A_TYPE *bar_type = makeType(T_FUNC);
	  setDeclaratorElementType(bar, bar_type);
	  A_ID *param = makeIdentifier("x");
	  param->kind = ID_PARM;
	  param->type = int_type;
	  setTypeField(bar_type, param);
	  A_SPECIFIER *bar_spec = makeSpecifier(int_type);
	  result = setFunctionDeclaratorSpecifier(bar, bar_spec);
	  assert(result == bar);
	  assert(bar->kind == ID_FUNC);
	  assert(bar->type->field == param);
	  assert(bar->type->element_type == int_type);
	  assert(searchIdentifier("x") == param);
	  /* 테스트에서 생성한 객체 해제 */
	  free(proto_spec);
	  free(def_spec);
	  free(bar_spec);
	  semTestFreeIdentifierChain();
	  free(int_type);
	  int_type = NULL;
	}

실행 결과 종료 코드 0	PASS [SEM:15] SEM_15_SetFunctionDeclaratorSpecifier set function declarator specifier SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 16. 함수 본문 연결

의도 / 기준	• 함수 본문 AST 연결과 함수 타입 보존
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=16

테스트 코드	void SEM_16_SetFunctionDeclaratorBody(void) {
	  assert(current_id == NULL);
	  assert(current_level == 0);
	  A_ID *fun = makeIdentifier("fun");
	  A_TYPE *func_type = makeType(T_FUNC);
	  setDeclaratorElementType(fun, func_type);
	  // 함수 정의라고 가정
	  fun->kind = ID_FUNC;
	  // { return ...; } 에 해당하는 AST 루트
	  A_NODE *body = calloc(1, sizeof(A_NODE));
	  assert(body != NULL);
	  body->name = N_STMT_COMPOUND;
	  // 1. 몸체 연결
	  A_ID *result = setFunctionDeclaratorBody(fun, body);
	  assert(result == fun);
	  assert(fun->type == func_type);
	  assert(func_type->expr == body);
	  // 2. AST가 변경되지 않았는지 확인
	  assert(func_type->expr->name == N_STMT_COMPOUND);
	  // 3. 함수 타입 유지 확인
	  assert(fun->kind == ID_FUNC);
	  assert(fun->type->kind == T_FUNC);
	  // 메모리 해제
	  free(body);
	  free(func_type);
	  free(fun->name);
	  free(fun);
	  current_id = NULL;
	}

실행 결과 종료 코드 0	PASS [SEM:16] SEM_16_SetFunctionDeclaratorBody set function declarator body SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 17. 매개변수 지정자

의도 / 기준	• 매개변수 타입, 익명 void 오류와 스코프 종료 처리

소스 / 명령

```
tests/sem/sem_declarator.c
make test-sem TEST=17
```

테스트 코드

```
void SEM_17_SetParameterDeclaratorSpecifier(void) {
  assert(current_id == NULL);
  assert(current_level == 0);
  assert(int_type == NULL);
  initialize();
  enterScope();
  // 1. int a
  A_ID *a = makeIdentifier("a");
  A_SPECIFIER *s1 = makeSpecifier(int_type);
  A_ID *result = setParameterDeclaratorSpecifier(a, s1);
  assert(result == a);
  assert(a->kind == ID_PARM);
  assert(a->type == int_type);
  assert(a->level == 1);
  // 2. int *b
  A_ID *b = makeIdentifier("b");
  setDeclaratorElementType(b, makeType(T_POINTER));
  A_SPECIFIER *s2 = makeSpecifier(int_type);
  result = setParameterDeclaratorSpecifier(b, s2);
  assert(result == b);
  assert(b->kind == ID_PARM);
  assert(b->type->kind == T_POINTER);
  assert(b->type->element_type == int_type);
  // 3. 이름 없는 void 파라미터
  A_ID *anonymous = makeIdentifier("");
  A_SPECIFIER *s3 = makeSpecifier(void_type);
  int previous_errors = syntax_err;
  result = setParameterDeclaratorSpecifier(anonymous, s3);
  assert(syntax_err == previous_errors + 1);
  assert(result == anonymous);
  assert(anonymous->kind == ID_PARM);
  assert(anonymous->type == void_type);
  // 4. 이름 검색
  assert(searchIdentifier("a") == a);
  assert(searchIdentifier("b") == b);
  // 5. 파라미터 목록 연결
  A_ID *params = linkDeclaratorList(a, b);
  assert(params == a);
  assert(a->link == b);
  // 스코프 종료
  leaveScope();
  assert(current_level == 0);
  assert(searchIdentifier("a") == NULL);
  assert(searchIdentifier("b") == NULL);
  // 테스트 객체 해제
  free(s1);
  free(s2);
  free(s3);
  free(b->type);
  free(anonymous->name);
  free(anonymous);
  free(b->name);
  free(b);
  free(a->name);
  free(a);
  semTestResetInitializedState();
}
```

실행 결과
종료 코드 0

```
line 1: syntax error: illegal type specifier in formal parameter at end
PASS [SEM:17] SEM_17_SetParameterDeclaratorSpecifier   set parameter declarator
specifier
SUMMARY [SEM] 1 passed, 0 failed, 1 total
```

2. AST, 파서 통합 테스트

AST 01. 함수 정의와 호출

의도 / 기준	• 함수 정의, 두 인자의 함수 호출과 반환값 대입 수용
소스 / 명령	tests/ast/valid/02_functions.c make test-ast TEST=1

입력 코드	int add(int left, int right)
	{
	    return left + right;
	}
	int main()
	{
	    int result;
	    result = add(2, 3);
	    return result;
	}

실행 결과	통과, 정상 입력 수용

AST 02. 제어 흐름 통합

의도 / 기준	• while, do-while, for, if-else, switch, break, continue 수용
소스 / 명령	tests/ast/valid/03_control_flow.c make test-ast TEST=2

입력 코드	int main()
	{
	    int i;
	    int sum;
	    i = 0;
	    sum = 0;
	    while (i < 3) {
	        sum = sum + i;
	        i++;
	    }
	    do {
	        sum = sum - 1;
	    } while (sum > 10);
	    for (i = 0; i < 5; i++) {
	        if (i == 2)
	            continue;
	        else
	            sum = sum + i;
	    }
	    switch (sum) {
	    case 0:
	        sum = 1;
	        break;
	    default:
	        break;
	    }
	    return sum;
	}

실행 결과	통과, 정상 입력 수용

AST 03. 재귀 함수

의도 / 기준	• factorial의 재귀 호출과 중첩 수식 수용
소스 / 명령	tests/ast/valid/08_recursion.c make test-ast TEST=3

입력 코드	int factorial(int value)
	{
	    if (value <= 1)
	        return 1;
	    return value * factorial(value - 1);
	}
	int main()
	{
	    return factorial(5);
	}

실행 결과	통과, 정상 입력 수용

AST 04. 공용체와 멤버 접근

의도 / 기준	• 공용체 정의, 전역 및 지역 변수, 멤버 대입과 참조 수용
소스 / 명령	tests/ast/valid/10_union.c make test-ast TEST=4

입력 코드	union Number {
	    int integer_value;
	    float float_value;
	} number;
	int main()
	{
	    union Number local;
	    local.integer_value = 42;
	    number.float_value = 1.5;
	    return local.integer_value;
	}

실행 결과	통과, 정상 입력 수용

AST 05. 포인터 연산과 매개변수

의도 / 기준	• 포인터 매개변수, 역참조 대입과 주소 전달 수용
소스 / 명령	`tests/ast/valid/11_pointers.c` `make test-ast TEST=5`

입력 코드	`void swap(int *left, int *right)`
	`{`
	`  int temporary;`
	`  temporary = *left;`
	`  *left = *right;`
	`  *right = temporary;`
	`  return;`
	`}`
	`int main()`
	`{`
	`  int first;`
	`  int second;`
	`  first = 10;`
	`  second = 20;`
	`  swap(&first, &second);`
	`  return first;`
	`}`

실행 결과	통과, 정상 입력 수용

AST 06. 중첩 조건문

의도 / 기준	• 중첩 if-else와 여러 인자를 전달하는 함수 호출 수용
소스 / 명령	`tests/ast/valid/12_nested_conditionals.c` `make test-ast TEST=6`

입력 코드	`int choose(int first, int second, int enabled)`
	`{`
	`  if (enabled)`
	`  if (first > second)`
	`  return first;`
	`  else`
	`  return second;`
	`  else`
	`  return 0;`
	`}`
	`int main()`
	`{`
	`  return choose(4, 7, 1);`
	`}`

실행 결과	통과, 정상 입력 수용

AST 07. 형 변환과 sizeof

의도 / 기준	• 형 변환과 sizeof 수식을 포함한 입력 수용
소스 / 명령	tests/ast/valid/14_cast_and_sizeof.c make test-ast TEST=7

입력 코드	int main()
	{
	    float value;
	    int result;
	    value = 4.5;
	    result = (int)value + sizeof(value);
	    return result;
	}

실행 결과	통과, 정상 입력 수용

AST 08. 세미콜론 누락

의도 / 기준	• 변수 선언 뒤 세미콜론 누락 검출
소스 / 명령	tests/ast/invalid/01_missing_semicolon.c make test-ast TEST=8

입력 코드	int main()
	{
	    int value
	    value = 1;
	    return value;
	}

실행 결과	통과, 문법 오류 입력 거부

AST 09. 불완전한 수식

의도 / 기준	• 덧셈 연산자의 오른쪽 피연산자 누락 검출
소스 / 명령	tests/ast/invalid/02_incomplete_expression.c make test-ast TEST=9

입력 코드	int main()
	{
	    return 1 + ;
	}

실행 결과	통과, 문법 오류 입력 거부

AST 10. 블록 닫기 누락

의도 / 기준	• 함수 본문의 닫는 중괄호 누락 검출
소스 / 명령	`tests/ast/invalid/03_unclosed_block.c` `make test-ast TEST=10`

입력 코드	`int main()`
	`{`
	`    int value;`
	`    value = 1;`
	`    return value;`

실행 결과	통과, 문법 오류 입력 거부

AST 11. 문장 뒤 선언

의도 / 기준	• 문장 뒤 선언으로 인한 선언부와 문장부 순서 위반 검출
소스 / 명령	`tests/ast/invalid/04_declaration_after_statement.c` `make test-ast TEST=11`

입력 코드	`int main()`
	`{`
	`    int first;`
	`    first = 1;`
	`    int second;`
	`    return first;`
	`}`

실행 결과	통과, 문법 오류 입력 거부

AST 12. 인자 목록 끝 쉼표

의도 / 기준	• 함수 호출의 쉼표 뒤 인자 누락 검출
소스 / 명령	`tests/ast/invalid/05_trailing_argument_comma.c` `make test-ast TEST=12`

입력 코드	`int add(int left, int right)`
	`{`
	`    return left + right;`
	`}`
	`int main()`
	`{`
	`    return add(1, );`
	`}`

실행 결과	통과, 문법 오류 입력 거부

AST 13. if 조건 괄호 누락

의도 / 기준	• if 조건을 감싸는 괄호 누락 검출
소스 / 명령	`tests/ast/invalid/06_if_without_parentheses.c` `make test-ast TEST=13`

입력 코드	`int main()`
	`{`
	`  int value;`
	`  value = 1;`
	`  if value`
	`  return 1;`
	`  return 0;`
	`}`

실행 결과	통과, 문법 오류 입력 거부

AST 14. 수식 괄호 닫기 누락

의도 / 기준	• 수식의 닫는 소괄호 누락 검출
소스 / 명령	`tests/ast/invalid/07_unclosed_parenthesis.c` `make test-ast TEST=14`

입력 코드	`int main()`
	`{`
	`  int value;`
	`  value = (1 + 2;`
	`  return value;`
	`}`

실행 결과	통과, 문법 오류 입력 거부

AST 15. 배열 선언자 오류

의도 / 기준	• 배열 선언자의 닫는 대괄호 누락 검출
소스 / 명령	`tests/ast/invalid/08_bad_array_declarator.c` `make test-ast TEST=15`

입력 코드	`int values[10;`
	`int main()`
	`{`
	`  return 0;`
	`}`

실행 결과	통과, 문법 오류 입력 거부

AST 16. 매개변수 쉼표 누락

의도 / 기준	• 함수 매개변수 사이 쉼표 누락 검출
소스 / 명령	`tests/ast/invalid/09_missing_parameter_comma.c` `make test-ast TEST=16`

입력 코드	`int add(int left int right)`
	`{`
	`    return left + right;`
	`}`

실행 결과	통과, 문법 오류 입력 거부

AST 17. if 없는 else

의도 / 기준	• 대응하는 if가 없는 else 검출
소스 / 명령	`tests/ast/invalid/10_else_without_if.c` `make test-ast TEST=17`

입력 코드	`int main()`
	`{`
	`    else`
	`        return 1;`
	`    return 0;`
	`}`

실행 결과	통과, 문법 오류 입력 거부

3. 개발 과정

개발 중 발생한 주요 오류와 처리 방법은 다음과 같습니다. 개별 코드와 실행 출력은 아래의 오류 및 해결 기록에 옮겼습니다.

개발 당시 문제	현재 코드의 대응	확인 근거
YYSTYPE 크기 불일치	parse_value.h의 공통 의미값 선언과 정적 크기 검사	w4/c.l, c.y, parse_value.h와 빌드 결과
타입과 선언자 구성 오류	이름 없는 식별자 생성과 타입 연결, 함수 선언자의 중복 및 호환성 검사	semantic.c, SEMERR 결과
함수 원형과 스코프 누출	매개변수 파싱 중 레벨 분리, 종료 후 current_id 복원	c.y, SEMERR 11~14 정상 입력 통과
AST 생성과 출력 오류	makeNode 계열로 AST 생성, 선언 목록 연결과 문자 코드 처리	semantic.c, c.l과 ASTEXP 7건 통과 결과
Lex/Yacc 규칙과 진단 오류	문법 규칙과 토큰 처리, 오류 분류 및 빌드 절차 정리	AST 17건, SEMERR 16건 통과 및 빌드 경고 없음

3-1. 오류 및 해결 기록

아래는 개발 당시 오류와 수정안을 정리한 기록입니다. 당시의 미해결 표기는 최종 테스트 결과와 구별했습니다.

콘솔 로그는 원문입니다. 해결 기록이 없는 항목은 수정안입니다.

01. 초기 빌드: 미구현 함수와 링크 오류

문제 / 원인	함수 구현 누락과 makeType·int_type의 정의를 찾지 못한 빌드 오류입니다.
문제 코드 실행 출력	In file included from C:\Users\egod1\proj\compiler\w4\test.c:3: C:\Users\egod1\proj\compiler\w4\test.c: In function 'main': C:\Users\egod1\proj\compiler\w4\test.c:11:18: warning: comparison between pointer and integer 11 | assert(t->kind != NULL); | ^~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c: In function 'makeNode': C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:28: warning: unused parameter 'name' [-Wunused-paramete] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ~~~~~~~~~~^~~~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:42: warning: unused parameter 'left' [-Wunused-paramete] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ~~~~~~~~^~~~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:56: warning: unused parameter 'center' [-Wunused-parameter] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ~~~~~~~~^~~~~~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:72: warning: unused parameter 'right' [-Wunused-parameter] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ~~~~~~~~^~~~~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:80: warning: control reaches end of non-void function [-Wreturn-type] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ^ C:/msys64/mingw64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\cc6VZ5qo.o:test.c:(.text+0x13): undefined reference to `makeType' collect2.exe: error: ld returned 1 exit status PS C:\Users\egod1\proj\compiler\w4> PS C:\Users\egod1\proj\compiler\w4> ./run C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\ccuBiCXf.o:test.c:(.rdata$.refptr.int_type[.refptr.int_type]+0x0): undefined reference to `int_type' collect2.exe: error: ld returned 1 exit status
해결 / 상태	함수 정의·링크 대상과 int_type의 정의·초기화를 확인할 항목입니다.

02. Specifier 처리: NULL 타입 오류를 GDB로 추적

문제 / 원인	Specifier 처리 중 NULL 타입이 전달된 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> ./run makeType test passed makeIdentifier test passed searchIdentifier test passed testScope passed testLinkDeclaratorList passed testSetTypeElementType passed testSetDeclaratorElementType passed testSetTypeExpr passed testSetTypeField passed Semantic error: NULL type in setTypeField PS C:\Users\egod1\proj\compiler\w4> gdb ./test.exe GNU gdb (GDB) 17.2 Copyright (C) 2025 Free Software Foundation, Inc. License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html> This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law. Type "show copying" and "show warranty" for details. This GDB was configured as "x86_64-w64-mingw32". Type "show configuration" for configuration details. For bug reporting instructions, please see: <https://www.gnu.org/software/gdb/bugs/>. Find the GDB manual and other documentation resources online at: <http://www.gnu.org/software/gdb/documentation/>. For help, type "help". Type "apropos word" to search for commands related to "word"... Reading symbols from ./test.exe... (gdb) break semanticError Breakpoint 1 at 0x14000342e: file C:\Users\egod1\proj\compiler\w4\semantic.c, line 111. (gdb) run Starting program: C:\Users\egod1\proj\compiler\w4\test.exe [New Thread 13820.0x657c] makeType test passed makeIdentifier test passed searchIdentifier test passed testScope passed testLinkDeclaratorList passed testSetTypeElementType passed testSetDeclaratorElementType passed testSetTypeExpr passed testSetTypeField passed Thread 1 hit Breakpoint 1, semanticError (message=0x7ff767b16a3d "NULL type in setTypeField") at C:\Users\egod1\proj\compiler\w4\semantic.c:111 111 fprintf(stderr, "Semantic error: %s\n", message); (gdb) bt #0 semanticError (message=0x7ff767b16a3d "NULL type in setTypeField") at C:\Users\egod1\proj\compiler\w4\semantic.c:111 #1 0x00007ff767b1363e in makeSpecifier (type=0x0, stor=S_TYPEDEF) at C:\Users\egod1\proj\compiler\w4\semantic.c:186 #2 0x00007ff767b12b99 in testMakeSpecifier () at C:\Users\egod1\proj\compiler\w4\test.c:360 #3 0x00007ff767b130df in main () at C:\Users\egod1\proj\compiler\w4\test.c:451 (gdb) quit A debugging session is active. Inferior 1 [process 13820] will be killed.
해결 / 상태	GDB의 중단점과 호출 스택을 확인하여 해결한 오류입니다.

03. 이름 없는 식별자 객체 미구현

문제 / 원인	이름 없는 식별자 객체가 구현되지 않은 상태입니다.
문제 코드 실행 출력	이름이 없는 식별자 객체는 구현 X -> 강의 자료에 나와있긴 한데, 정확한 구현이 없어서
해결 / 상태	이름 없는 식별자를 생성하는 기능의 구현이 필요한 상태입니다.

04. Lex 오타: yytest와 잘못된 대입 연산자

문제 / 원인	yytext의 이름과 대입 연산자를 잘못 작성한 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c -o parser.exe c.y: In function 'yyparse': c.y:308:52: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] 308	$$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL); | ^ c.y:308:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] 308 | $$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL); 	^ c.y:310:22: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] 310	A_NODE *node = (A_NODE*)$$; 	^ c.y:311:55: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] 311	fprintf(stderr, "[AST] N_EXP_INT_CONST: %ld\n", (long)node->clink); 	^ c.l: In function 'yylex': c.l:105:18: error: 'yytest' undeclared (first use in this function); did you mean 'yytext'? 105	yylval == atol(yytest); 	^~~~~ 	yytext c.l:105:18: note: each undeclared identifier is reported only once for each function it appears in lex.yy.c: At top level: lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function] 1572	#else 	^ lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function] 1529 | 	^ PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	yytest를 yytext로, ==를 =로 변경하는 수정안입니다.
수정안	yylval = atol(yytext);

05. makeNode 중복 정의: 빌드 목록 중복

문제 / 원인	syntax_tree.c를 두 번 링크하여 발생한 중복 정의 오류입니다.
문제 코드 실행 출력	`gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c semantic.c syntax_tree.c -o parser.exe` 원문: multiple definition of `makeNode'; ... syntax_tree.c:4: first defined here 원문: collect2.exe: error: ld returned 1 exit status
해결 / 상태	빌드 명령에서 중복된 syntax_tree.c 하나를 제거하는 수정안입니다.
수정안	`gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c semantic.c -o parser.exe`

06. 조건식의 비트 OR와 논리 OR 혼동

문제 / 원인	논리 OR 대신 비트 OR를 사용한 오류입니다.
문제 코드 실행 출력	원문: semantic.c:418:16: warning: suggest parentheses around comparison in operand of '|' [-Wparentheses] 문제 코드: if (id->kind != ID_FUNC | id->type == NULL || id->type->kind != T_FUNC)
해결 / 상태	|를 ||로 변경하는 수정안입니다.
수정안	`if (id->kind != ID_FUNC || id->type == NULL || id->type->kind != T_FUNC)`

07. YYSTYPE: 포인터와 정수 크기 불일치

문제 / 원인	long과 포인터의 크기 차이로 발생한 형 변환 경고입니다.
문제 코드 실행 출력	c.y: In function 'yyparse': c.y:308:52: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] 308 | $$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL); \| ^ c.y:308:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] 308 \| $$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL); | ^ c.y:310:22: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] 310 | A_NODE *node = (A_NODE*)$$; | ^ c.y:311:55: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] 311 | fprintf(stderr, "[AST] N_EXP_INT_CONST: %ld\n", (long)node->clink); | ^
해결 / 상태	포인터 너비의 YYSTYPE을 사용하고 Lex/Yacc 선언을 일치시키는 수정안입니다.

수정안	`#include <stdint.h>`
	`typedef intptr_t YYSTYPE;`
	`#define YY(x) ((YYSTYPE)(intptr_t)(x))`
	`#define PTR(T, x) ((T *)(intptr_t)(x))`

08. Lex 생성 코드의 미사용 함수 경고

문제 / 원인	Lex에서 생성된 input·yyunput이 사용되지 않은 경고입니다.
문제 코드 실행 출력	lex.yy.c: At top level: lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function] 1572 | #else | ^ lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function] 1529 | | ^
해결 / 상태	비치명적 경고이며, 아래 옵션으로 미사용 함수 생성을 끄는 방안입니다.
수정안	%option noinput nounput

09. Yacc 기호명·함수명 오타와 규칙 경계 오류

문제 / 원인	Yacc 기호명·함수명 오타와 문법 규칙 경계의 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> $env:AST_TEST_IDENTIFIER = "1" PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/05_identifier.c C:\\Users\\egod1\\proj\\compiler\\w4\\c.y:270.10-51: error: symbol 'parameter_ttestUpdateSpecifierype_list_opt' is used, but is not defined as a token and has no rules 270 | | LP parameter_ttestUpdateSpecifierype_list_opt RP | ^~~~~~~ [ERROR] bison failed. See "C:\Users\egod1\proj\compiler\w4\build.log". PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/05_identifier.c C:\Users\egod1\proj\compiler\w4\c.y: In function 'yyparse': C:\Users\egod1\proj\compiler\w4\c.y:124:36: error: implicit declaration of function 'linkDeclarator'; did you mean 'linkDeclaratorList'? [-Wimplicit-function-declaration] 124 | $$ = (YYSTYPE) (intptr_t) linkDeclarator(| ^~~~~~~~~~~~ | linkDeclaratorList lex.yy.c:1571:16: warning: 'input' defined but not used [-Wunused-function] 1571 | #else | ^ lex.yy.c:1528:17: warning: 'yyunput' defined but not used [-Wunused-function] 1528 | | ^ [ERROR] gcc failed. See "C:\Users\egod1\proj\compiler\w4\build.log". C:\\Users\\egod1\\proj\\compiler\\w4\\c.y:418.1-17: error: unexpected identifier 418 | labeled_statement | ^~~~~~~~~~~~~~~~~ [ERROR] bison failed. See "C:\Users\egod1\proj\compiler\w4\build.log". PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	기호명·함수명을 수정하고 앞 규칙의 ; 및 {}를 점검할 항목입니다.

10. 중간 액션과 중첩 함수 호출의 괄호 위치

문제 / 원인	중간 액션과 중첩 함수 호출의 괄호 위치를 혼동한 오류입니다.
문제 코드 실행 출력	enum_specifier : ENUM_SYM IDENTIFIER {} LR enumerator_list RR | ENUM_SYM LR enumerator_list RR | ENUM_SYM IDENTIFIER ; C:\Users\egod1\proj\compiler\w4\c.y:459:61: error: too few arguments to function 'setTypeExpr'; expected 2, have 1 459 | $$ = YY(setTypeElementType(PTR(A_TYPE, $1), setTypeExpr(makeType(T_ARRAY)), \| ^~~~~~~~~~ C:\Users\egod1\proj\compiler\w4\parse_value.h:18:36: note: in definition of macro 'YY' 18 \| #define YY(x) ((YYSTYPE)(intptr_t)(x)) \| ^ In file included from C:\Users\egod1\proj\compiler\w4\c.y:7: C:\Users\egod1\proj\compiler\w4\semantic.h:33:9: note: declared here 33 \| A_TYPE *setTypeExpr(A_TYPE *type, A_NODE *expr); \| ^~~~~~~~~~ C:\Users\egod1\proj\compiler\w4\c.y:459:18: error: too many arguments to function 'setTypeElementType'; expected 2, have 3 459 \| $$ = YY(setTypeElementType(PTR(A_TYPE, $1), setTypeExpr(makeType(T_ARRAY)), | ^~~~~~~~~~~~~~~~~~ C:\Users\egod1\proj\compiler\w4\parse_value.h:18:36: note: in definition of macro 'YY' 18 | #define YY(x) ((YYSTYPE)(intptr_t)(x)) | ^ C:\Users\egod1\proj\compiler\w4\semantic.h:28:9: note: declared here 28 | A_TYPE *setTypeElementType(A_TYPE *t1, A_TYPE *t2); | ^~~~~~~~~~~~~~~~~~ lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function] 1625 | #else | ^ lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function] 1582 | | ^ [ERROR] gcc failed. See "C:\Users\egod1\proj\compiler\w4\build.log". PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	중간 액션을 구분하고, 함수별 인수 두 개를 정확히 묶는 수정안입니다.
수정안	setTypeExpr(type, expr); setTypeElementType(t1, t2);

11. 괄호 수식: 파싱 성공 후 AST 검증 실패

문제 / 원인	괄호 수식의 AST가 검증 조건과 일치하지 않은 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/04_parentheses.c lex.yy.c:1571:16: warning: 'input' defined but not used [-Wunused-function] 1571 | #else | ^ lex.yy.c:1528:17: warning: 'yyunput' defined but not used [-Wunused-function] 1528 | | ^ [DEBUG] N_EXP_INT_CONST: 1 [DEBUG] N_EXP_INT_CONST: 2 [DEBUG] N_EXP_INT_CONST: 3 Parsing success [ERROR] Semantic error: Expected N_EXP_ADD [ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\04_parentheses.c" PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	괄호 규칙과 AST 검증 조건을 대조할 항목입니다. 해결 여부는 미확인입니다.

12. for문: 잘못된 문법 규칙과 AST 연결

문제 / 원인	iteration_statement의 for 문법 규칙 작성 오류입니다.
문제 코드 실행 출력	[SUCCESS] Parser test passed: "C:\Users\egod1\proj\compiler\w4\tests\ast\39_do_while_statement.c" lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function] 1593 | #else | ^ lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function] 1550 | | ^ [DEBUG] N_EXP_INT_CONST: 0 [DEBUG] N_EXP_INT_CONST: 3 Parsing success [ERROR] Semantic error: Invalid for AST [ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\40_for_statement.c" Exception: Line | 7 | throw "FAILED: (case[1])" | ~~~~~~~~~~~~~~~~~~~~~~~~~~에서 | FAILED: 40_for_statement.c
해결 / 상태	for 규칙과 AST 연결을 수정할 항목입니다. 수정 후 통과 여부는 미기록입니다.

13. 함수 인자 목록: 종료 노드 불일치

문제 / 원인	함수 인자 목록의 종료 노드가 예상과 다른 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/26_function_call_args.c >> lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function] 1593 | #else | ^ lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function] 1550 | | ^ C:\Users\egod1\proj\compiler\w4\syntax_tree.c: In function 'makeNodeList': C:\Users\egod1\proj\compiler\w4\syntax_tree.c:28:13: warning: variable 'nil' set but not used [-Wunused-but-set-variable=] 28 | NODE_NAME nil; | ^~~ [DEBUG] N_EXP_INT_CONST: 1 [DEBUG] N_EXP_INT_CONST: 2 [ERROR] Semantic error: Invalid list terminator [ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\26_function_call_args.c" PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	GDB로 리스트 종료 노드를 확인할 항목입니다. 해결 여부는 미확인입니다.

14. sizeof(type): 타입 노드 연결 검증 실패

문제 / 원인	sizeof(type)의 AST 구조가 검증 조건과 다른 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> >> PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/51_sizeof_type.c >> lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function] 1593 | #else | ^ lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function] 1550 | | ^ Parsing success [ERROR] Semantic error: Invalid sizeof type AST [ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\51_sizeof_type.c"
해결 / 상태	타입 정보와 N_EXP_SIZE_TYPE 연결을 확인할 항목입니다. 해결 여부는 미확인입니다.

15. static 처리 오류와 의도된 중복 지정자 오류

문제 / 원인	static int가 잘못 거부된 오류입니다. static auto의 거부는 의도된 결과입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> Set-Content "$env:TEMP\w4_static.c" 'static int counter; int main(void) { return counter; }' PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd "$env:TEMP\w4_static.c" lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function] 1625 | #else | ^ lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function] 1582 | | ^ [ERROR] Semantic error: Duplicate type specifier [ERROR] Parser test failed: "C:\Users\egod1\AppData\Local\Temp\w4_static.c" PS C:\Users\egod1\proj\compiler\w4> PS C:\Users\egod1\proj\compiler\w4> Set-Content "$env:TEMP\w4_storage_invalid.c" 'static auto int x;' PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd "$env:TEMP\w4_storage_invalid.c" lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function] 1625 | #else | ^ lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function] 1582 | | ^ [ERROR] Semantic error: Duplicate storage class specifier [ERROR] Parser test failed: "C:\Users\egod1\AppData\Local\Temp\w4_storage_invalid.c" PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	타입 지정자와 저장 클래스 지정자의 중복 검사를 분리할 항목입니다.

16. 전역 선언 목록: add 함수 출력 누락

문제 / 원인	전역 선언 목록에서 add 함수가 출력되지 않은 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> $env:AST_PRINT="1"; .\parser.cmd tests/valid/02_functions.c lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function] 1625 | #else | ^ lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function] 1582 | | ^ [DEBUG] N_EXP_INT_CONST: 2 [DEBUG] N_EXP_INT_CONST: 3 Parsing success ======= syntax tree ========== N_PROGRAM (0000000000000000,0) | (ID="main") TYPE:000001fa9b2c1f80 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="") TYPE:000001fa9b2c7e20 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (void)| | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0000000000000000,0) | | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | | N_STMT_LIST (0000000000000000,0) | | | | | | | | N_STMT_EXPRESSION (0000000000000000,0) | | | | | | | | | N_EXP_ASSIGN (0000000000000000,0) | | | | | | | | | | N_EXP_IDENT (0000000000000000,0) | | | | | | | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_EXP_FUNCTION_CALL (0000000000000000,0) | | | | | | | | | | N_EXP_IDENT (0000000000000000,0) | | | | | | | | | | | (ID="add") TYPE:000001fa9b2c1690 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | | | | | N_ARG_LIST (0000000000000000,0) | | | | | | | | | | | N_EXP_INT_CONST (0000000000000000,0) | | | | | | | | | | | | 2 | | | | | | | | | | | N_ARG_LIST (0000000000000000,0) | | | | | | | | | | | | N_EXP_INT_CONST (0000000000000000,0) | | | | | | | | | | | | | 3 | | | | | | | | | | | | N_ARG_LIST_NIL (0000000000000000,0) | | | | | | | N_STMT_LIST (0000000000000000,0) | | | | | | | | N_STMT_RETURN (0000000000000000,0) | | | | | | | | | N_EXP_IDENT (0000000000000000,0) | | | | | | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0 | | | | | | | | N_STMT_LIST_NIL (0000000000000000,0) [SUCCESS] Parser test passed: "C:\Users\egod1\proj\compiler\w4\tests\valid\02_functions.c" PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	A_ID.link로 선언 목록을 연결하여 add → main 순서의 출력이 확인된 상태입니다.

17. 문자 상수: 문자열 주소를 문자 코드로 출력

문제 / 원인	문자 코드 대신 문자열 주소의 일부가 출력된 오류입니다.
문제 코드 실행 출력	`int main(void) { return 'A'; }` `실제 출력 전체, 종료 코드 0:` `Parsing success` `=======  syntax tree  ==========` `N_PROGRAM ((nil),0)` `|  (ID="main") TYPE:0x561e365c38b0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|				(ID="") TYPE:0x561e365be360 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0` `|					TYPE` `|						(void)|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND ((nil),0)` `|					N_STMT_LIST ((nil),0)` `|						N_STMT_RETURN ((nil),0)` `|								N_EXP_CHAR_CONST ((nil),0)` `|									912013584` `|						N_STMT_LIST_NIL ((nil),0)` `실제 '\n' 출력:									7584016` `실제 '\0' 출력:									158300432` `Get-ChildItem Env:AST_TEST*	Remove-Item` `$env:AST_PRINT="1"` `.\parser.cmd tests/ast/62_char_string.c`
해결 / 상태	문자 코드를 정수로 저장하고 검증기의 기대값도 바꾸어야 하는 오류입니다.

18. 함수 원형 매개변수 스코프 누출

문제 / 원인	함수 매개변수가 전역 스코프에 남아 잘못 검색되는 오류입니다.
문제 코드 실행 출력	재현 A, 원형과 정의가 같은 매개변수 이름을 사용: int f(int x); int f(int x) { return x; } int main(void) { return f(1); } [ERROR] Semantic error: Redeclaration of parameter 재현 B, 선언되지 않은 이름이 다른 함수에서 허용됨: int f(int x); int main(void) { return x; } 실제 출력 전체, 종료 코드 0: Parsing success ======= syntax tree ========== N_PROGRAM ((nil),0) | (ID="f") TYPE:0x55e1ef94b8d0 KIND:FUNC SPEC=AUTO LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="x") TYPE:0x55e1ef9462a0 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (int) | | | | TYPE | | | | | (int) | (ID="main") TYPE:0x55e1ef94ba10 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="") TYPE:0x55e1ef946360 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (void)| | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND ((nil),0) | | | | | | N_STMT_LIST ((nil),0) | | | | | | | N_STMT_RETURN ((nil),0) | | | | | | | | N_EXP_IDENT ((nil),0) | | | | | | | | | (ID="x") TYPE:0x55e1ef9462a0 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | | N_STMT_LIST_NIL ((nil),0) 재현 C, 전역 변수와 함수 매개변수의 정상적인 가림도 거부됨: int x; int f(int x) { return x; } [ERROR] Semantic error: Redeclaration of parameter 'int f(int x); int f(int x) { return x; }' | .\parser.exe 'int f(int x); int main(void) { return x; }' | .\parser.exe 'int x; int f(int x) { return x; }' | .\parser.exe
해결 / 상태	매개변수 스코프를 분리하고 종료 시 검색 상태를 복원해야 하는 오류입니다.

19. AST 검증기 6건: 순서와 스코프 가정이 오래됨

문제 / 원인	검증기가 이전 선언 순서와 스코프 규칙을 사용하는 오류입니다.
문제 코드 실행 출력	전역 선언 목록 재현: int a; int b; int c; Parsing success [ERROR] Semantic error: Incorrect global declaration order 지역 변수 관련 재현: AST_TEST_ADDRESS_OF / 15_address_of.c, AST_TEST_PRE_INC / 18_pre_increment.c, AST_TEST_PRE_DEC / 19_pre_decrement.c, AST_TEST_POST_INC / 20_post_increment.c, AST_TEST_POST_DEC / 21_post_decrement.c. Parsing success [ERROR] Semantic error: Identifier mismatch Get-ChildItem Env:AST_TEST* | Remove-Item $env:AST_TEST_ADDRESS_OF="1" .\parser.cmd tests/ast/15_address_of.c Remove-Item Env:AST_TEST_ADDRESS_OF $env:AST_TEST_PROGRAM="1" 'int a; int b; int c;' | .\parser.exe Remove-Item Env:AST_TEST_PROGRAM
해결 / 상태	전역 목록은 link 순회, 지역 변수는 함수 본문의 선언 목록으로 검사하는 수정안입니다.

20. void 출력 뒤 줄바꿈 누락

문제 / 원인	(void) 뒤에 줄바꿈이 없어 다음 출력이 붙는 서식 오류입니다.
문제 코드 실행 출력	void f(void) { return; } int main(void) { f(); return 0; } 실제 출력 발췌: | | | | | | | (void)| | | | TYPE | | | | | (void)| | | | BODY | | | | | | | (void)| | | | TYPE
해결 / 상태	개행을 추가하는 수정안입니다. 제공된 print.c 보존을 위해 미적용입니다.
수정안	printf("(void)\n");

21. typedef 가림과 미지원 문법의 별도 분류

문제 / 원인	내부 스코프의 typedef 가림과 미지원 문법 처리에 관한 항목입니다.
문제 코드 실행 출력	typedef int I; int main(void) { typedef float I; I y; return 0; } [ERROR] Semantic error: Duplicate type specifier 01_declarations, 06_storage_qualifiers, 07_arrays_and_calls: Initializers are not supported. 04_struct_enum: Enumerator initializers are not supported. 05_operators: Left shift is not supported. 09_bitwise: Bitwise AND is not supported. 13_variadic_function: Variadic parameters are not supported. 15_comma_and_conditional: Comma expressions are not supported.
해결 / 상태	typedef 가림은 지원 범위 확인 대상입니다. 미지원 문법 실패는 별도 분류 대상입니다.

컴파일러 과제5 보고서

20221500 양성준

목차

1. 구현 1
    1-1. 구현 환경 1
    1-2. 프로그램 구성 1
    1-3. 구현 및 변경 사항 2
    1-4. 주요 구현 원리 2
    1-5. 구현 범위 및 제한사항 2
2. 검증 3
    2-1. 테스트 구성 3
    2-2. 테스트 결과 3
    2-3. AST 구조 검증 4
    2-4. 오류 검증 10
    2-5. 테스트 케이스 상세 11
3. 개발 과정 33
    3-1. 오류 및 해결 기록 33

1. 구현

1-1. 구현 환경

64비트 UNIX(macOS ARM64) 환경에서 Bison, lex, GCC를 직접 사용해 빌드하고 테스트했습니다.

1-2. 프로그램 구성

교수님 부록 A1~A4, A9를 바탕으로 코드를 작성했습니다.

파일	주요 역할
c.l	토큰 분석, yylval, 줄 번호 처리
c.y	문법 분석, 신택스 트리 생성
support.c / support.h	심볼, 타입, 스코프 관리, 오류 처리, 함수 선언
type.h	자료구조 정의
print.c	신택스 트리 출력
main.c	초기화, 파싱 및 출력 실행

1-3. 구현 및 변경 사항

교수님 부록 코드의 문법과 자료구조를 바탕으로 다음 사항을 수정했습니다.

1-3-1. 구문분석기 실행 흐름 정리

- **기존 동작:** 어셈블리 파일 생성, 의미분석과 코드생성 수행
- **제거 범위:** semantic_analysis(), code_generation(), a.asm 생성
- **실행부 수정:** initialize(); yyparse(); print_ast(root); 순서로 호출
- **오류 처리 코드:** if (yyparse() || syntax_err) return 1; 추가
- **출력:** 정상 입력에서 print_ast(root)로 신택스 트리 출력

1-3-2. 64비트 환경의 YYSTYPE 선언

- **기존 문제:** 4바이트 int에 64비트 포인터를 저장하면 값 손실 가능
- **선언 추가:** c.l과 c.y에 typedef long YYSTYPE; 적용
- **선언 정리:** #define YYSTYPE_IS_DECLARED 1 추가, extern int yylval 삭제
- **적용 환경:** long과 포인터가 8바이트인 64비트 UNIX의 LP64
- **유지 사항:** 기존 문법 규칙과 신택스 트리, 심볼 및 타입 자료구조 유지

1-3-3. 함수 프로토타입과 공통 헤더

- **기존 문제:** support.c의 함수 선언을 c.y에서 참조할 수 없는 구조
- **선언 통합:** makeNode(), makeType() 등의 프로토타입을 support.h로 이동
- **포함 코드:** c.y와 support.c에 #include "support.h" 추가
- **수정 목적:** 포인터 반환 함수를 정수 반환 함수로 잘못 해석하는 문제 방지
- **유지 사항:** 기존 함수의 이름, 반환형과 구현 유지

1-3-4. 직접 컴파일을 위한 선언 보완

- **빌드 도구:** Bison, Lex, GCC 명령으로 직접 컴파일
- **c.y 보완:** stdio.h 포함, yylex()와 yyerror() 선언 추가
- **c.l 보완:** 표준 헤더와 로컬 함수 선언, noyywrap 옵션 추가
- **support.c 보완:** string.h 포함, isPointerOrArrayType() 구현 추가
- **빌드 진단:** dangling else 충돌과 원본 구조의 비치명적 경고 유지

1-3-5. 불필요한 선언과 헤더 정리

- **정리 대상:** 사용하지 않는 지역 변수와 불필요한 헤더 의존성
- **checkIdentifier():** 사용하지 않는 char *t 선언 제거
- **setDefaultSpecifier():** 사용하지 않는 A_TYPE *t 선언 제거
- **헤더 삭제:** support.c의 #include "y.tab.h" 제거
- **유지 사항:** 교수님이 제공한 type.h와 print.c 유지

2. 검증

2-1. 테스트 구성

구문분석기의 기능과 출력 결과를 검증하기 위해 **총 64개의 테스트**를 작성했습니다. 정상 파싱 19개, 신택스 트리 구조 15개, 오류 처리 20개, 엣지케이스 10개로 구성했습니다.

그룹	검증 목적
정상 파싱	정상 선언, 함수, 제어문 등의 파싱 검증
AST	노드 깊이, 전위 순서, 연산자 우선순위 비교
오류 처리	yyerror(), syntax_error() 출력과 종료 코드 확인
엣지케이스	빈 블록, 이름 가림, 전방 참조, 복합 선언자 검증

2-2. 테스트 결과

테스트 실행 결과는 다음과 같습니다.

그룹	통과	실패
정상 파싱	19	0
AST	15	0
오류 처리	20	0
엣지케이스	10	0
합계	64	0

전체 64개 테스트가 통과했고 **실패는 0개**였습니다.

2-3. 정상 파싱 검증

변수 선언, 함수 정의, 구조체와 제어문 등 정상 입력 19개의 파싱과 신택스 트리 출력을 확인했습니다.

P01. 전역 변수 선언

의도 / 기준	전역 변수 선언
입력 코드	`int global_value;`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P01_global_variable.c`
	`=======  syntax tree  ==========`
	`N_PROGRAM (0x0,0)`
	`|  (ID="global_value") TYPE:0x100371920 KIND:VAR LEV=0 VAL=0 ADDR=0`
	`|  |  TYPE`
	`|  |  |  (int)`

P02. 다중 선언자

의도 / 기준	다중 선언자
입력 코드	`int first, second, third;`

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P02_multiple_declarators.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="first") TYPE:0x104d75920 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | (int) | (ID="second") TYPE:0x104d75920 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | (int) | (ID="third") TYPE:0x104d75920 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | (int)

P03. 정수 초기화 선언

의도 / 기준	정수 초기화 선언
입력 코드	int answer = 42;
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P03_scalar_initializer.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="answer") TYPE:0x105181920 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | (int)

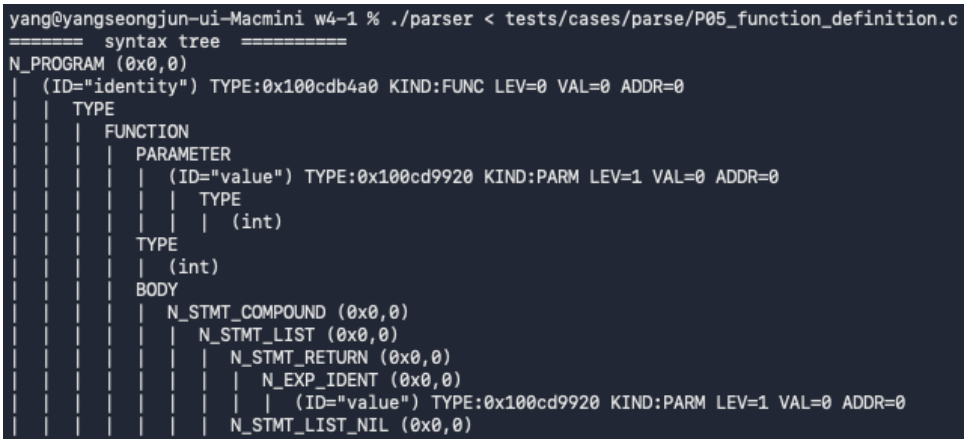
P04. 함수 원형 선언

의도 / 기준	함수 원형 선언
입력 코드	int add(int left, int right);
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P04_function_prototype.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="add") TYPE:0x100c93510 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="left") TYPE:0x100c91920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (int) | | | | | (ID="right") TYPE:0x100c91920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (int) | | | | TYPE | | | | | (int)

P05. 함수 정의

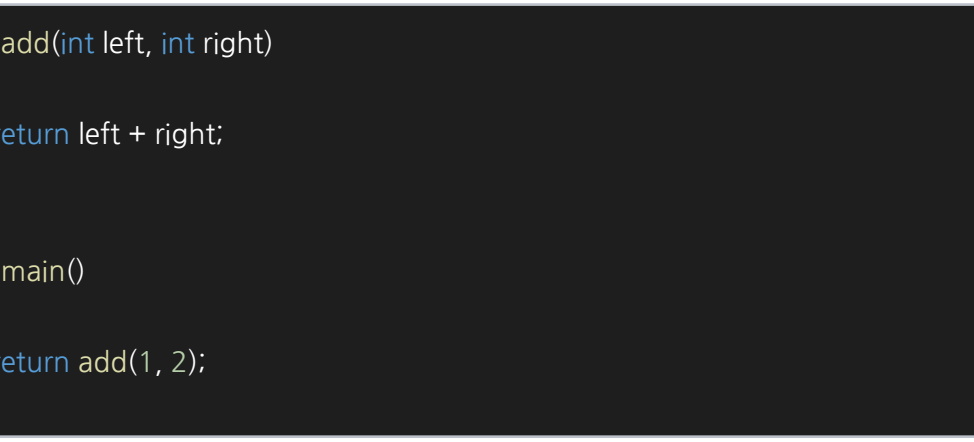
의도 / 기준	함수 정의

입력 코드	int identity(int value)
	{
	  return value;
	}

실행 결과	

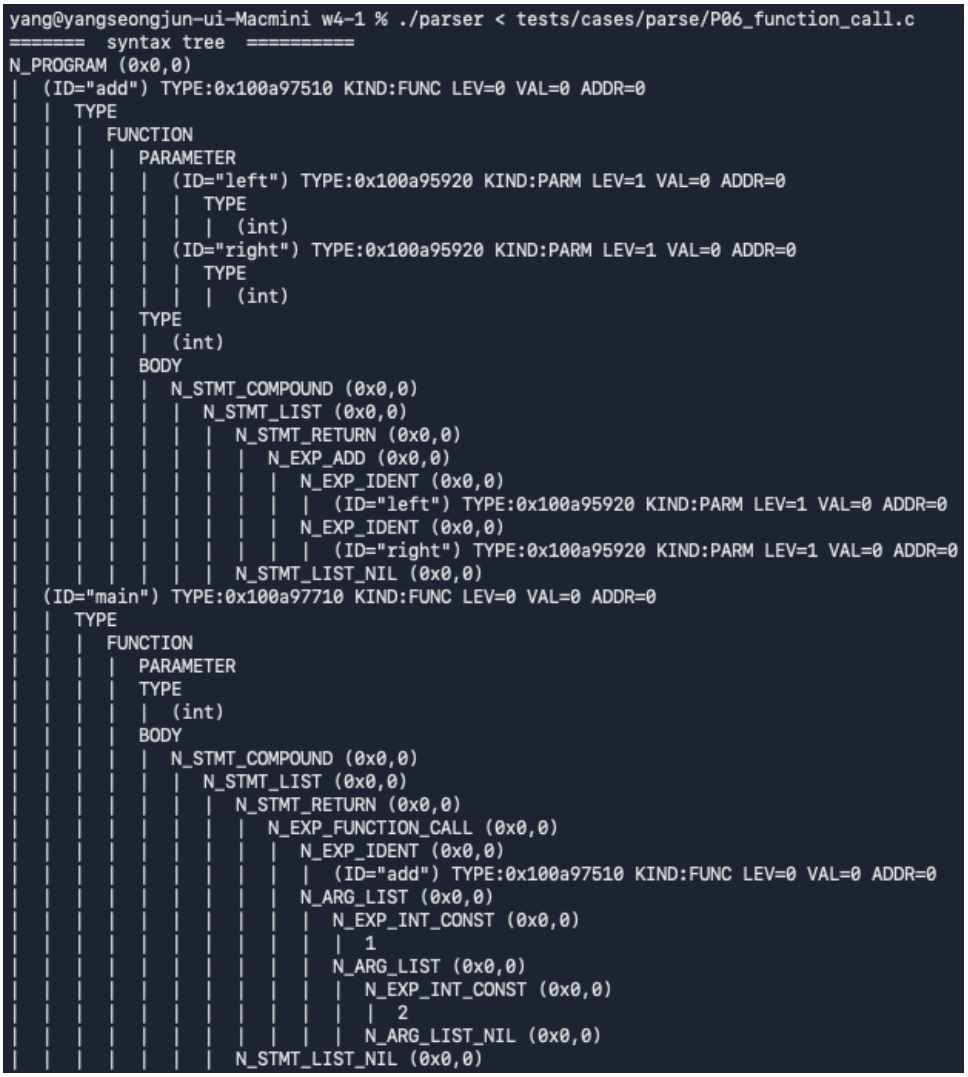
```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P05_function_definition.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="identity") TYPE:0x100cdb4a0 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="value") TYPE:0x100cd9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x100cd9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

P06. 함수 호출

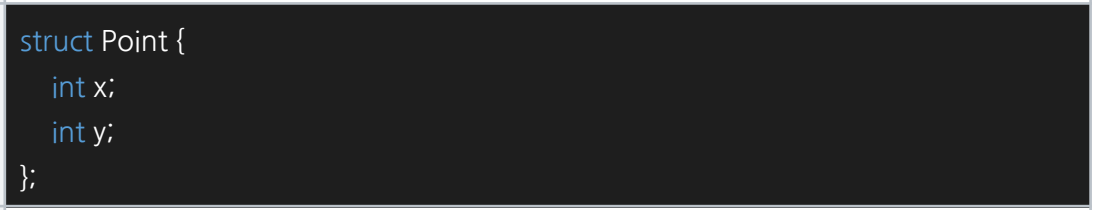
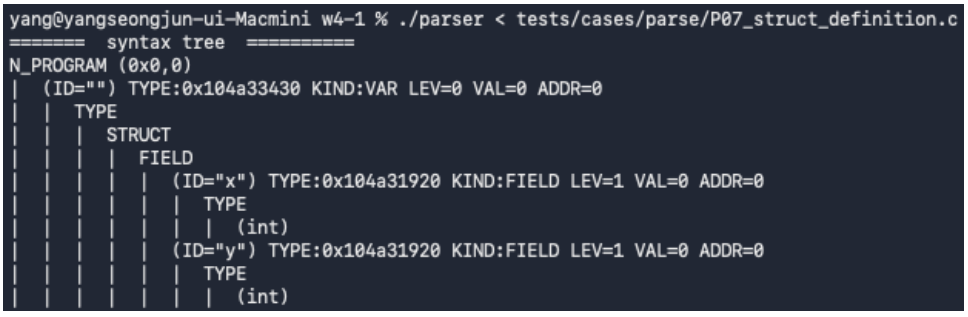
의도 / 기준	함수 호출
입력 코드	

```c
int add(int left, int right)
{
    return left + right;
}

int main()
{
    return add(1, 2);
}
```

실행 결과	

P07. 구조체 정의

의도 / 기준	구조체 정의
입력 코드	
실행 결과	

P09. 나열형 정의

의도 / 기준	나열형 정의

입력 코드	enum Color {
	  RED,
	  GREEN,
	  BLUE
	};

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P09_enum_definition.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="") TYPE:0x103033430 KIND:VAR LEV=0 VAL=0 ADDR=0 |	TYPE |		ENUM |			ENUMERATORS |				(ID="RED") TYPE:0x0 KIND:ENUM_LITERAL LEV=0 VAL=0 ADDR=0 |				(ID="GREEN") TYPE:0x0 KIND:ENUM_LITERAL LEV=0 VAL=0 ADDR=0 |				(ID="BLUE") TYPE:0x0 KIND:ENUM_LITERAL LEV=0 VAL=0 ADDR=0

P10. 배열 선언

의도 / 기준	배열 선언
입력 코드	int values[4];
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P10_array_declaration.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="values") TYPE:0x100ed3460 KIND:VAR LEV=0 VAL=0 ADDR=0 |	TYPE |		ARRAY |			INDEX |				N_EXP_INT_CONST (0x0,0) |					4 |			ELEMENT_TYPE |				(int)

P11. 포인터 선언

의도 / 기준	포인터 선언
입력 코드	int *pointer;
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P11_pointer_declaration.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="pointer") TYPE:0x1010633e0 KIND:VAR LEV=0 VAL=0 ADDR=0 |	TYPE |		POINTER |			ELEMENT_TYPE |				(int)

P12. 이중 포인터 선언

의도 / 기준	이중 포인터 선언
입력 코드	int **pointer;

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P12_double_pointer.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="pointer") TYPE:0x102a9b3e0 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | POINTER | | | | ELEMENT_TYPE | | | | | POINTER | | | | | | ELEMENT_TYPE | | | | | | | (int)

P13. 포인터 배열

의도 / 기준	포인터 배열
입력 코드	int *items[3];
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P13_array_of_pointers.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="items") TYPE:0x100f634a0 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | ARRAY | | | | INDEX | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | 3 | | | | ELEMENT_TYPE | | | | | POINTER | | | | | | ELEMENT_TYPE | | | | | | | (int)

P14. 배열 포인터 선언

의도 / 기준	배열 포인터 선언
입력 코드	int (*matrix)[3];
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P14_pointer_to_array.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="matrix") TYPE:0x1012bf3e0 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | POINTER | | | | ELEMENT_TYPE | | | | | ARRAY | | | | | | INDEX | | | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | | | 3 | | | | | | ELEMENT_TYPE | | | | | | | (int)

P15. if-else

의도 / 기준	if-else

입력 코드	int choose(int condition)
	{
	  if (condition)
	  return 1;
	  else
	  return 0;
	}

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P15_if_else.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="choose") TYPE:0x1012c34a0 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="condition") TYPE:0x1012c1920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  |  |  |  |  TYPE
|  |  |  |  |  |  (int)
|  |  |  |  |  BODY
|  |  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  |  N_STMT_IF_ELSE (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  (ID="condition") TYPE:0x1012c1920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  0
|  |  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

P16. while

의도 / 기준	while

입력 코드

```c
int count_down(int value)
{
    while (value)
        value = value - 1;
    return value;
}
```

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P16_while_loop.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="count_down") TYPE:0x1017ab4a0 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  |  |  |  |  TYPE
|  |  |  |  |  |  (int)
|  |  |  |  |  BODY
|  |  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  |  N_STMT_WHILE (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  N_STMT_EXPRESSION (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_ASSIGN (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  |  |  N_EXP_SUB (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

P17. do-while

의도 / 기준	do-while

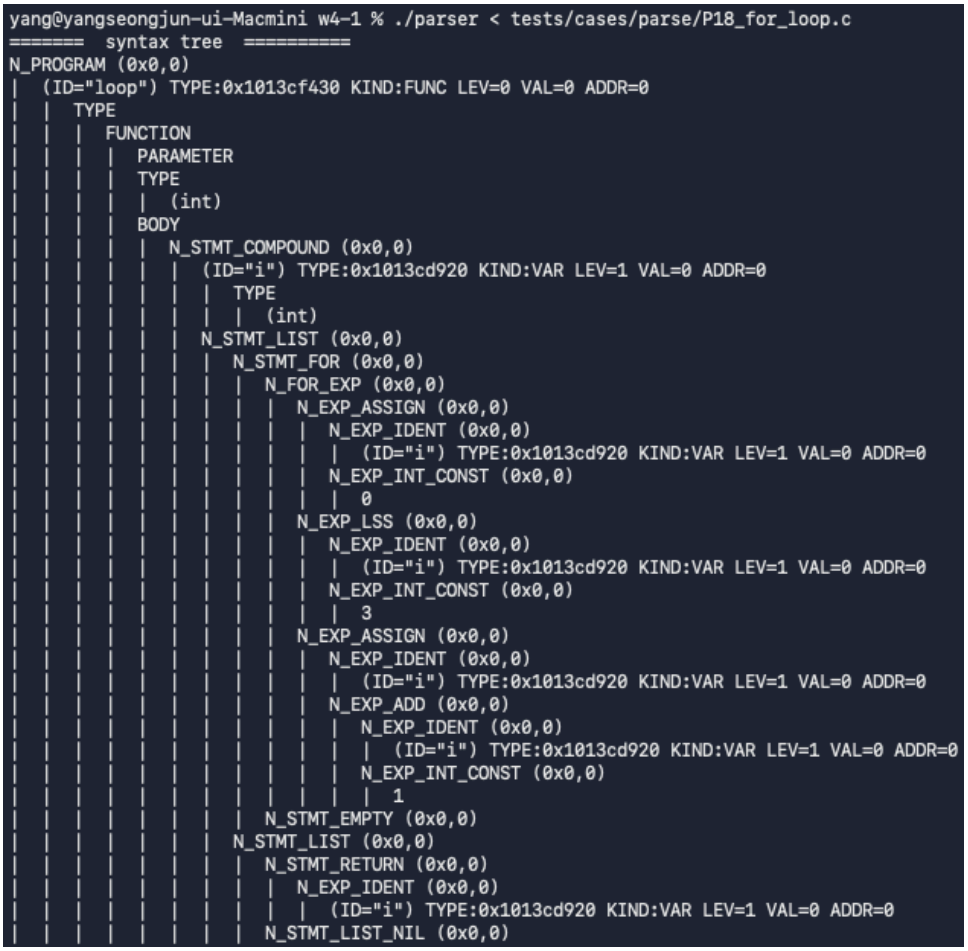
입력 코드	int count_once(int value)
	{
	  do
	  value = value - 1;
	  while (value);
	  return value;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P17_do_while_loop.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="count_once") TYPE:0x1033d34a0 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (int) | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_DO (0x0,0) | | | | | | | | N_STMT_EXPRESSION (0x0,0) | | | | | | | | | N_EXP_ASSIGN (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | | | N_EXP_SUB (0x0,0) | | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | | | | | | | 1 | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | N_STMT_LIST (0x0,0) | | | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_STMT_LIST_NIL (0x0,0)

P18. for

의도 / 기준	for

입력 코드	int loop()
	{
	  int i;
	  for (i = 0; i < 3; i = i + 1)
	  ;
	  return i;
	}

실행 결과	

P19. switch와 레이블

의도 / 기준	switch와 레이블

입력 코드	```int select_value(int value)
	{
	  switch (value) {
	  case 0:
	  return 1;
	  default:
	  return 2;
	  }
	}```

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P19_switch_statement.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="select_value") TYPE:0x10430f4a0 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="value") TYPE:0x10430d920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  N_STMT_SWITCH (0x0,0)
|  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x10430d920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_STMT_LABEL_CASE (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  0
|  |  |  |  |  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  N_STMT_LABEL_DEFAULT (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  |  |  2
|  |  |  |  |  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
|  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

P20. typedef 사용

의도 / 기준	typedef 사용
입력 코드	`typedef int number;` `number value;`
실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P20_typedef_use.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="number") TYPE:0x100b8d920 KIND:TYPE LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  (int)
|  (ID="value") TYPE:0x100b8d920 KIND:VAR LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  (int)
```

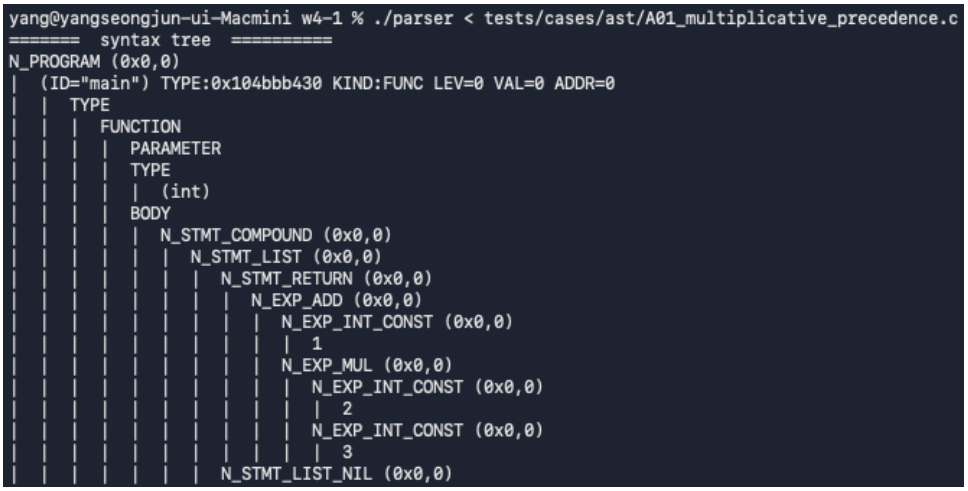
2-4. AST 구조 검증

연산자 우선순위, 노드 깊이와 자식 연결 순서를 AST 테스트 15개로 확인했습니다.

A01. 곱셈 우선순위

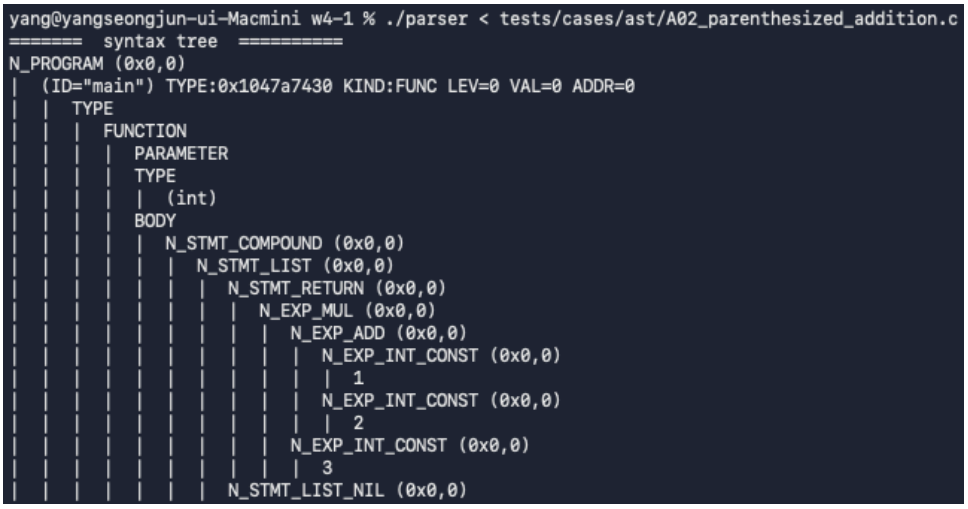
의도 / 기준	곱셈 우선순위

입력 코드	`int main()`
	`{`
	`  return 1 + 2 * 3;`
	`}`

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A01_multiplicative_precedence.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="main") TYPE:0x104bbb430 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  N_EXP_ADD (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  |  N_EXP_MUL (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  2
|  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  3
|  |  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

A02. 괄호 우선순위

의도 / 기준	괄호 우선순위
입력 코드	
실행 결과	

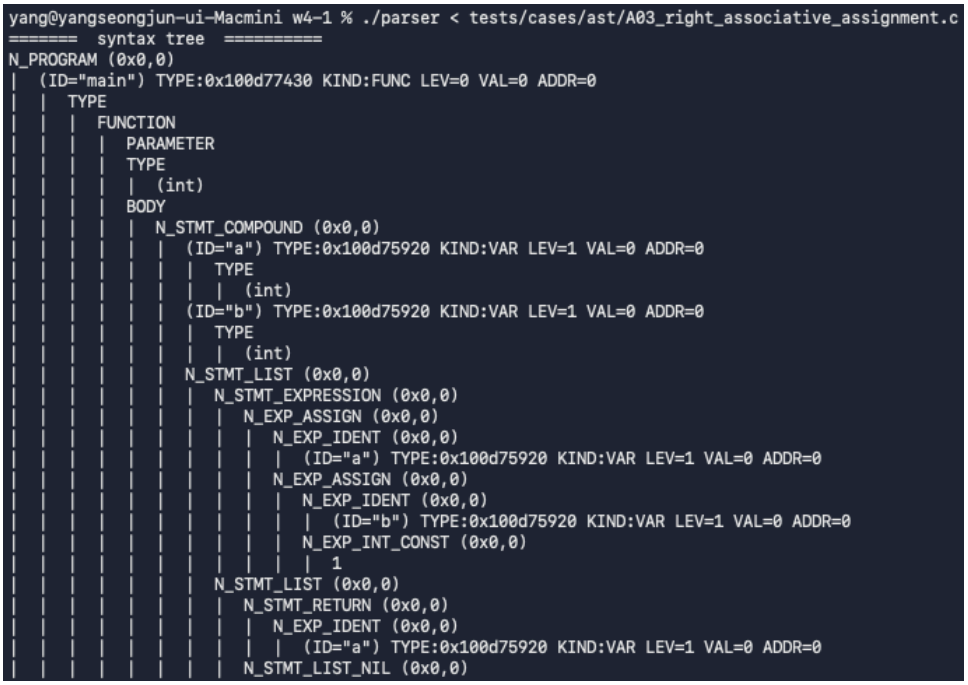
```
int main()
{
    return (1 + 2) * 3;
}
```

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A02_parenthesized_addition.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="main") TYPE:0x1047a7430 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  N_EXP_MUL (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_ADD (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  2
|  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  3
|  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

A03. 대입 결합 순서

의도 / 기준	대입 결합 순서
입력 코드	

```
int main()
{
    int a;
    int b;
    a = b = 1;
    return a;
}
```

실행 결과	

A04. 논리 연산자 우선순위

의도 / 기준	논리 연산자 우선순위

입력 코드	```int main()
	{
	  int a;
	  int b;
	  int c;
	  return a || b && c;
	}```

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A04_logical_precedence.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="main") TYPE:0x100e17430 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | (ID="a") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | (ID="b") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | (ID="c") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | N_EXP_OR (0x0,0) | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | (ID="a") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_EXP_AND (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="b") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="c") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | N_STMT_LIST_NIL (0x0,0)

A05. 관계 및 동등 연산자

의도 / 기준	관계 및 동등 연산자

입력 코드	int main()
	{
	  int a;
	  int b;
	  int c;
	  return a == b < c;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A05_equality_relational_precedence.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="main") TYPE:0x10149f430 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | (ID="a") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | (ID="b") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | (ID="c") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | N_EXP_EQL (0x0,0) | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | (ID="a") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_EXP_LSS (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="b") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="c") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | N_STMT_LIST_NIL (0x0,0)

A06. 중첩 단항 연산

의도 / 기준	중첩 단항 연산

입력 코드	`int main()`
	`{`
	`  int value;`
	`  return !-value;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A06_nested_unary.c`
	`=======  syntax tree  ==========`
	`N_PROGRAM (0x0,0)`
	`|  (ID="main") TYPE:0x1013c7430 KIND:FUNC LEV=0 VAL=0 ADDR=0`
	`|  |  TYPE`
	`|  |  |  FUNCTION`
	`|  |  |  |  PARAMETER`
	`|  |  |  |  TYPE`
	`|  |  |  |  |  (int)`
	`|  |  |  |  BODY`
	`|  |  |  |  |  N_STMT_COMPOUND (0x0,0)`
	`|  |  |  |  |  |  (ID="value") TYPE:0x1013c5920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  |  TYPE`
	`|  |  |  |  |  |  |  |  (int)`
	`|  |  |  |  |  |  N_STMT_LIST (0x0,0)`
	`|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)`
	`|  |  |  |  |  |  |  |  N_EXP_NOT (0x0,0)`
	`|  |  |  |  |  |  |  |  |  N_EXP_MINUS (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1013c5920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)`

A07. 전위와 후위 증감

의도 / 기준	전위와 후위 증감

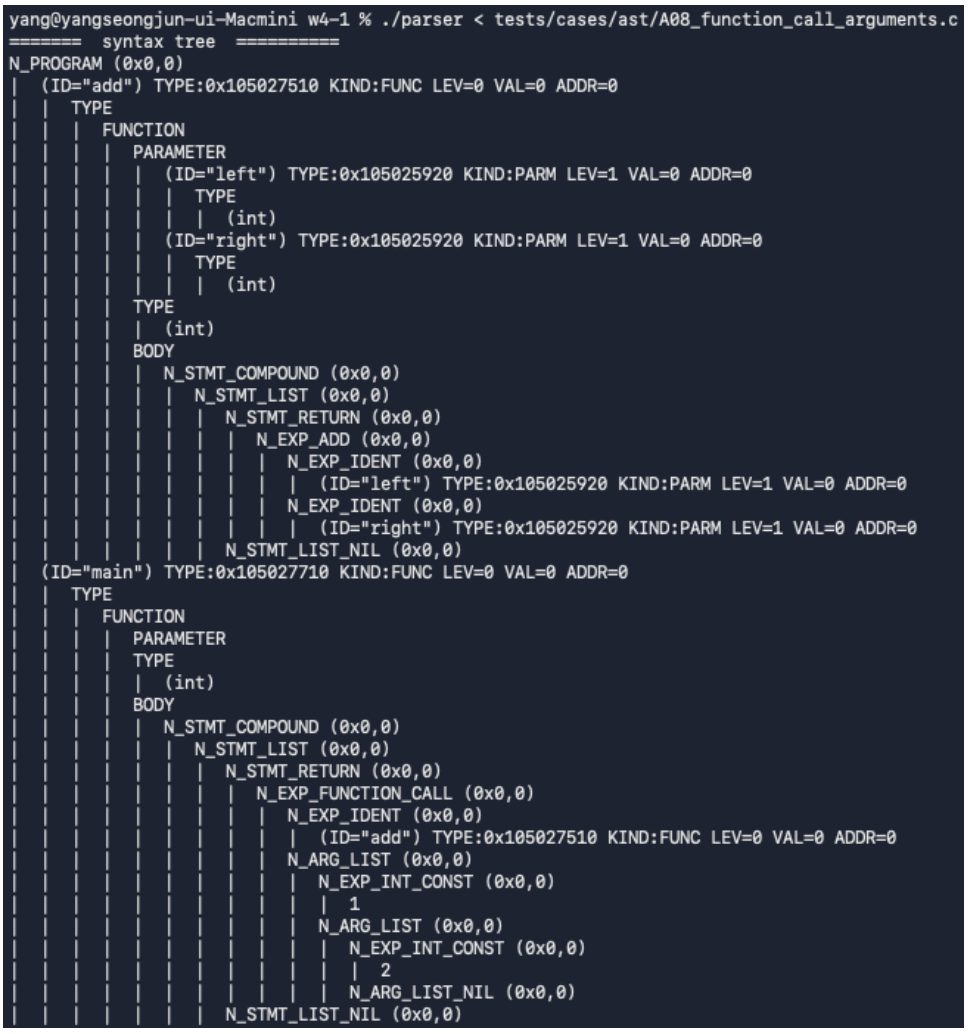
입력 코드	`int main()`
	`{`
	`  int value;`
	`  return ++value + value--;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A07_pre_and_post_increment.c`
	`=======  syntax tree  ==========`
	`N_PROGRAM (0x0,0)`
	`|  (ID="main") TYPE:0x10314f430 KIND:FUNC LEV=0 VAL=0 ADDR=0`
	`|  |  TYPE`
	`|  |  |  FUNCTION`
	`|  |  |  |  PARAMETER`
	`|  |  |  |  TYPE`
	`|  |  |  |  |  (int)`
	`|  |  |  |  BODY`
	`|  |  |  |  |  N_STMT_COMPOUND (0x0,0)`
	`|  |  |  |  |  |  (ID="value") TYPE:0x10314d920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  |  TYPE`
	`|  |  |  |  |  |  |  |  (int)`
	`|  |  |  |  |  |  N_STMT_LIST (0x0,0)`
	`|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)`
	`|  |  |  |  |  |  |  |  N_EXP_ADD (0x0,0)`
	`|  |  |  |  |  |  |  |  |  N_EXP_PRE_INC (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x10314d920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  |  |  |  N_EXP_POST_DEC (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x10314d920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)`

A08. 함수 호출 인자

의도 / 기준	함수 호출 인자

입력 코드	`int add(int left, int right)`
	`{`
	`  return left + right;`
	`}`
	
	`int main()`
	`{`
	`  return add(1, 2);`
	`}`

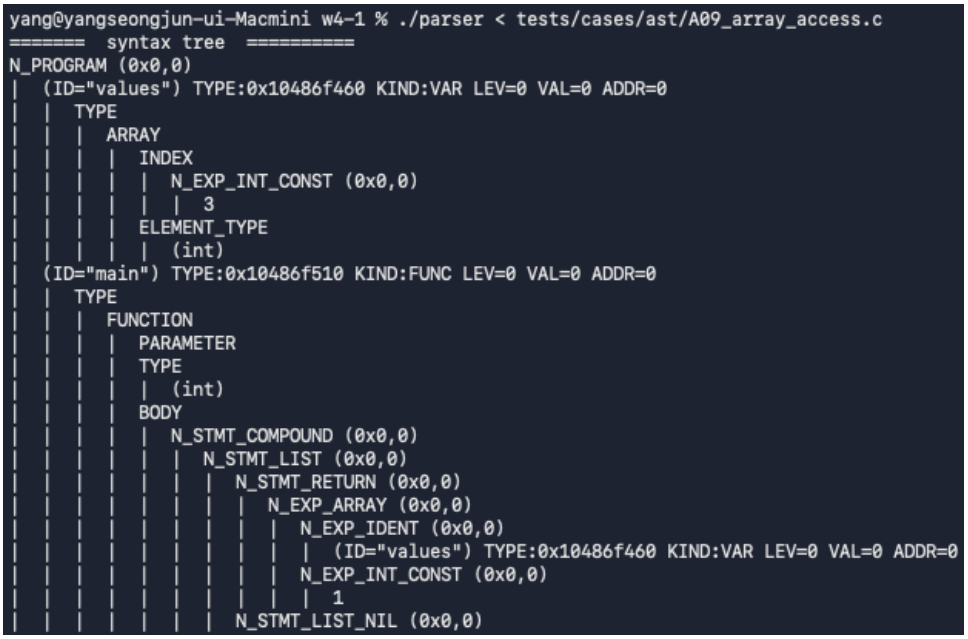
실행 결과	

A09. 배열 접근

의도 / 기준	배열 접근

입력 코드	`int values[3];`
	
	`int main()`

	{
	  return values[1];
	}

실행 결과	

A10. 구조체 멤버

의도 / 기준	구조체 멤버

입력 코드	struct Point {
	  int x;
	};
	
	struct Point point;
	
	int main()
	{
	  return point.x;
	}

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A10_struct_member.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="") TYPE:0x104eab430 KIND:VAR LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  STRUCT
|  |  |  |  FIELD
|  |  |  |  |  (ID="x") TYPE:0x104ea9920 KIND:FIELD LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  (ID="point") TYPE:0x104eab430 KIND:VAR LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  (DONE:0x104eab430)
|  (ID="main") TYPE:0x104eab600 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  N_EXP_STRUCT (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  (ID="point") TYPE:0x104eab430 KIND:VAR LEV=0 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  |  x
|  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

A11. 포인터 멤버

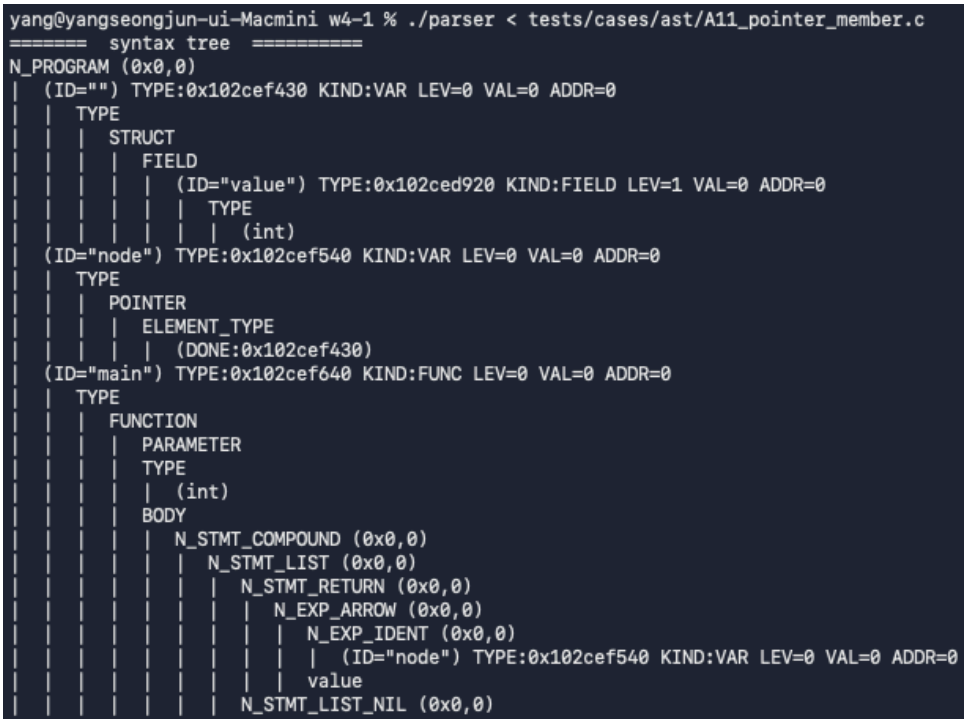
의도 / 기준	포인터 멤버

입력 코드

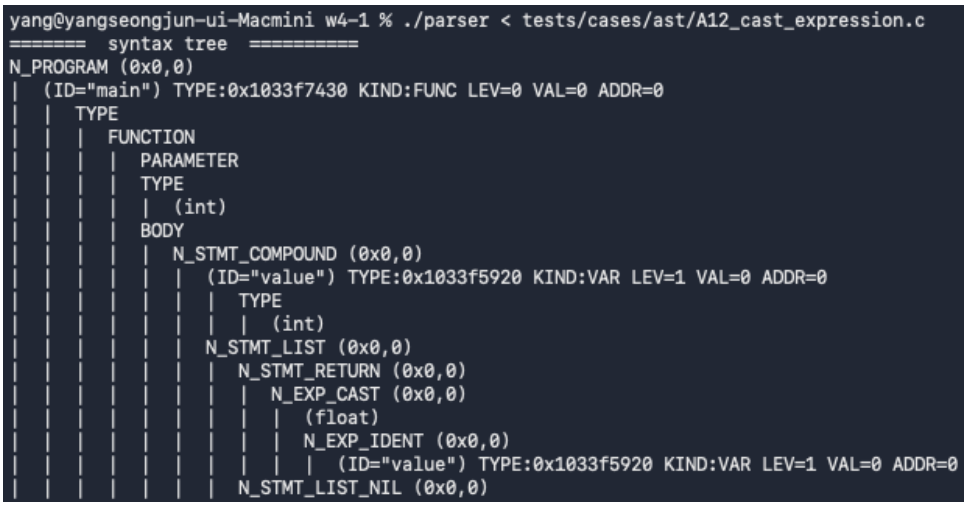
```c
struct Node {
    int value;
};

struct Node *node;

int main()
{
    return node->value;
}
```

실행 결과	

A12. 형 변환

의도 / 기준	형 변환
입력 코드	
실행 결과	

A13. sizeof

의도 / 기준	sizeof
입력 코드	

	int value;
	return sizeof value + sizeof(int);
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A13_sizeof_forms.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="main") TYPE:0x102b6b430 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | (ID="value") TYPE:0x102b69920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | N_EXP_ADD (0x0,0) | | | | | | | | | N_EXP_SIZE_EXP (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="value") TYPE:0x102b69920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_EXP_SIZE_TYPE (0x0,0) | | | | | | | | | | (int) | | | | | | | N_STMT_LIST_NIL (0x0,0)

A14. if-else 구조

의도 / 기준	if-else 구조

입력 코드	int choose(int condition)
	{
	if (condition)
	return 1;
	else
	return 2;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A14_if_else_children.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="choose") TYPE:0x102bf74a0 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="condition") TYPE:0x102bf5920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (int) | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_IF_ELSE (0x0,0) | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | (ID="condition") TYPE:0x102bf5920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | | | | | 1 | | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | | | | | 2 | | | | | | | N_STMT_LIST_NIL (0x0,0)

A15. for의 수식

의도 / 기준	for의 수식

입력 코드	`int main()`
	`{`
	`  int i;`
	`  for (i = 0; i < 2; i = i + 1)`
	`  ;`
	`  return i;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A15_for_expression_children.c` `=======  syntax tree  ==========` `N_PROGRAM (0x0,0)` `|  (ID="main") TYPE:0x104bc3430 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND (0x0,0)` `|					(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|						TYPE` `|							(int)` `|					N_STMT_LIST (0x0,0)` `|						N_STMT_FOR (0x0,0)` `|							N_FOR_EXP (0x0,0)` `|								N_EXP_ASSIGN (0x0,0)` `|									N_EXP_IDENT (0x0,0)` `|										(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|									N_EXP_INT_CONST (0x0,0)` `|										0` `|								N_EXP_LSS (0x0,0)` `|									N_EXP_IDENT (0x0,0)` `|										(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|									N_EXP_INT_CONST (0x0,0)` `|										2` `|								N_EXP_ASSIGN (0x0,0)` `|									N_EXP_IDENT (0x0,0)` `|										(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|									N_EXP_ADD (0x0,0)` `|										N_EXP_IDENT (0x0,0)` `|											(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|										N_EXP_INT_CONST (0x0,0)` `|											1` `|								N_STMT_EMPTY (0x0,0)` `|							N_STMT_LIST (0x0,0)` `|								N_STMT_RETURN (0x0,0)` `|									N_EXP_IDENT (0x0,0)` `|										(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|							N_STMT_LIST_NIL (0x0,0)`

2-5. 오류 검증

잘못된 선언과 문법을 포함한 입력 20개에 대해 오류 메시지와 신택스 트리 출력 여부를 확인했습니다.

E01. 전역 변수 중복

의도 / 기준	전역 변수 중복 AST 출력 없음
입력 코드	`int value;` `int value;`
실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E01_duplicate_global.c` `line 2: syntax error: redeclaration of identifier value near ;`

E02. 지역 변수 중복

의도 / 기준	지역 변수 중복 AST 출력 없음

입력 코드	`int main()`
	`{`
	`  int value;`
	`  int value;`
	`  return 0;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E02_duplicate_local.c` `line 4: syntax error: redeclaration of identifier value near ;`

E03. 매개변수 중복

의도 / 기준	매개변수 중복 AST 출력 없음

입력 코드	`int compare(int value, int value)`
	`{`
	`  return value;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E03_duplicate_parameter.c` `line 1: syntax error: redeclaration of identifier value near )`

E04. 구조체 필드 중복

의도 / 기준	구조체 필드 중복 AST 출력 없음

입력 코드	`struct Point {`
	`  int x;`
	`  int x;`
	`};`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E04_duplicate_struct_field.c` `line 3: syntax error: redeclaration of identifier x near ;`

E05. 미선언 식별자

의도 / 기준	미선언 식별자 AST 출력 없음

입력 코드	`int main()`
	`{`
	`  return missing;`
	`}`

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E05_undeclared_identifier.c line 3: syntax error: undefined identifier missing near missing

E06. 미선언 함수

의도 / 기준	미선언 함수 AST 출력 없음

입력 코드	int main()
	{
	  return missing(1);
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E06_undeclared_function.c line 3: syntax error: undefined identifier missing near missing

E07. void 매개변수

의도 / 기준	void 매개변수 AST 출력 없음

입력 코드	int invalid(void value)
	{
	  return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E07_named_void_parameter.c line 1: syntax error: illegal type specifier in formal parameter near)

E08. 매개변수 저장 클래스

의도 / 기준	매개변수 저장 클래스 AST 출력 없음

입력 코드	int invalid(static int value)
	{
	  return value;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E08_parameter_storage_class.c line 1: syntax error: illegal type specifier in formal parameter near)

E09. 형 변환의 저장 클래스

의도 / 기준	형 변환의 저장 클래스 AST 출력 없음
입력 코드	int main() { int value; return (static int)value;

	} 실행 결과 yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E09_storage_class_in_cast.c line 4: syntax error: illegal storage class in type specifiers near)

E10. 잘못된 함수 선언자

의도 / 기준	잘못된 함수 선언자 AST 출력 없음

입력 코드	int value[1]
	{
	return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E10_illegal_function_declarator.c line 2: syntax error: illegal function declarator near { line 5: syntax error: undefined type for identifier value at end

E11. 원형의 매개변수 타입 충돌

의도 / 기준	원형의 매개변수 타입 충돌 AST 출력 없음

입력 코드	int calculate(int value);
	
	int calculate(float value)
	{
	return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E11_parameter_type_conflict.c line 4: syntax error: conflicting parm type in prototype function calculate near {

E12. 이름 없는 정의 매개변수

의도 / 기준	이름 없는 정의 매개변수 AST 출력 없음

입력 코드	int unnamed(int)
	{
	return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E12_unnamed_definition_parameter.c line 2: syntax error: empty parameter name near {

E13. 충돌하는 타입 명시자

의도 / 기준	충돌하는 타입 명시자 AST 출력 없음

입력 코드	int float value;
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E13_conflicting_type_specifiers.c line 1: syntax error: illegal declaration specifiers near value

E14. 함수 저장 클래스

의도 / 기준	함수 저장 클래스 AST 출력 없음

입력 코드	static int main()
	{
	  return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E14_function_storage_class.c line 2: syntax error: illegal function specifiers near {

E15. 반환 타입 충돌

의도 / 기준	반환 타입 충돌 AST 출력 없음

입력 코드	int convert(int value);
	
	float convert(int value)
	{
	  return 0.0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E15_return_type_conflict.c line 4: syntax error: illegal or conflicting return type in function convert near {

E16. 태그 종류 충돌

의도 / 기준	태그 종류 충돌 AST 출력 없음

입력 코드	struct Item {
	  int value;
	};
	
	enum Item *item;

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E16_tag_kind_conflict.c line 5: syntax error: illegal referencing struct or union identifier Item near * line 6: syntax error: incomplete forward reference for identifier Item at end

E17. 미완성 구조체

의도 / 기준	미완성 구조체

	AST 출력 없음
입력 코드	`struct Missing *value;`
실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E17_incomplete_struct_reference.c` `line 2: syntax error: incomplete forward reference for identifier Missing at end`

E18. 미완성 나열형

의도 / 기준	미완성 나열형 AST 출력 없음
입력 코드	`enum Missing *value;`
실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E18_incomplete_enum_reference.c` `line 2: syntax error: incomplete forward reference for identifier Missing at end`

E19. 세미콜론 누락

의도 / 기준	세미콜론 누락 AST 출력 없음

입력 코드	`int main()`
	`{`
	`  int value`
	`  return 0;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E19_missing_semicolon.c` `line 4: syntax error near return`

E20. 잘못된 인자 목록

의도 / 기준	잘못된 인자 목록 AST 출력 없음

입력 코드	`int identity(int value)`
	`{`
	`  return value;`
	`}`
	
	`int main()`
	`{`
	`  return identity(1,);`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E20_trailing_argument_comma.c` `line 8: syntax error near )`

2-6. 엣지케이스 검증

빈 블록, 이름 가림, 전방 참조와 복합 선언자 등 엣지케이스 10개를 확인했습니다.

B01. 빈 함수 몸체

의도 / 기준	빈 함수 몸체

입력 코드	`int empty()`
	`{`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/boundary/B01_empty_function_body.c` `=======  syntax tree  ==========` `N_PROGRAM (0x0,0)` `|  (ID="empty") TYPE:0x1015a7430 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND (0x0,0)` `|					N_STMT_LIST_NIL (0x0,0)`

B02. 빈 중첩 블록

의도 / 기준	빈 중첩 블록

입력 코드	`int main()`
	`{`
	`  {`
	`  }`
	`  return 0;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/boundary/B02_empty_nested_block.c` `=======  syntax tree  ==========` `N_PROGRAM (0x0,0)` `|  (ID="main") TYPE:0x104913430 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND (0x0,0)` `|					N_STMT_LIST (0x0,0)` `|						N_STMT_COMPOUND (0x0,0)` `|							N_STMT_LIST_NIL (0x0,0)` `|						N_STMT_LIST (0x0,0)` `|							N_STMT_RETURN (0x0,0)` `|								N_EXP_INT_CONST (0x0,0)` `|									0` `|							N_STMT_LIST_NIL (0x0,0)`

B03. 빈 명령문

의도 / 기준	빈 명령문

입력 코드	`int main()`
	`{`
	`  ;`
	`  return 0;`
	`}`

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/boundary/B03_empty_statement.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="main") TYPE:0x100a3f430 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_EMPTY (0x0,0) | | | | | | | N_STMT_LIST (0x0,0) | | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | | | | | 0 | | | | | | | | N_STMT_LIST_NIL (0x0,0)

B04. 선언만 있는 블록

의도 / 기준	선언만 있는 블록

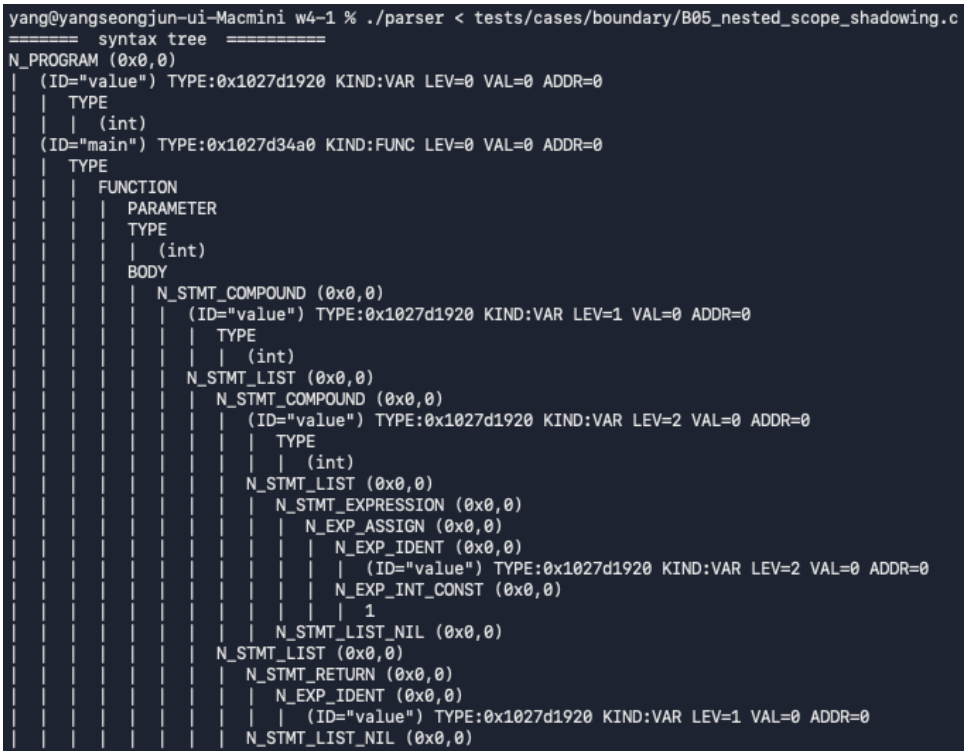
입력 코드	int declarations_only()
	{
	  int value;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/boundary/B04_declaration_only_block.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="declarations_only") TYPE:0x100a83440 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | (ID="value") TYPE:0x100a81920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | N_STMT_LIST_NIL (0x0,0)

B05. 중첩 스코프의 이름 가림

의도 / 기준	중첩 스코프의 이름 가림

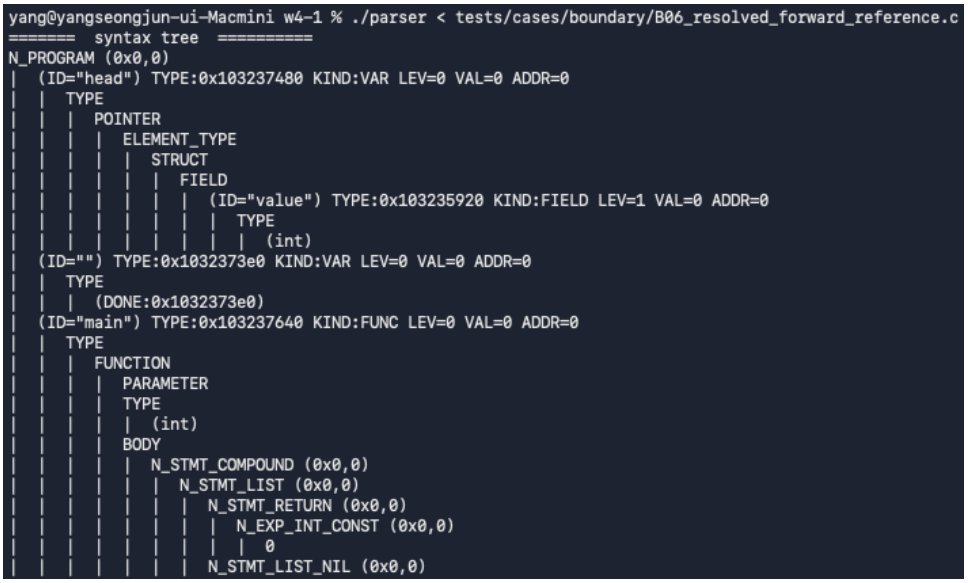
입력 코드	int value;
	
	int main()
	{
	  int value;
	  {
	  int value;
	  value = 1;
	  }
	  return value;
	}

실행 결과	

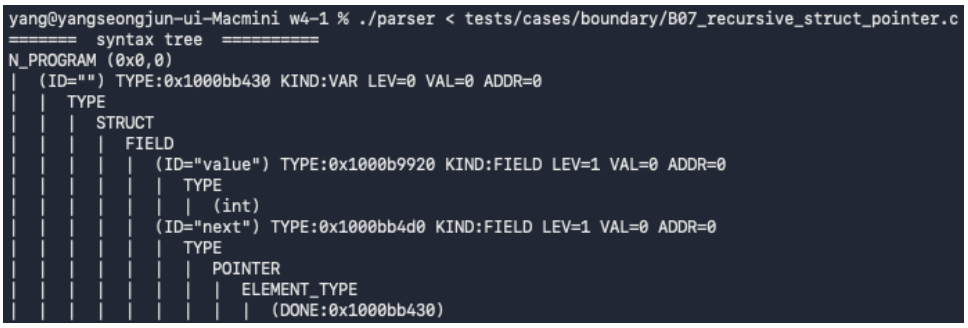
B06. 완성되는 전방 참조

의도 / 기준	완성되는 전방 참조

입력 코드	```
	struct Node *head;
	
	struct Node {
	    int value;
	};
	
	int main()
	{
	    return 0;
	}
	``

실행 결과	

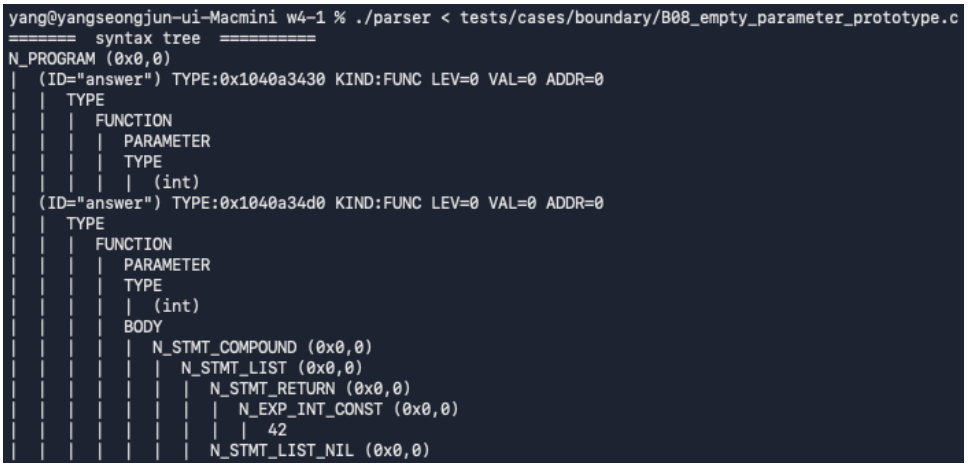
B07. 자기 참조 구조체

의도 / 기준	자기 참조 구조체
입력 코드	```struct Node { int value; struct Node *next; };```
실행 결과	

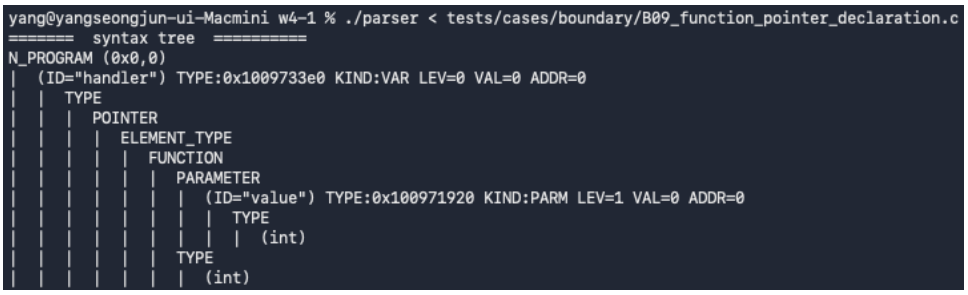
B08. 빈 매개변수 함수 원형

의도 / 기준	빈 매개변수 함수 원형

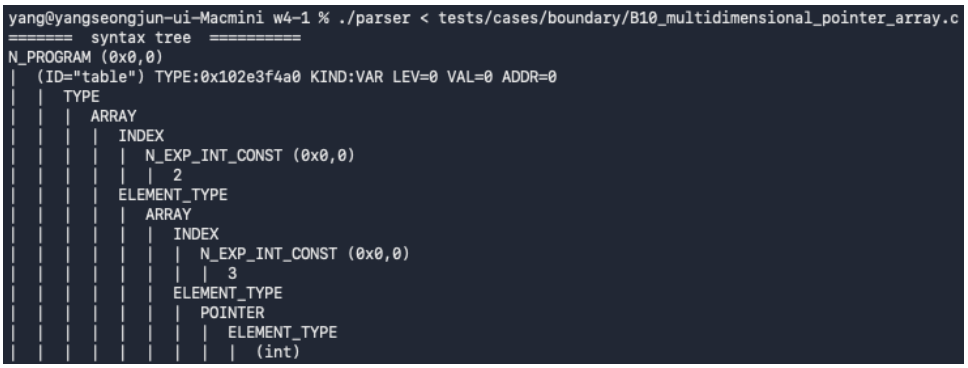
입력 코드	```int answer();
	
	int answer()
	{
	  return 42;
	}```

실행 결과	

B09. 함수 포인터

의도 / 기준	함수 포인터
입력 코드	`int (*handler)(int value);`
실행 결과	

B10. 다차원 포인터 배열

의도 / 기준	다차원 포인터 배열
입력 코드	`int *table[2][3];`
실행 결과	

3. 개발 과정

3-1. 오류 및 해결 기록

01. 초기 빌드: 함수와 전역 변수의 정의 누락

문제 / 원인	개발 당시 makeNode() 등의 함수 구현과 makeType(), int_type의 정의 또는 링크가 누락된 문제입니다.
오류 출력	w4-1/test.c:11:18: warning: comparison between pointer and integer w4-1/support.c:4:28: warning: unused parameter 'name' [-Wunused-paramete] w4-1/support.c:4:42: warning: unused parameter 'left' [-Wunused-paramete] w4-1/support.c:4:56: warning: unused parameter 'center' [-Wunused-parameter] w4-1/support.c:4:72: warning: unused parameter 'right' [-Wunused-parameter] w4-1/support.c:4:80: warning: control reaches end of non-void function [-Wreturn-type] C:/msys64/mingw64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\cc6VZ5qo.o:test.c:(.text+0x13): undefined reference to `makeType' collect2.exe: error: ld returned 1 exit status C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\ccuBiCXf.o:test.c:(.rdata$.refptr.int_type[.refptr.int_type]+0x0): undefined reference to `int_type'
현재 구현 / 상태	함수와 전역 변수의 정의 위치를 정리했습니다. initialize()에서 기본 타입과 내장 함수를 등록합니다.

02. Specifier 처리: NULL 타입의 잘못된 검사

문제 / 원인	타입을 생략한 makeSpecifier(NULL, S_TYPEDEF) 호출을 오류로 처리한 문제입니다. 선언 지정자의 NULL 타입은 기본 타입을 보완하기 전에는 허용되어야 합니다.
오류 출력	Semantic error: NULL type in setTypeField
현재 구현 / 상태	GDB 호출 스택으로 makeSpecifier()의 NULL 처리 경로를 확인했습니다. 타입 생략을 허용하고 setDefaultSpecifier()에서 int_type으로 보완합니다.

03. 이름 없는 식별자 객체 생성

문제 / 원인	개발 당시 makeDummyIdentifier() 구현이 누락되어 이름 없는 식별자를 생성하지 못했던 항목입니다.
오류 출력	/usr/bin/ld: y.tab.o: undefined reference to `makeDummyIdentifier' collect2: error: ld returned 1 exit status
현재 구현 / 상태	name은 빈 문자열, type과 link는 NIL, prev는 0으로 초기화합니다.

04. Lex 오타: yytext와 대입 연산자

문제 / 원인	개발 당시 yytext를 yytest로, 대입 연산자 =를 비교 연산자 ==로 잘못 작성한 문제입니다.
오류 출력	c.y:308:52: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] c.y:308:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] c.y:310:22: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] c.y:311:55: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] c.l:105:18: error: 'yytest' undeclared (first use in this function); did you mean 'yytext'? lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function] lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function]
현재 구현 / 상태	yytest를 yytext로, ==를 =로 바로잡았습니다. 현재 정수 변환에는 atoi()를 사용합니다.
현재 코드	yylval=atoi(yytext); return(INTEGER_CONSTANT);

05. makeNode 중복 정의: 빌드 대상 중복

문제 / 원인	개발 당시 동일한 소스 파일을 두 번 링크하여 makeNode의 중복 정의가 발생한 문제입니다.
오류 출력	multiple definition of `makeNode'; ... support.c:4: first defined here collect2.exe: error: ld returned 1 exit status
현재 구현 / 상태	중복된 빌드 대상을 제거하고 현재 파일 구성에 맞게 정리했습니다.
현재 코드	gcc -o parser y.tab.c lex.yy.c support.c print.c main.c

06. 조건식의 비트 OR와 논리 OR 혼동

문제 / 원인	개발 당시 논리 조건 결합에 || 대신 |를 사용한 문제입니다.

오류 출력	support.c:418:16: warning: suggest parentheses around comparison in operand of '|' [-Wparentheses]
현재 구현 / 상태	조건을 논리 연산자로 결합하도록 수정했습니다. 아래 조건은 선행 선언 a가 있는 경우에 검사합니다.

현재 코드	if (a->kind != ID_FUNC || a->type->expr) {
	  syntax_error(12, id->name);
	}

07. Lex 생성 코드의 미사용 함수 경고

문제 / 원인	Flex가 생성하는 input, yyunput이 사용되지 않을 때 빌드 옵션에 따라 경고가 발생할 수 있는 항목입니다.
오류 출력	lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function] lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function]
현재 구현 / 상태	파서 동작 오류와 구분되는 빌드 경고입니다. 경고 발생 여부는 Flex 생성 코드와 컴파일 옵션에 따라 달라집니다.

08. Yacc 기호명, 함수명과 규칙 경계

문제 / 원인	개발 당시 문법 기호와 함수 이름의 오타, 문법 규칙 경계 누락이 기록된 항목입니다.
오류 출력	w4-1/c.y:270.10-51: error: symbol 'parameter_ttestUpdateSpecifierype_list_opt' is used, but is not defined as a token and has no rules w4-1/c.y:124:36: error: implicit declaration of function 'linkDeclarator'; did you mean 'linkDeclaratorList'? [-Wimplicit-function-declaration] w4-1/c.y:418.1-17: error: unexpected identifier
현재 구현 / 상태	기호명과 함수명의 오타, 규칙 끝의 세미콜론과 액션 경계를 확인했습니다.

09. 중간 액션과 중첩 함수 호출의 괄호

문제 / 원인	개발 당시 중간 액션의 의미값 위치와 중첩 함수 호출의 인수 배치를 혼동한 항목입니다.
오류 출력	w4-1/c.y:459:61: error: too few arguments to function 'setTypeExpr'; expected 2, have 1 w4-1/c.y:459:18: error: too many arguments to function 'setTypeElementType'; expected 2, have 3
현재 구현 / 상태	배열 크기 수식을 setTypeExpr()의 두 번째 인수로 전달하도록 수정했습니다.
현재 코드	$$=setTypeElementType($1,setTypeExpr(makeType(T_ARRAY),$3));

10. 전역 선언 목록과 식별자 출력

문제 / 원인	과거 전역 선언 목록에서 일부 함수의 출력이 누락되었던 항목입니다. 현재 선언 목록 연결 방식과 출력 서식을 구분하여 확인했습니다.
오류 출력	N_PROGRAM (0000000000000000,0) | (ID="main") TYPE:000001fa9b2c1f80 KIND:FUNC LEV=0 VAL=0 ADDR=0
현재 구현 / 상태	식별자의 이름, 타입, 종류와 스코프 정보를 출력합니다. 전역 선언 목록은 linkDeclaratorList()로 연결합니다.

11. 문자 상수의 값 전달 오류

문제 / 원인	과거 문자 상수에서 문자 코드 대신 예상과 다른 큰 정수가 출력되었던 항목입니다.
오류 출력	| | | | | | | | N_EXP_CHAR_CONST ((nil),0) | | | | | | | | | 912013584
현재 구현 / 상태	현재는 문자값을 yylval에 저장하여 'A'가 65로 출력됩니다.

컴파일러 과제5 보고서

20221500 양성준

목차

1. 구현 1
1-1. 구현 환경 1
1-2. 프로그램 구성 1
1-3. 구현 및 변경 사항 2
2. 검증 4
2-1. 테스트 구성 4
2-2. 테스트 결과 4
2-3. 정상 파싱 검증 4
2-4. AST 구조 검증 13
2-5. 오류 검증 23
2-6. 엣지케이스 검증 28
3. 개발 과정 34
3-1. 오류 및 해결 기록 34

1. 구현

1-1. 구현 환경

64비트 UNIX(macOS ARM64) 환경에서 Bison, lex, GCC를 직접 사용해 빌드하고 테스트했습니다.

1-2. 프로그램 구성

교수님 부록 A1~A4, A9를 바탕으로 코드를 작성했습니다.

파일	주요 역할
c.l	토큰 분석, yylval, 줄 번호 처리
c.y	문법 분석, 신택스 트리 생성
support.c / support.h	심볼, 타입, 스코프 관리, 오류 처리, 함수 선언
type.h	자료구조 정의
print.c	신택스 트리 출력
main.c	초기화, 파싱 및 출력 실행

1-3. 구현 및 변경 사항

부록 A1~A4, A9의 문법·자료구조는 유지하고, 과제 범위와 64비트 실행 환경에 맞춰 아래 코드를 수정했습니다.

1-3-1. 실행 흐름 — main.c

컴파일을 위해 의미 분석 및 코드 생성 단계와 a.asm 파일 생성을 제거했습니다. yyparse() 성공 시 신택스 트리만 출력하도록 변경했습니다.

```
extern A_NODE *root;
void print_ast(A_NODE *);
int main() {
  initialize();
  if (yyparse() || syntax_err)
    return (1);
  print_ast(root);
  return (0);
}
```

1-3-2. 64비트 의미값 처리 — c.y, c.l

기본 정수형 YYSTYPE을 사용하지 않고 c.y와 c.l 모두에서 YYSTYPE을 8바이트 long 타입으로 선언했습니다. c.l은 y.tab.h에 선언된 YYSTYPE형 yylval을 사용합니다.

c.y, c.l 선언부

```
#define YYSTYPE_IS_DECLARED 1
typedef long YYSTYPE;
```

1-3-3. 함수 프로토타입 공유 — support.h

사진 원본 함수 원형이 support.c 내부에만 있어 c.y에서는 이를 참조할 수 없습니다.

c.y, support.c

```
#include "support.h"
```

위와 같이 수정했습니다.

1-3-4. 불필요한 코드 제거

1. 미사용 선언 및 헤더 제거

```
char *t;                // c.l: checkIdentifier()
A_TYPE *t;              // support.c: setDefaultSpecifier()
#include "y.tab.h"      // support.c
```

checkIdentifier()의 char *t와 setDefaultSpecifier()의 A_TYPE *t는 선언 후 한 번도 참조되지 않았습니다. 또한 support.c에 필요한 자료형과 함수 선언은 support.h와 type.h에서 제공되므로 y.tab.h를 포함할 필요가 없습니다.

2. 미사용 상태 제거

```
t->local_var_size = 0;
t->check = FALSE;
int_type->size = 4;
int_type->check = TRUE;
float_type->size = 4;
float_type->check = TRUE;
char_type->check = TRUE;
void_type->size = 0;
void_type->check = TRUE;
string_type->size = 4;
string_type->check = TRUE;
```

local_var_size, check와 위 size 값은 이후 의미분석 단계에서 사용하는 정보입니다. 현재 프로그램은 구문분석과 신택스 트리 출력까지만 수행하며 이 값을 읽지 않으므로 제거했습니다.

3. 행 번호 저장문 제거

```
m->line = line_no;      // makeNode(), makeNodeList()
id->line = line_no;     // makeIdentifier(), makeDummyIdentifier()
t->line = line_no;      // makeType()
p->line = line_no;      // makeSpecifier()
```

각 자료구조의 line 필드에 저장한 값은 이후 읽는 곳이 없습니다. 오류 메시지는 어휘분석기가 갱신하는 전역 line_no를 직접 사용하므로, 객체별 line 저장문 6개를 제거해도 오류 행 번호 출력에는 영향이 없습니다.

2. 검증

2-1. 테스트 구성

구문분석기의 기능과 출력 결과를 검증하기 위해 **총 64개의 테스트**를 작성했습니다. 정상 파싱 19개, 신택스 트리 구조 15개, 오류 처리 20개, 엣지케이스 10개로 구성했습니다.

그룹	검증 목적
정상 파싱	정상 선언, 함수, 제어문 등의 파싱 검증
AST	노드 깊이, 전위 순서, 연산자 우선순위 비교
오류 처리	yyerror(), syntax_error() 출력과 종료 코드 확인
엣지케이스	빈 블록, 이름 가림, 전방 참조, 복합 선언자 검증

2-2. 테스트 결과

테스트 실행 결과는 다음과 같습니다.

그룹	통과	실패
정상 파싱	19	0
AST	15	0
오류 처리	20	0
엣지케이스	10	0
합계	64	0

전체 64개 테스트가 통과했고 **실패는 0개**였습니다.

2-3. 정상 파싱 검증

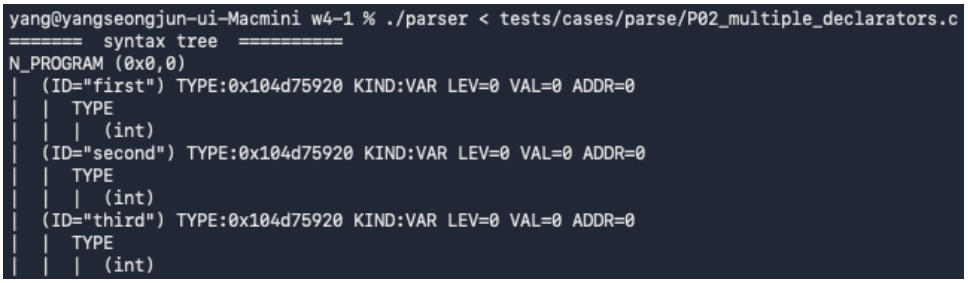
변수 선언, 함수 정의, 구조체와 제어문 등 정상 입력 19개의 파싱과 신택스 트리 출력을 확인했습니다.

P01. 전역 변수 선언

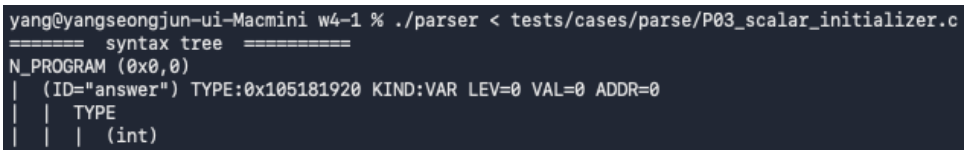
의도 / 기준	전역 변수 선언
입력 코드	`int global_value;`
실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P01_global_variable.c` `=======  syntax tree  ==========` `N_PROGRAM (0x0,0)` `|  (ID="global_value") TYPE:0x100371920 KIND:VAR LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		(int)`

P02. 다중 선언자

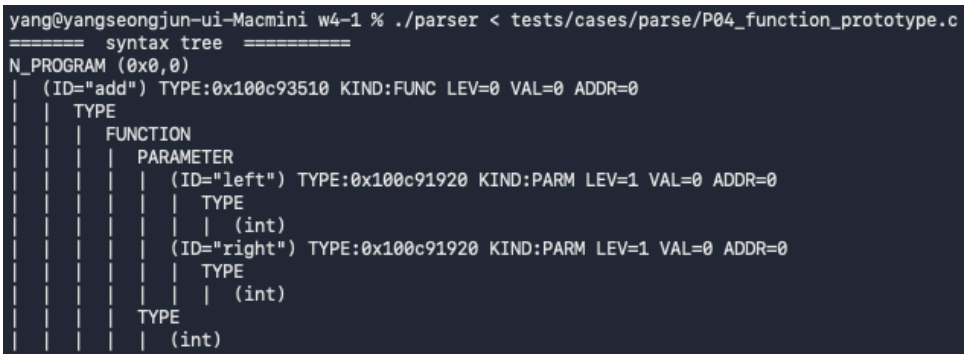
의도 / 기준	다중 선언자
입력 코드	`int first, second, third;`

실행 결과	

P03. 정수 초기화 선언

의도 / 기준	정수 초기화 선언
입력 코드	`int answer = 42;`
실행 결과	

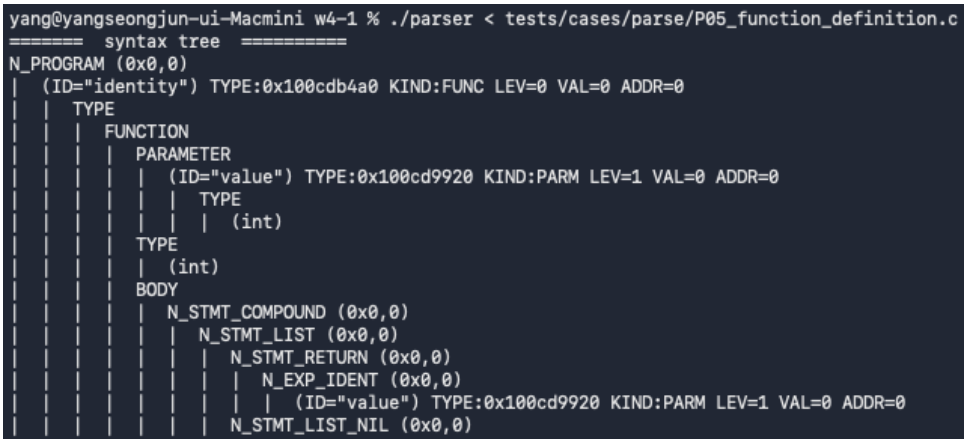
P04. 함수 원형 선언

의도 / 기준	함수 원형 선언
입력 코드	`int add(int left, int right);`
실행 결과	

P05. 함수 정의

의도 / 기준	함수 정의

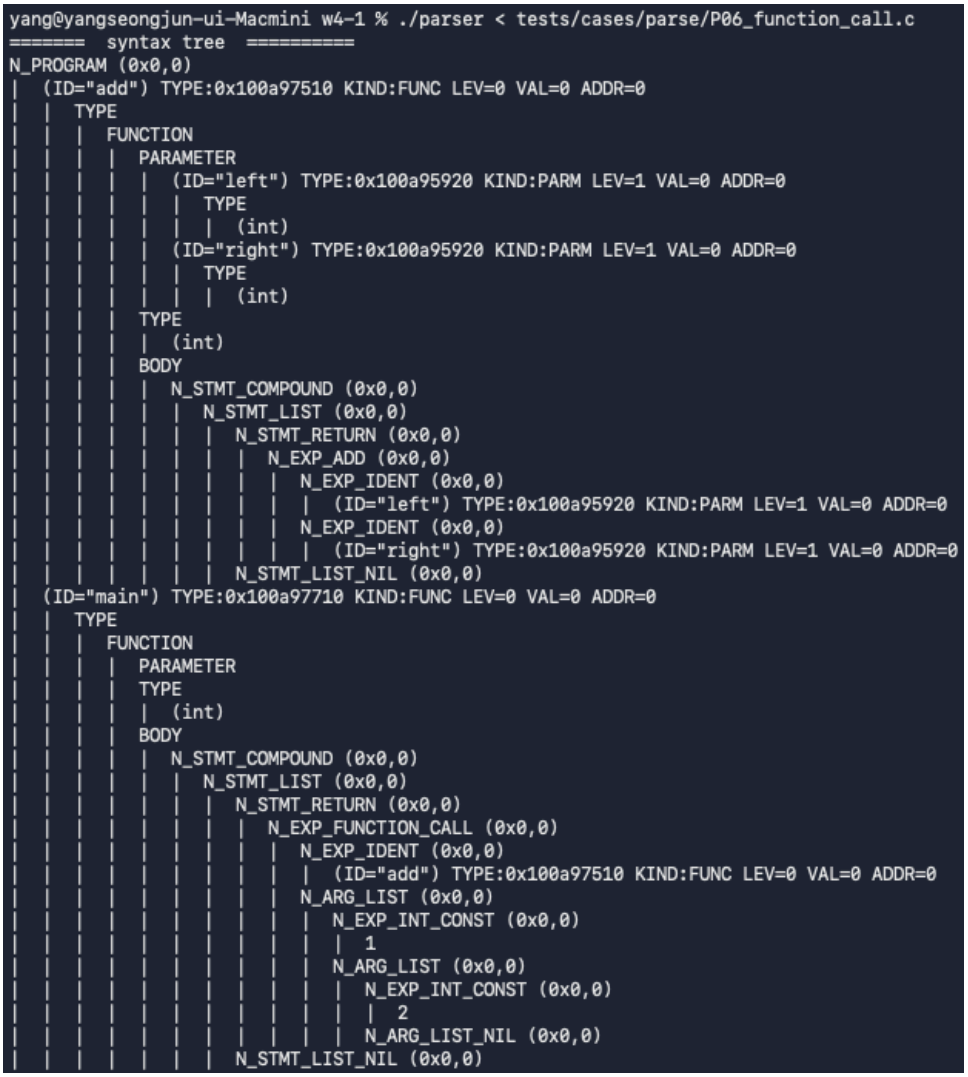
입력 코드	`int identity(int value)`
	`{`
	`  return value;`
	`}`

실행 결과	

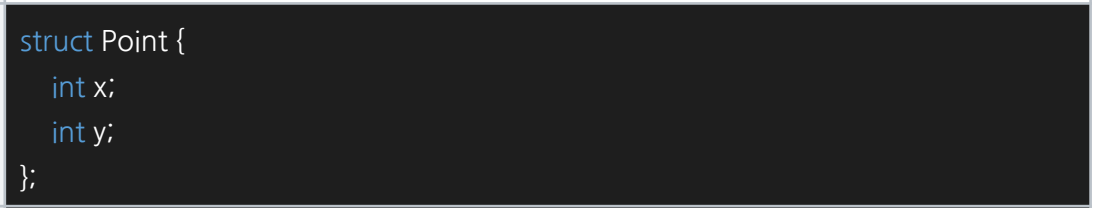
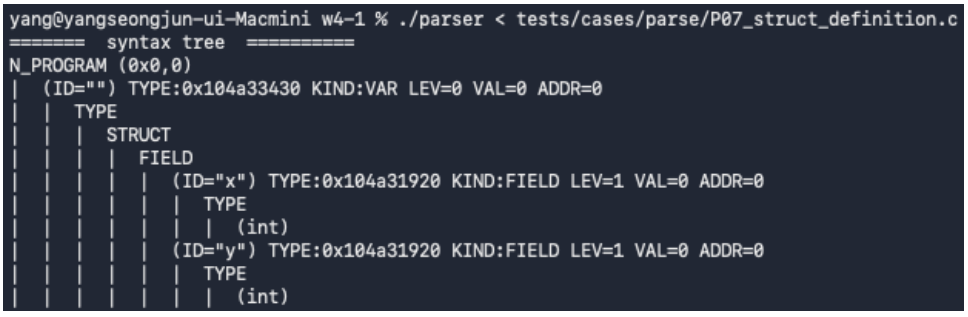
P06. 함수 호출

의도 / 기준	함수 호출

입력 코드	```int add(int left, int right)
	{
	  return left + right;
	}
	
	int main()
	{
	  return add(1, 2);
	}```

실행 결과	

P07. 구조체 정의

의도 / 기준	구조체 정의
입력 코드	
실행 결과	

P09. 나열형 정의

의도 / 기준	나열형 정의

입력 코드	enum Color {
	  RED,
	  GREEN,
	  BLUE
	};

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P09_enum_definition.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="") TYPE:0x103033430 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | ENUM | | | | ENUMERATORS | | | | | (ID="RED") TYPE:0x0 KIND:ENUM_LITERAL LEV=0 VAL=0 ADDR=0 | | | | | (ID="GREEN") TYPE:0x0 KIND:ENUM_LITERAL LEV=0 VAL=0 ADDR=0 | | | | | (ID="BLUE") TYPE:0x0 KIND:ENUM_LITERAL LEV=0 VAL=0 ADDR=0

P10. 배열 선언

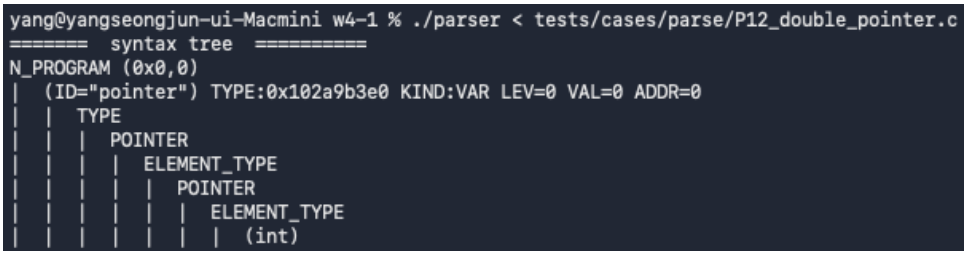
의도 / 기준	배열 선언
입력 코드	int values[4];
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P10_array_declaration.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="values") TYPE:0x100ed3460 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | ARRAY | | | | INDEX | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | 4 | | | | ELEMENT_TYPE | | | | | (int)

P11. 포인터 선언

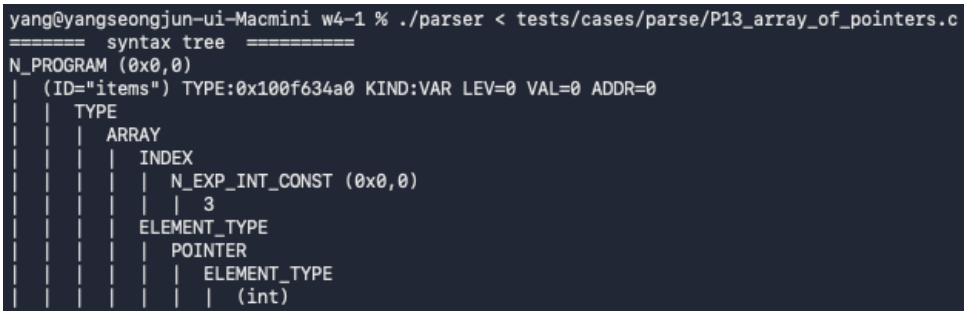
의도 / 기준	포인터 선언
입력 코드	int *pointer;
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P11_pointer_declaration.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="pointer") TYPE:0x1010633e0 KIND:VAR LEV=0 VAL=0 ADDR=0 | | TYPE | | | POINTER | | | | ELEMENT_TYPE | | | | | (int)

P12. 이중 포인터 선언

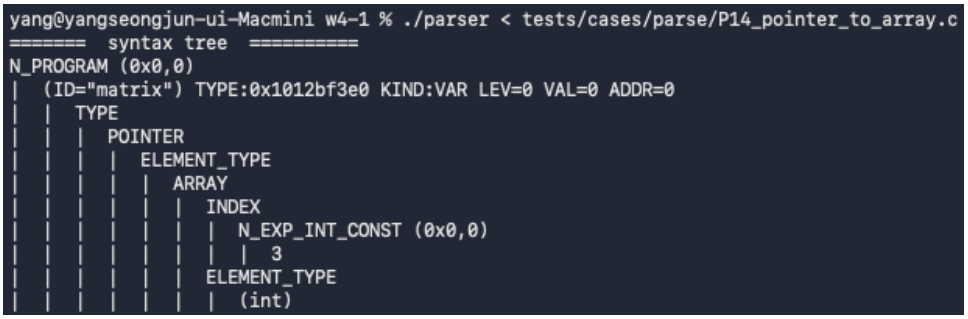
의도 / 기준	이중 포인터 선언
입력 코드	int **pointer;

실행 결과	

P13. 포인터 배열

의도 / 기준	포인터 배열
입력 코드	`int *items[3];`
실행 결과	

P14. 배열 포인터 선언

의도 / 기준	배열 포인터 선언
입력 코드	`int (*matrix)[3];`
실행 결과	

P15. if-else

의도 / 기준	if-else

입력 코드	```int choose(int condition)
	{
	  if (condition)
	  return 1;
	  else
	  return 0;
	}```

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P15_if_else.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="choose") TYPE:0x1012c34a0 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="condition") TYPE:0x1012c1920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  |  |  |  |  TYPE
|  |  |  |  |  |  (int)
|  |  |  |  |  BODY
|  |  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  |  N_STMT_IF_ELSE (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  (ID="condition") TYPE:0x1012c1920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  0
|  |  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

P16. while

의도 / 기준	while
입력 코드	

```c
int count_down(int value)
{
    while (value)
        value = value - 1;
    return value;
}
```

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P16_while_loop.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="count_down") TYPE:0x1017ab4a0 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  |  |  |  |  TYPE
|  |  |  |  |  |  (int)
|  |  |  |  |  BODY
|  |  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  |  N_STMT_WHILE (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  N_STMT_EXPRESSION (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_ASSIGN (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  |  |  N_EXP_SUB (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1017a9920 KIND:PARM LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

P17. do-while

의도 / 기준	do-while

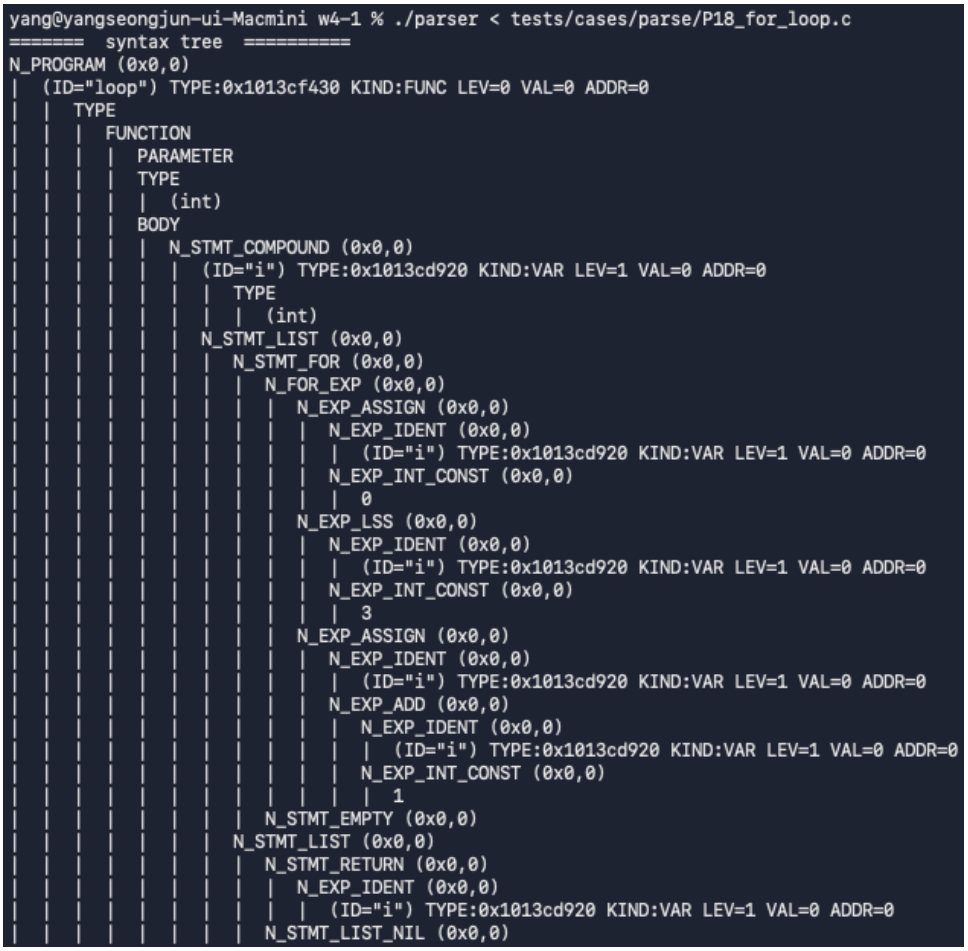
입력 코드	int count_once(int value)
	{
	  do
	  value = value - 1;
	  while (value);
	  return value;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/parse/P17_do_while_loop.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="count_once") TYPE:0x1033d34a0 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (int) | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_DO (0x0,0) | | | | | | | | N_STMT_EXPRESSION (0x0,0) | | | | | | | | | N_EXP_ASSIGN (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | | | N_EXP_SUB (0x0,0) | | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | | | | | | | 1 | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | N_STMT_LIST (0x0,0) | | | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="value") TYPE:0x1033d1920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_STMT_LIST_NIL (0x0,0)

P18. for

의도 / 기준	for

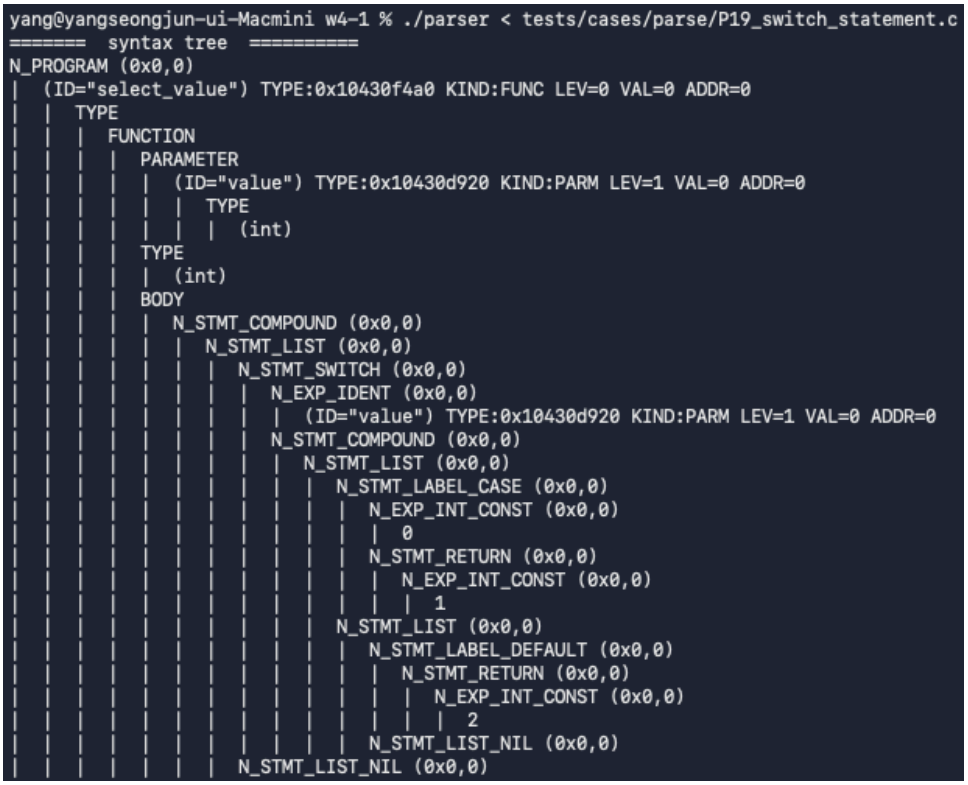
입력 코드	int loop()
	{
	  int i;
	  for (i = 0; i < 3; i = i + 1)
	  ;
	  return i;
	}

실행 결과	

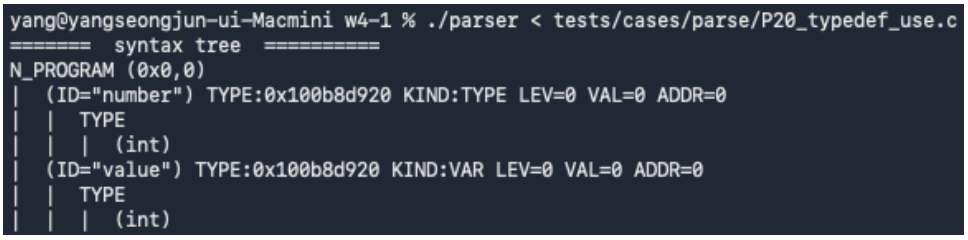
P19. switch와 레이블

의도 / 기준	switch와 레이블

입력 코드	```int select_value(int value)
	{
	  switch (value) {
	  case 0:
	  return 1;
	  default:
	  return 2;
	  }
	}```

실행 결과	

P20. typedef 사용

의도 / 기준	typedef 사용
입력 코드	`typedef int number;` `number value;`
실행 결과	

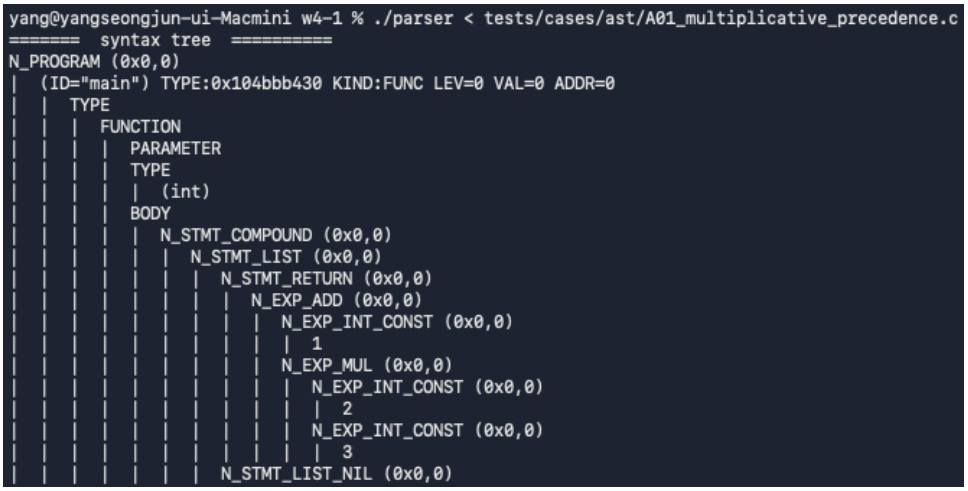
2-4. AST 구조 검증

연산자 우선순위, 노드 깊이와 자식 연결 순서를 AST 테스트 15개로 확인했습니다.

A01. 곱셈 우선순위

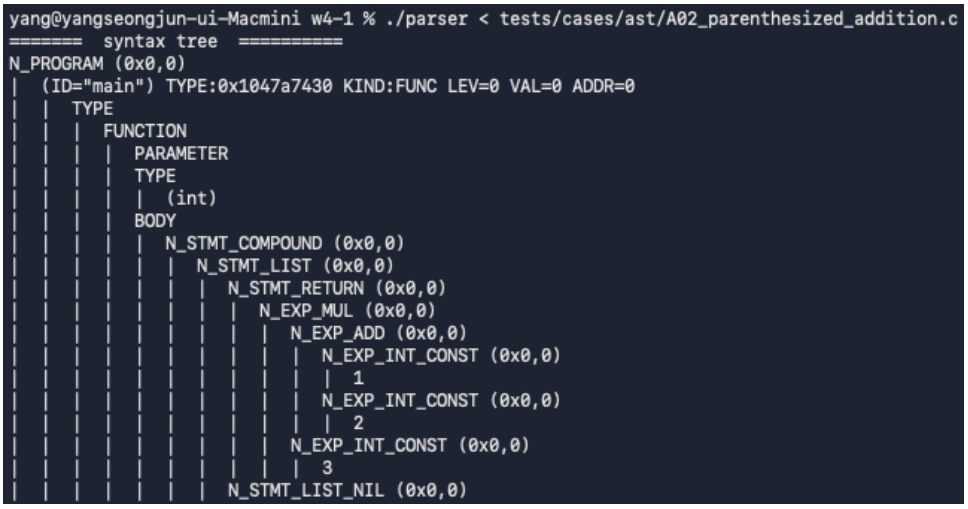
의도 / 기준	곱셈 우선순위

입력 코드	`int main()`
	`{`
	`  return 1 + 2 * 3;`
	`}`

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A01_multiplicative_precedence.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="main") TYPE:0x104bbb430 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  N_EXP_ADD (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  |  N_EXP_MUL (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  2
|  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  3
|  |  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

A02. 괄호 우선순위

의도 / 기준	괄호 우선순위
입력 코드	
실행 결과	

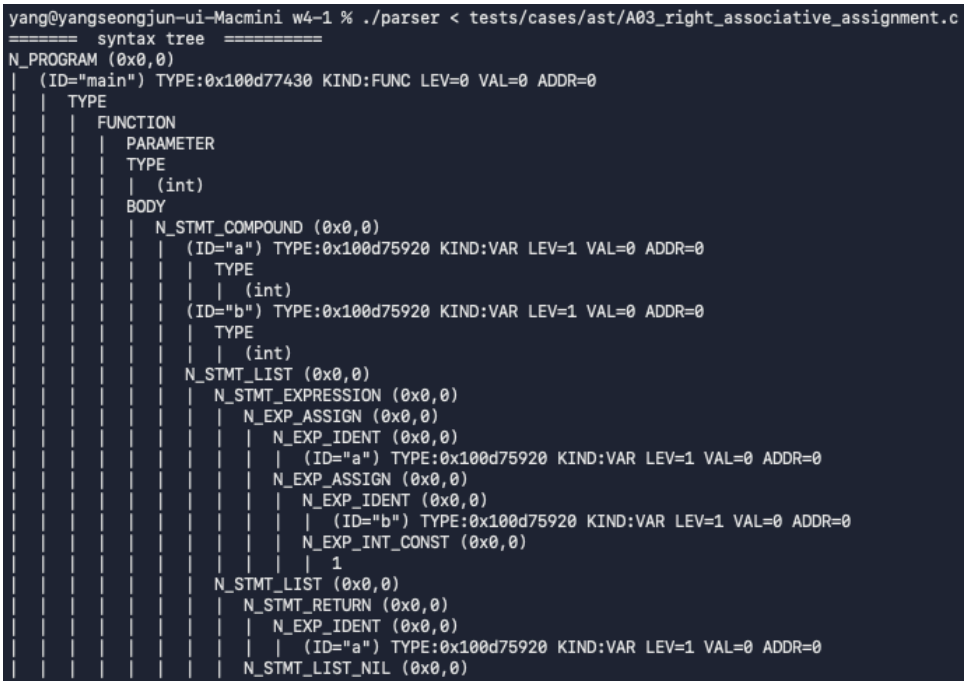
```
int main()
{
   return (1 + 2) * 3;
}
```

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A02_parenthesized_addition.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="main") TYPE:0x1047a7430 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  N_EXP_MUL (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_ADD (0x0,0)
|  |  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  1
|  |  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  |  2
|  |  |  |  |  |  |  |  N_EXP_INT_CONST (0x0,0)
|  |  |  |  |  |  |  |  |  |  3
|  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

A03. 대입 결합 순서

의도 / 기준	대입 결합 순서
입력 코드	

```
int main()
{
   int a;
   int b;
   a = b = 1;
   return a;
}
```

실행 결과	

A04. 논리 연산자 우선순위

의도 / 기준	논리 연산자 우선순위

입력 코드	```int main()
	{
	  int a;
	  int b;
	  int c;
	  return a || b && c;
	}```

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A04_logical_precedence.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="main") TYPE:0x100e17430 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | (ID="a") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | (ID="b") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | (ID="c") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | N_EXP_OR (0x0,0) | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | (ID="a") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_EXP_AND (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="b") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="c") TYPE:0x100e15920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | N_STMT_LIST_NIL (0x0,0)

A05. 관계 및 동등 연산자

의도 / 기준	관계 및 동등 연산자

입력 코드	int main()
	{
	  int a;
	  int b;
	  int c;
	  return a == b < c;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A05_equality_relational_precedence.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="main") TYPE:0x10149f430 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | (ID="a") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | (ID="b") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | (ID="c") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | N_EXP_EQL (0x0,0) | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | (ID="a") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_EXP_LSS (0x0,0) | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="b") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | (ID="c") TYPE:0x10149d920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | N_STMT_LIST_NIL (0x0,0)

A06. 중첩 단항 연산

의도 / 기준	중첩 단항 연산

입력 코드	`int main()`
	`{`
	`  int value;`
	`  return !-value;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A06_nested_unary.c`
	`=======  syntax tree  ==========`
	`N_PROGRAM (0x0,0)`
	`|  (ID="main") TYPE:0x1013c7430 KIND:FUNC LEV=0 VAL=0 ADDR=0`
	`|  |  TYPE`
	`|  |  |  FUNCTION`
	`|  |  |  |  PARAMETER`
	`|  |  |  |  TYPE`
	`|  |  |  |  |  (int)`
	`|  |  |  |  BODY`
	`|  |  |  |  |  N_STMT_COMPOUND (0x0,0)`
	`|  |  |  |  |  |  (ID="value") TYPE:0x1013c5920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  |  TYPE`
	`|  |  |  |  |  |  |  |  (int)`
	`|  |  |  |  |  |  N_STMT_LIST (0x0,0)`
	`|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)`
	`|  |  |  |  |  |  |  |  N_EXP_NOT (0x0,0)`
	`|  |  |  |  |  |  |  |  |  N_EXP_MINUS (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x1013c5920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)`

A07. 전위와 후위 증감

의도 / 기준	전위와 후위 증감

입력 코드	`int main()`
	`{`
	`  int value;`
	`  return ++value + value--;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A07_pre_and_post_increment.c`
	`=======  syntax tree  ==========`
	`N_PROGRAM (0x0,0)`
	`|  (ID="main") TYPE:0x10314f430 KIND:FUNC LEV=0 VAL=0 ADDR=0`
	`|  |  TYPE`
	`|  |  |  FUNCTION`
	`|  |  |  |  PARAMETER`
	`|  |  |  |  TYPE`
	`|  |  |  |  |  (int)`
	`|  |  |  |  BODY`
	`|  |  |  |  |  N_STMT_COMPOUND (0x0,0)`
	`|  |  |  |  |  |  (ID="value") TYPE:0x10314d920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  |  TYPE`
	`|  |  |  |  |  |  |  |  (int)`
	`|  |  |  |  |  |  N_STMT_LIST (0x0,0)`
	`|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)`
	`|  |  |  |  |  |  |  |  N_EXP_ADD (0x0,0)`
	`|  |  |  |  |  |  |  |  |  N_EXP_PRE_INC (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x10314d920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  |  |  |  N_EXP_POST_DEC (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)`
	`|  |  |  |  |  |  |  |  |  |  |  (ID="value") TYPE:0x10314d920 KIND:VAR LEV=1 VAL=0 ADDR=0`
	`|  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)`

A08. 함수 호출 인자

의도 / 기준	함수 호출 인자

입력 코드	`int add(int left, int right)`
	`{`
	`  return left + right;`
	`}`
	
	`int main()`
	`{`
	`  return add(1, 2);`
	`}`

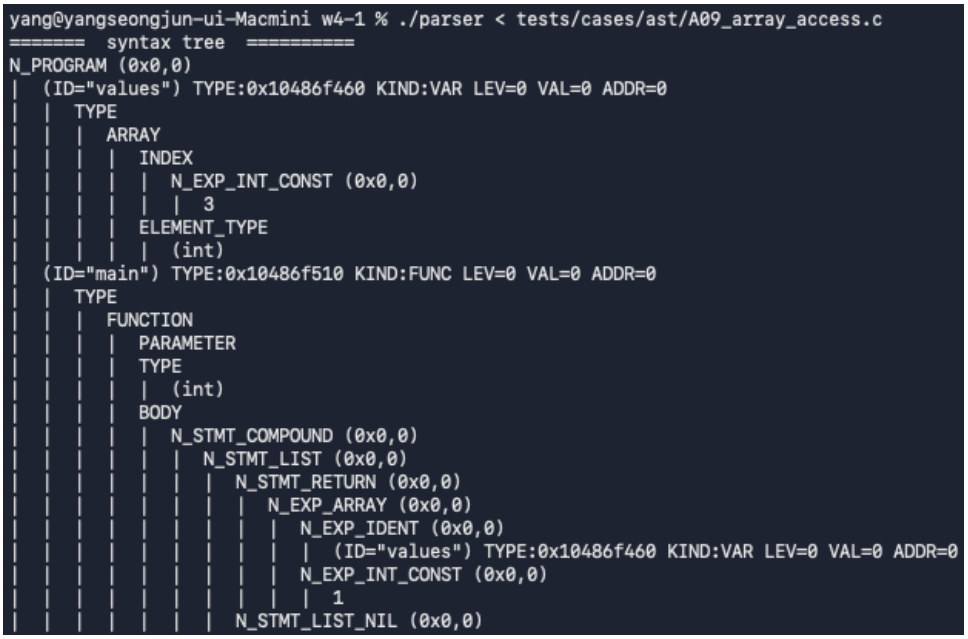
실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A08_function_call_arguments.c` `=======  syntax tree  ==========` `N_PROGRAM (0x0,0)` `|  (ID="add") TYPE:0x105027510 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|				(ID="left") TYPE:0x105025920 KIND:PARM LEV=1 VAL=0 ADDR=0` `|					TYPE` `|						(int)` `|				(ID="right") TYPE:0x105025920 KIND:PARM LEV=1 VAL=0 ADDR=0` `|					TYPE` `|						(int)` `|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND (0x0,0)` `|					N_STMT_LIST (0x0,0)` `|						N_STMT_RETURN (0x0,0)` `|							N_EXP_ADD (0x0,0)` `|								N_EXP_IDENT (0x0,0)` `|									(ID="left") TYPE:0x105025920 KIND:PARM LEV=1 VAL=0 ADDR=0` `|								N_EXP_IDENT (0x0,0)` `|									(ID="right") TYPE:0x105025920 KIND:PARM LEV=1 VAL=0 ADDR=0` `|					N_STMT_LIST_NIL (0x0,0)` `| (ID="main") TYPE:0x105027710 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND (0x0,0)` `|					N_STMT_LIST (0x0,0)` `|						N_STMT_RETURN (0x0,0)` `|							N_EXP_FUNCTION_CALL (0x0,0)` `|								N_EXP_IDENT (0x0,0)` `|									(ID="add") TYPE:0x105027510 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|								N_ARG_LIST (0x0,0)` `|									N_EXP_INT_CONST (0x0,0)` `|										1` `|								N_ARG_LIST (0x0,0)` `|									N_EXP_INT_CONST (0x0,0)` `|											2` `|									N_ARG_LIST_NIL (0x0,0)` `|						N_STMT_LIST_NIL (0x0,0)`

A09. 배열 접근

의도 / 기준	배열 접근

입력 코드	`int values[3];`
	
	`int main()`

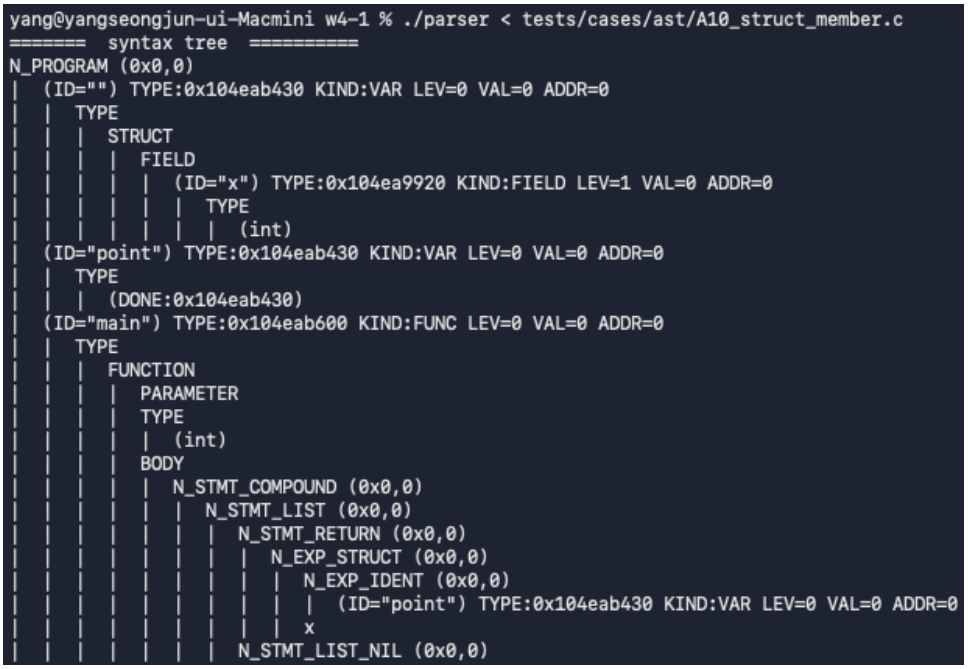
	{
	  return values[1];
	}

실행 결과	

A10. 구조체 멤버

의도 / 기준	구조체 멤버

입력 코드	struct Point {
	  int x;
	};
	
	struct Point point;
	
	int main()
	{
	  return point.x;
	}

실행 결과	

```
yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A10_struct_member.c
=======  syntax tree  ==========
N_PROGRAM (0x0,0)
|  (ID="") TYPE:0x104eab430 KIND:VAR LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  STRUCT
|  |  |  |  FIELD
|  |  |  |  |  (ID="x") TYPE:0x104ea9920 KIND:FIELD LEV=1 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  (ID="point") TYPE:0x104eab430 KIND:VAR LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  (DONE:0x104eab430)
|  (ID="main") TYPE:0x104eab600 KIND:FUNC LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND (0x0,0)
|  |  |  |  |  |  N_STMT_LIST (0x0,0)
|  |  |  |  |  |  |  N_STMT_RETURN (0x0,0)
|  |  |  |  |  |  |  |  N_EXP_STRUCT (0x0,0)
|  |  |  |  |  |  |  |  |  N_EXP_IDENT (0x0,0)
|  |  |  |  |  |  |  |  |  |  (ID="point") TYPE:0x104eab430 KIND:VAR LEV=0 VAL=0 ADDR=0
|  |  |  |  |  |  |  |  |  |  x
|  |  |  |  |  |  |  N_STMT_LIST_NIL (0x0,0)
```

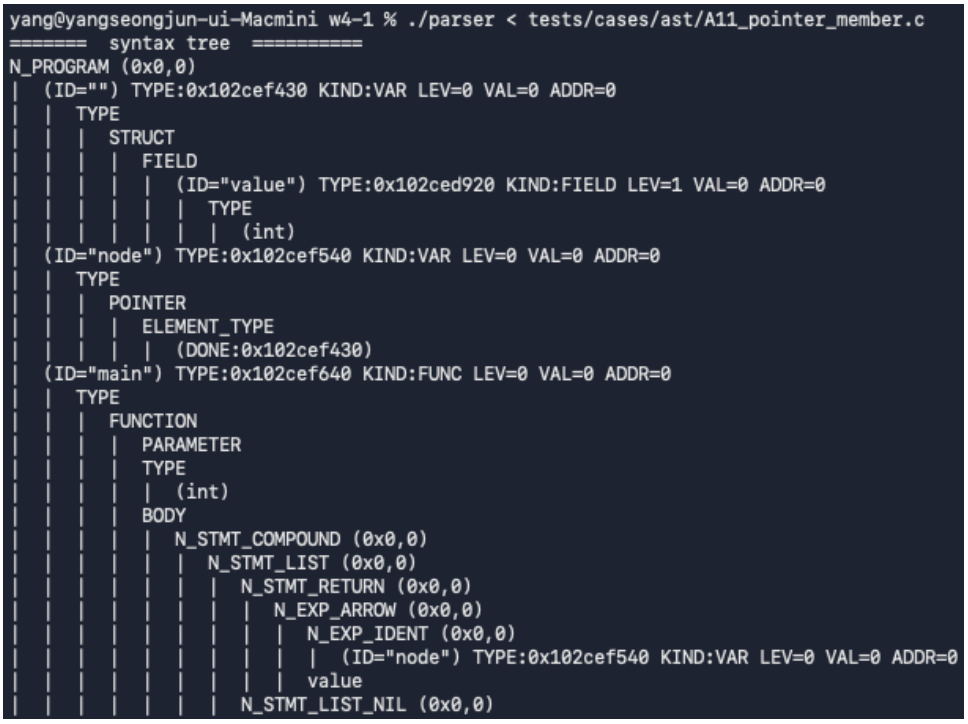
A11. 포인터 멤버

의도 / 기준	포인터 멤버
입력 코드	

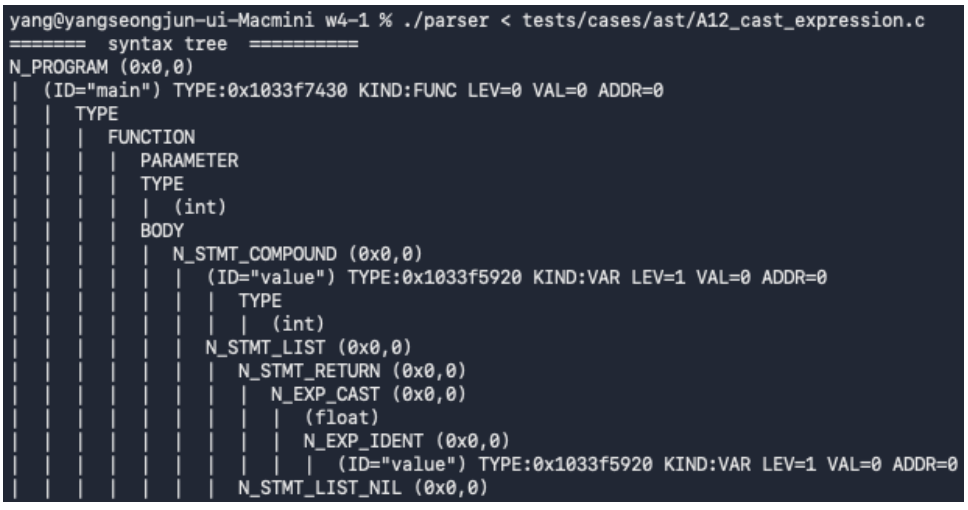
```c
struct Node {
    int value;
};

struct Node *node;

int main()
{
    return node->value;
}
```

실행 결과	

A12. 형 변환

의도 / 기준	형 변환
입력 코드	
실행 결과	

A13. sizeof

의도 / 기준	sizeof
입력 코드	

	int value;
	return sizeof value + sizeof(int);
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A13_sizeof_forms.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="main") TYPE:0x102b6b430 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | (ID="value") TYPE:0x102b69920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | | N_STMT_LIST (0x0,0) | | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | | N_EXP_ADD (0x0,0) | | | | | | | | | | N_EXP_SIZE_EXP (0x0,0) | | | | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | | | | (ID="value") TYPE:0x102b69920 KIND:VAR LEV=1 VAL=0 ADDR=0 | | | | | | | | | | N_EXP_SIZE_TYPE (0x0,0) | | | | | | | | | | | (int) | | | | | | | N_STMT_LIST_NIL (0x0,0)

A14. if-else 구조

의도 / 기준	if-else 구조

입력 코드	int choose(int condition)
	{
	if (condition)
	return 1;
	else
	return 2;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A14_if_else_children.c ======= syntax tree ========== N_PROGRAM (0x0,0) | (ID="choose") TYPE:0x102bf74a0 KIND:FUNC LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="condition") TYPE:0x102bf5920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (int) | | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0x0,0) | | | | | | N_STMT_LIST (0x0,0) | | | | | | | N_STMT_IF_ELSE (0x0,0) | | | | | | | | N_EXP_IDENT (0x0,0) | | | | | | | | | (ID="condition") TYPE:0x102bf5920 KIND:PARM LEV=1 VAL=0 ADDR=0 | | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | | | | | 1 | | | | | | | | N_STMT_RETURN (0x0,0) | | | | | | | | | N_EXP_INT_CONST (0x0,0) | | | | | | | | | | 2 | | | | | | | N_STMT_LIST_NIL (0x0,0)

A15. for의 수식

의도 / 기준	for의 수식

입력 코드	`int main()`
	`{`
	`  int i;`
	`  for (i = 0; i < 2; i = i + 1)`
	`  ;`
	`  return i;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/ast/A15_for_expression_children.c` `=======  syntax tree  ==========` `N_PROGRAM (0x0,0)` `|  (ID="main") TYPE:0x104bc3430 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND (0x0,0)` `|					(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|						TYPE` `|							(int)` `|					N_STMT_LIST (0x0,0)` `|						N_STMT_FOR (0x0,0)` `|							N_FOR_EXP (0x0,0)` `|								N_EXP_ASSIGN (0x0,0)` `|									N_EXP_IDENT (0x0,0)` `|										(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|									N_EXP_INT_CONST (0x0,0)` `|										0` `|								N_EXP_LSS (0x0,0)` `|									N_EXP_IDENT (0x0,0)` `|										(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|									N_EXP_INT_CONST (0x0,0)` `|										2` `|								N_EXP_ASSIGN (0x0,0)` `|									N_EXP_IDENT (0x0,0)` `|										(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|									N_EXP_ADD (0x0,0)` `|										N_EXP_IDENT (0x0,0)` `|											(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|										N_EXP_INT_CONST (0x0,0)` `|											1` `|								N_STMT_EMPTY (0x0,0)` `|							N_STMT_LIST (0x0,0)` `|								N_STMT_RETURN (0x0,0)` `|									N_EXP_IDENT (0x0,0)` `|										(ID="i") TYPE:0x104bc1920 KIND:VAR LEV=1 VAL=0 ADDR=0` `|							N_STMT_LIST_NIL (0x0,0)`

2-5. 오류 검증

잘못된 선언과 문법을 포함한 입력 20개에 대해 오류 메시지와 신택스 트리 출력 여부를 확인했습니다.

E01. 전역 변수 중복

의도 / 기준	전역 변수 중복 AST 출력 없음
입력 코드	`int value;` `int value;`
실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E01_duplicate_global.c` `line 2: syntax error: redeclaration of identifier value near ;`

E02. 지역 변수 중복

의도 / 기준	지역 변수 중복 AST 출력 없음

입력 코드	`int main()`
	`{`
	`  int value;`
	`  int value;`
	`  return 0;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E02_duplicate_local.c` `line 4: syntax error: redeclaration of identifier value near ;`

E03. 매개변수 중복

의도 / 기준	매개변수 중복 AST 출력 없음

입력 코드	`int compare(int value, int value)`
	`{`
	`  return value;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E03_duplicate_parameter.c` `line 1: syntax error: redeclaration of identifier value near )`

E04. 구조체 필드 중복

의도 / 기준	구조체 필드 중복 AST 출력 없음

입력 코드	`struct Point {`
	`  int x;`
	`  int x;`
	`};`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E04_duplicate_struct_field.c` `line 3: syntax error: redeclaration of identifier x near ;`

E05. 미선언 식별자

의도 / 기준	미선언 식별자 AST 출력 없음

입력 코드	`int main()`
	`{`
	`  return missing;`
	`}`

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E05_undeclared_identifier.c line 3: syntax error: undefined identifier missing near missing

E06. 미선언 함수

의도 / 기준	미선언 함수 AST 출력 없음

입력 코드	int main()
	{
	  return missing(1);
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E06_undeclared_function.c line 3: syntax error: undefined identifier missing near missing

E07. void 매개변수

의도 / 기준	void 매개변수 AST 출력 없음

입력 코드	int invalid(void value)
	{
	  return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E07_named_void_parameter.c line 1: syntax error: illegal type specifier in formal parameter near)

E08. 매개변수 저장 클래스

의도 / 기준	매개변수 저장 클래스 AST 출력 없음

입력 코드	int invalid(static int value)
	{
	  return value;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E08_parameter_storage_class.c line 1: syntax error: illegal type specifier in formal parameter near)

E09. 형 변환의 저장 클래스

의도 / 기준	형 변환의 저장 클래스 AST 출력 없음
입력 코드	int main() { int value; return (static int)value;

	} 실행 결과 yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E09_storage_class_in_cast.c line 4: syntax error: illegal storage class in type specifiers near)

E10. 잘못된 함수 선언자

의도 / 기준	잘못된 함수 선언자 AST 출력 없음

입력 코드	int value[1]
	{
	return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E10_illegal_function_declarator.c line 2: syntax error: illegal function declarator near { line 5: syntax error: undefined type for identifier value at end

E11. 원형의 매개변수 타입 충돌

의도 / 기준	원형의 매개변수 타입 충돌 AST 출력 없음

입력 코드	int calculate(int value);
	
	int calculate(float value)
	{
	return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E11_parameter_type_conflict.c line 4: syntax error: conflicting parm type in prototype function calculate near {

E12. 이름 없는 정의 매개변수

의도 / 기준	이름 없는 정의 매개변수 AST 출력 없음

입력 코드	int unnamed(int)
	{
	return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E12_unnamed_definition_parameter.c line 2: syntax error: empty parameter name near {

E13. 충돌하는 타입 명시자

의도 / 기준	충돌하는 타입 명시자 AST 출력 없음

입력 코드	int float value;
실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E13_conflicting_type_specifiers.c line 1: syntax error: illegal declaration specifiers near value

E14. 함수 저장 클래스

의도 / 기준	함수 저장 클래스 AST 출력 없음

입력 코드	static int main()
	{
	  return 0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E14_function_storage_class.c line 2: syntax error: illegal function specifiers near {

E15. 반환 타입 충돌

의도 / 기준	반환 타입 충돌 AST 출력 없음

입력 코드	int convert(int value);
	
	float convert(int value)
	{
	  return 0.0;
	}

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E15_return_type_conflict.c line 4: syntax error: illegal or conflicting return type in function convert near {

E16. 태그 종류 충돌

의도 / 기준	태그 종류 충돌 AST 출력 없음

입력 코드	struct Item {
	  int value;
	};
	
	enum Item *item;

실행 결과	yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E16_tag_kind_conflict.c line 5: syntax error: illegal referencing struct or union identifier Item near * line 6: syntax error: incomplete forward reference for identifier Item at end

E17. 미완성 구조체

의도 / 기준	미완성 구조체

	AST 출력 없음
입력 코드	`struct Missing *value;`
실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E17_incomplete_struct_reference.c` `line 2: syntax error: incomplete forward reference for identifier Missing at end`

E18. 미완성 나열형

의도 / 기준	미완성 나열형 AST 출력 없음
입력 코드	`enum Missing *value;`
실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E18_incomplete_enum_reference.c` `line 2: syntax error: incomplete forward reference for identifier Missing at end`

E19. 세미콜론 누락

의도 / 기준	세미콜론 누락 AST 출력 없음

입력 코드	`int main()`
	`{`
	`  int value`
	`  return 0;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E19_missing_semicolon.c` `line 4: syntax error near return`

E20. 잘못된 인자 목록

의도 / 기준	잘못된 인자 목록 AST 출력 없음

입력 코드	`int identity(int value)`
	`{`
	`  return value;`
	`}`
	
	`int main()`
	`{`
	`  return identity(1,);`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/error/E20_trailing_argument_comma.c` `line 8: syntax error near )`

2-6. 엣지케이스 검증

빈 블록, 이름 가림, 전방 참조와 복합 선언자 등 엣지케이스 10개를 확인했습니다.

B01. 빈 함수 몸체

의도 / 기준	빈 함수 몸체

입력 코드	`int empty()`
	`{`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/boundary/B01_empty_function_body.c` `=======  syntax tree  ==========` `N_PROGRAM (0x0,0)` `|  (ID="empty") TYPE:0x1015a7430 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND (0x0,0)` `|					N_STMT_LIST_NIL (0x0,0)`

B02. 빈 중첩 블록

의도 / 기준	빈 중첩 블록

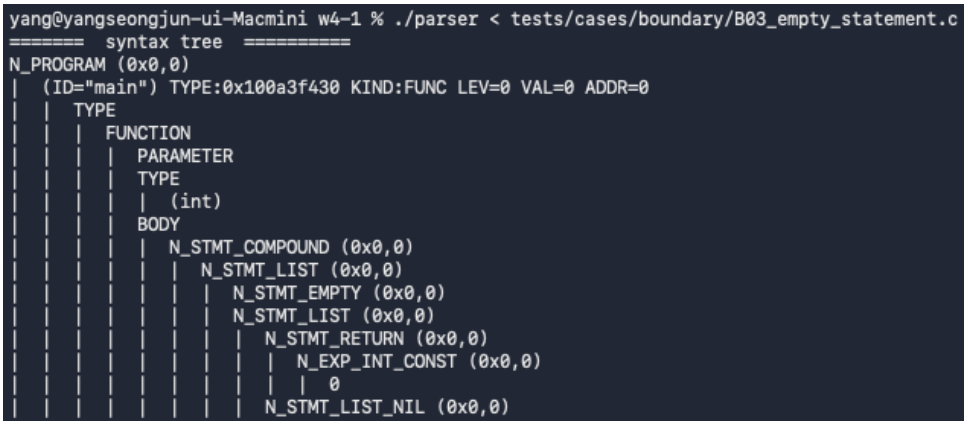
입력 코드	`int main()`
	`{`
	`  {`
	`  }`
	`  return 0;`
	`}`

실행 결과	`yang@yangseongjun-ui-Macmini w4-1 % ./parser < tests/cases/boundary/B02_empty_nested_block.c` `=======  syntax tree  ==========` `N_PROGRAM (0x0,0)` `|  (ID="main") TYPE:0x104913430 KIND:FUNC LEV=0 VAL=0 ADDR=0` `|	TYPE` `|		FUNCTION` `|			PARAMETER` `|			TYPE` `|				(int)` `|			BODY` `|				N_STMT_COMPOUND (0x0,0)` `|					N_STMT_LIST (0x0,0)` `|						N_STMT_COMPOUND (0x0,0)` `|							N_STMT_LIST_NIL (0x0,0)` `|						N_STMT_LIST (0x0,0)` `|							N_STMT_RETURN (0x0,0)` `|								N_EXP_INT_CONST (0x0,0)` `|									0` `|							N_STMT_LIST_NIL (0x0,0)`

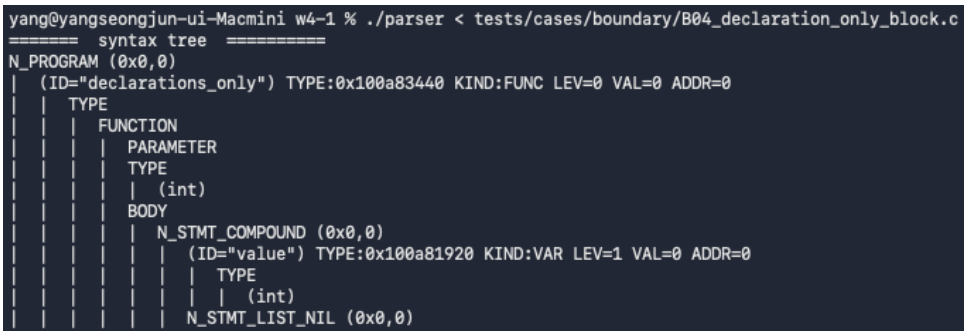
B03. 빈 명령문

의도 / 기준	빈 명령문

입력 코드	`int main()`
	`{`
	`  ;`
	`  return 0;`
	`}`

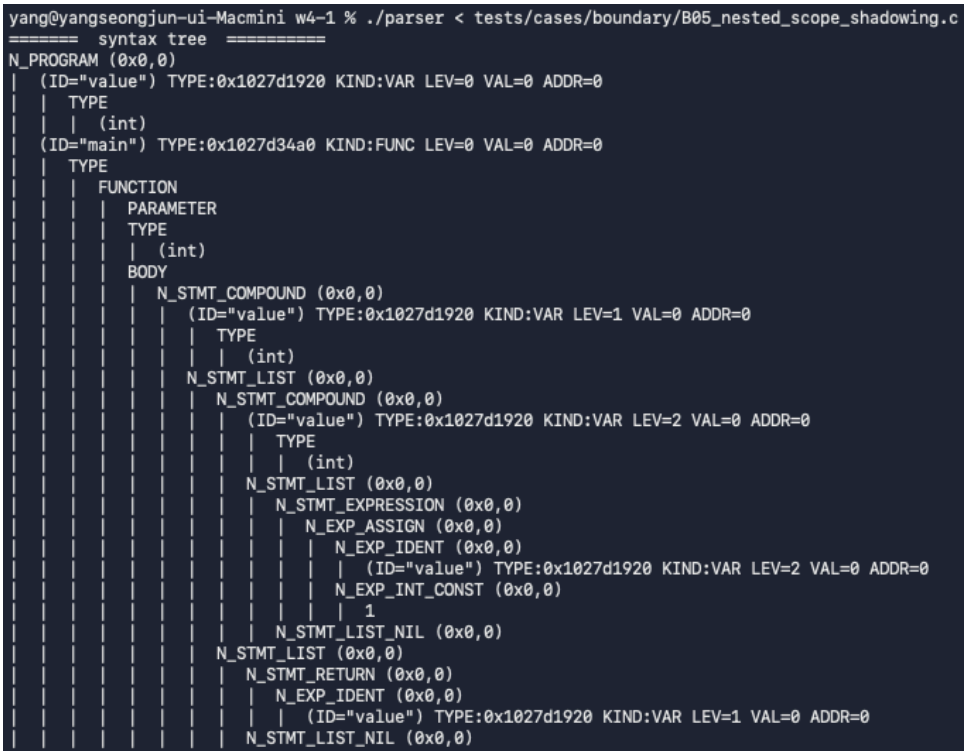
실행 결과	

B04. 선언만 있는 블록

의도 / 기준	선언만 있는 블록
입력 코드	
실행 결과	

B05. 중첩 스코프의 이름 가림

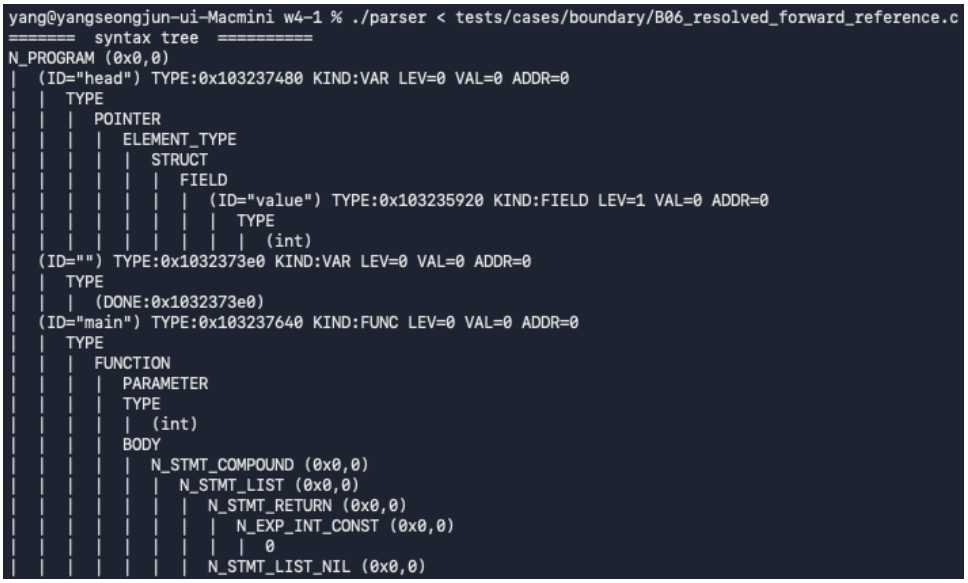
의도 / 기준	중첩 스코프의 이름 가림
입력 코드	

실행 결과	

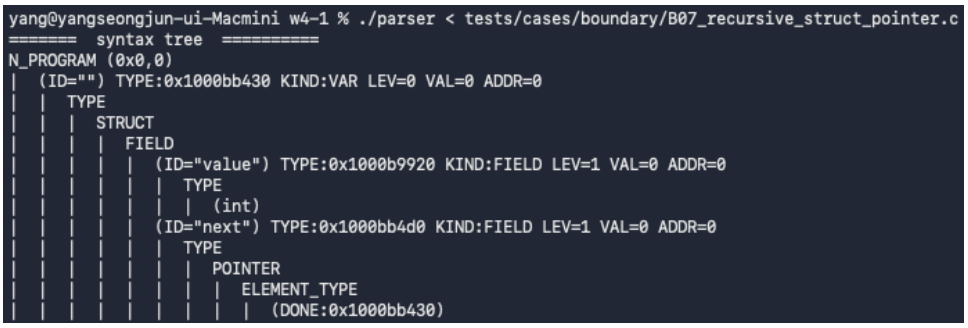
B06. 완성되는 전방 참조

의도 / 기준	완성되는 전방 참조

입력 코드	```
	struct Node *head;
	
	struct Node {
	    int value;
	};
	
	int main()
	{
	    return 0;
	}
	``

실행 결과	

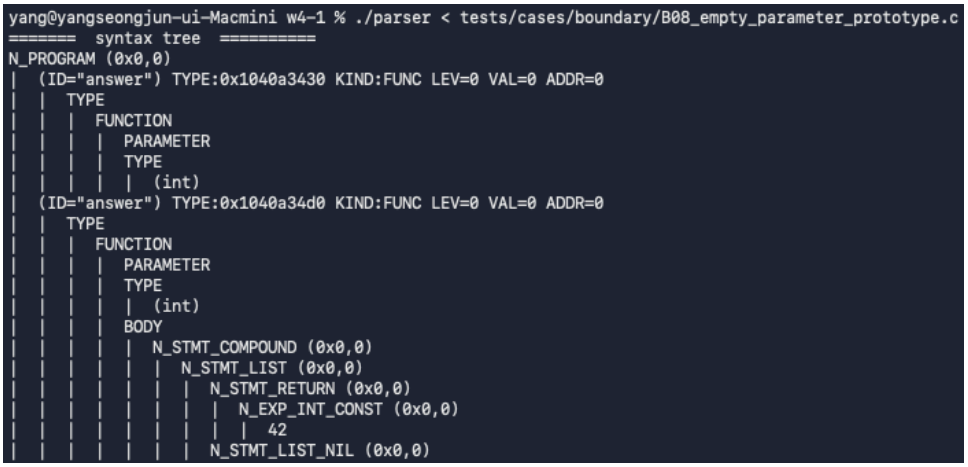
B07. 자기 참조 구조체

의도 / 기준	자기 참조 구조체
입력 코드	`struct Node { int value; struct Node *next; };`
실행 결과	

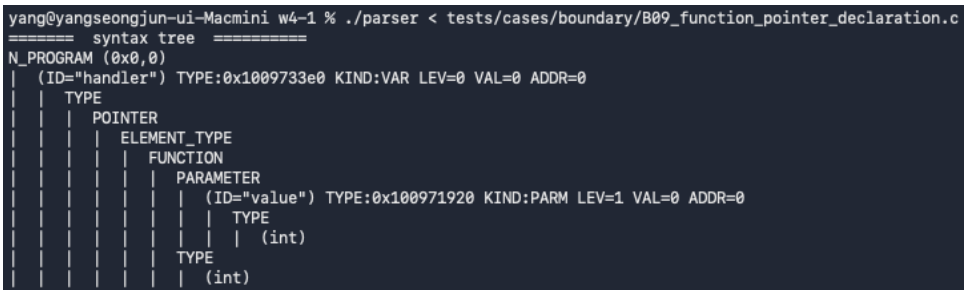
B08. 빈 매개변수 함수 원형

의도 / 기준	빈 매개변수 함수 원형

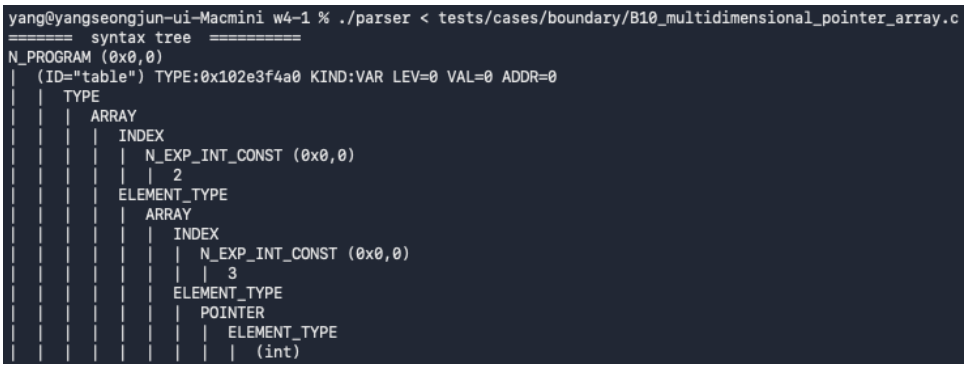
입력 코드	`int answer();`
	
	`int answer()`
	`{`
	`  return 42;`
	`}`

실행 결과	

B09. 함수 포인터

의도 / 기준	함수 포인터
입력 코드	`int (*handler)(int value);`
실행 결과	

B10. 다차원 포인터 배열

의도 / 기준	다차원 포인터 배열
입력 코드	`int *table[2][3];`
실행 결과	

3. 개발 과정

3-1. 오류 및 해결 기록

01. 초기 빌드: 함수와 전역 변수의 정의 누락

문제 / 원인	개발 당시 makeNode() 등의 함수 구현과 makeType(), int_type의 정의 또는 링크가 누락된 문제입니다.
오류 출력	w4-1/test.c:11:18: warning: comparison between pointer and integer w4-1/support.c:4:28: warning: unused parameter 'name' [-Wunused-paramete] w4-1/support.c:4:42: warning: unused parameter 'left' [-Wunused-paramete] w4-1/support.c:4:56: warning: unused parameter 'center' [-Wunused-parameter] w4-1/support.c:4:72: warning: unused parameter 'right' [-Wunused-parameter] w4-1/support.c:4:80: warning: control reaches end of non-void function [-Wreturn-type] C:/msys64/mingw64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\cc6VZ5qo.o:test.c:(.text+0x13): undefined reference to `makeType' collect2.exe: error: ld returned 1 exit status C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\ccuBiCXf.o:test.c:(.rdata$.refptr.int_type[.refptr.int_type]+0x0): undefined reference to `int_type'
현재 구현 / 상태	함수와 전역 변수의 정의 위치를 정리했습니다. initialize()에서 기본 타입과 내장 함수를 작성했습니다.

02. Specifier 처리: NULL 타입의 잘못된 검사

문제 / 원인	타입을 생략한 makeSpecifier(NULL, S_TYPEDEF) 호출을 오류로 처리한 문제입니다. 선언 지정자의 NULL 타입은 기본 타입을 보완하기 전에는 허용되어야 합니다.
오류 출력	Semantic error: NULL type in setTypeField
현재 구현 / 상태	GDB 호출 스택으로 makeSpecifier()의 NULL 처리 경로를 확인했습니다. 타입 생략을 허용하고 setDefaultSpecifier()에서 int_type으로 보완합니다.

03. 이름 없는 식별자 객체 생성

문제 / 원인	개발 당시 makeDummyIdentifier() 구현이 누락되어 이름 없는 식별자를 생성하지 못했던 항목입니다.
오류 출력	/usr/bin/ld: y.tab.o: undefined reference to `makeDummyIdentifier' collect2: error: ld returned 1 exit status
현재 구현 / 상태	name은 빈 문자열, type과 link는 NIL, prev는 0으로 초기화합니다.

04. Lex 오타: yytext와 대입 연산자

문제 / 원인	개발 당시 yytext를 yytest로, 대입 연산자 =를 비교 연산자 ==로 잘못 작성한 문제입니다.
오류 출력	c.y:308:52: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] c.y:308:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] c.y:310:22: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] c.y:311:55: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] c.l:105:18: error: 'yytest' undeclared (first use in this function); did you mean 'yytext'? lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function] lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function]
현재 구현 / 상태	yytest를 yytext로, ==를 =로 바로잡았습니다. 현재 정수 변환에는 atoi()를 사용합니다.
현재 코드	yylval=atoi(yytext); return(INTEGER_CONSTANT);

05. makeNode 중복 정의: 빌드 대상 중복

문제 / 원인	개발 당시 동일한 소스 파일을 두 번 링크하여 makeNode의 중복 정의가 발생한 문제입니다.
오류 출력	multiple definition of `makeNode'; ... support.c:4: first defined here collect2.exe: error: ld returned 1 exit status
현재 구현 / 상태	중복된 빌드 대상을 제거하고 현재 파일 구성에 맞게 정리했습니다.
현재 코드	gcc -o parser y.tab.c lex.yy.c support.c print.c main.c

06. 조건식의 비트 OR와 논리 OR 혼동

문제 / 원인	개발 당시 논리 조건 결합에 || 대신 |를 사용한 문제입니다.

오류 출력	support.c:418:16: warning: suggest parentheses around comparison in operand of '|' [-Wparentheses]
현재 구현 / 상태	조건을 논리 연산자로 결합하도록 수정했습니다. 아래 조건은 선행 선언 a가 있는 경우에 검사합니다.

현재 코드	if (a->kind != ID_FUNC || a->type->expr) {
	  syntax_error(12, id->name);
	}

07. Lex 생성 코드의 미사용 함수 경고

문제 / 원인	Flex가 생성하는 input, yyunput이 사용되지 않을 때 빌드 옵션에 따라 경고가 발생할 수 있는 항목입니다.
오류 출력	lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function] lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function]
현재 구현 / 상태	파서 동작 오류와 구분되는 빌드 경고입니다. 경고 발생 여부는 Flex 생성 코드와 컴파일 옵션에 따라 달라집니다.

08. Yacc 기호명, 함수명과 규칙 경계

문제 / 원인	개발 당시 문법 기호와 함수 이름의 오타, 문법 규칙 경계 누락이 기록된 항목입니다.
오류 출력	w4-1/c.y:270.10-51: error: symbol 'parameter_ttestUpdateSpecifierype_list_opt' is used, but is not defined as a token and has no rules w4-1/c.y:124:36: error: implicit declaration of function 'linkDeclarator'; did you mean 'linkDeclaratorList'? [-Wimplicit-function-declaration] w4-1/c.y:418.1-17: error: unexpected identifier
현재 구현 / 상태	기호명과 함수명의 오타, 규칙 끝의 세미콜론과 액션 경계를 확인했습니다.

09. 중간 액션과 중첩 함수 호출의 괄호

문제 / 원인	개발 당시 중간 액션의 의미값 위치와 중첩 함수 호출의 인수 배치를 혼동한 항목입니다.
오류 출력	w4-1/c.y:459:61: error: too few arguments to function 'setTypeExpr'; expected 2, have 1 w4-1/c.y:459:18: error: too many arguments to function 'setTypeElementType'; expected 2, have 3
현재 구현 / 상태	배열 크기 수식을 setTypeExpr()의 두 번째 인수로 전달하도록 수정했습니다.
현재 코드	$$=setTypeElementType($1,setTypeExpr(makeType(T_ARRAY),$3));

10. 전역 선언 목록과 식별자 출력

문제 / 원인	과거 전역 선언 목록에서 일부 함수의 출력이 누락되었던 항목입니다. 현재 선언 목록 연결 방식과 출력 서식을 구분하여 확인했습니다.
오류 출력	N_PROGRAM (0000000000000000,0) | (ID="main") TYPE:000001fa9b2c1f80 KIND:FUNC LEV=0 VAL=0 ADDR=0
현재 구현 / 상태	식별자의 이름, 타입, 종류와 스코프 정보를 출력합니다. 전역 선언 목록은 linkDeclaratorList()로 연결합니다.

11. 문자 상수의 값 전달 오류

문제 / 원인	과거 문자 상수에서 문자 코드 대신 예상과 다른 큰 정수가 출력되었던 항목입니다.
오류 출력	| | | | | | | | N_EXP_CHAR_CONST ((nil),0) | | | | | | | | | 912013584
현재 구현 / 상태	현재는 문자값을 yylval에 저장하여 'A'가 65로 출력됩니다.

먼저 함수 시그니처 선언 후 컴파일 에러

```
In file included from C:\Users\egod1\proj\compiler\w4\test.c:3:
C:\Users\egod1\proj\compiler\w4\test.c: In function 'main':
C:\Users\egod1\proj\compiler\w4\test.c:11:18: warning: comparison between pointer and
integer
   11 |   assert(t->kind != NULL);
      |                  ^~
C:\Users\egod1\proj\compiler\w4\syntax_tree.c: In function 'makeNode':
C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:28: warning: unused parameter 'name'
[-Wunused-paramete]
    4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE
*right) {}
      |                  ~~~~~~~~~~^~~~
C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:42: warning: unused parameter 'left'
[-Wunused-paramete]
    4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE
*right) {}
      |                           ~~~~~~~~^~~~
C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:56: warning: unused parameter 'center'
[-Wunused-parameter]
    4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE
*right) {}
      |                                    ~~~~~~~~^~~~~~
C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:72: warning: unused parameter 'right'
[-Wunused-parameter]
    4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE
*right) {}
      |                                                ~~~~~~~~^~~~~
C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:80: warning: control reaches end of
non-void function [-Wreturn-type]
    4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE
*right) {}
      |                                                                ^
C:/msys64/mingw64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw3
2/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\cc6VZ5qo.o:test.c:(.text+0x13): undefined
reference to `makeType'
collect2.exe: error: ld returned 1 exit status
PS C:\Users\egod1\proj\compiler\w4>
```

이거는 updateSpecifier 구현오류

```
PS C:\Users\egod1\proj\compiler\w4> ./run
makeType test passed
makeIdentifier test passed
searchIdentifier test passed
testScope passed
```

```
testLinkDeclaratorList passed
testSetTypeElementType passed
testSetDeclaratorElementType passed
testSetTypeExpr passed
testSetTypeField passed
Semantic error: NULL type in setTypeField

PS C:\Users\egod1\proj\compiler\w4> gdb ./test.exe
GNU gdb (GDB) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-w64-mingw32".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test.exe...
(gdb) break semanticError
Breakpoint 1 at 0x14000342e: file C:\Users\egod1\proj\compiler\w4\semantic.c, line 111.
(gdb) run
Starting program: C:\Users\egod1\proj\compiler\w4\test.exe
[New Thread 13820.0x657c]
makeType test passed
makeIdentifier test passed
searchIdentifier test passed
testScope passed
testLinkDeclaratorList passed
testSetTypeElementType passed
testSetDeclaratorElementType passed
testSetTypeExpr passed
testSetTypeField passed

Thread 1 hit Breakpoint 1, semanticError (message=0x7ff767b16a3d "NULL type in
setTypeField")
    at C:\Users\egod1\proj\compiler\w4\semantic.c:111
111       fprintf(stderr, "Semantic error: %s\n", message);
(gdb) bt
#0  semanticError (message=0x7ff767b16a3d "NULL type in setTypeField") at
C:\Users\egod1\proj\compiler\w4\semantic.c:111
#1  0x00007ff767b1363e in makeSpecifier (type=0x0, stor=S_TYPEDEF) at
C:\Users\egod1\proj\compiler\w4\semantic.c:186
```

```
#2  0x00007ff767b12b99 in testMakeSpecifier () at
C:\Users\egod1\proj\compiler\w4\test.c:360
#3  0x00007ff767b130df in main () at C:\Users\egod1\proj\compiler\w4\test.c:451
(gdb) quit
A debugging session is active.

        Inferior 1 [process 13820] will be killed.
```

gdb로 디버그해서 해결

```
PS C:\Users\egod1\proj\compiler\w4> ./run
C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bi
n/ld.exe:
C:\Users\egod1\AppData\Local\Temp\ccuBiCXf.o:test.c:(.rdata$.refptr.int_type[.refptr.int_typ
e]+0x0): undefined reference to `int_type'
collect2.exe: error: ld returned 1 exit status
```

int_type 전역 변수 초기화 안해서 그런거

이름이 없는 식별자 객체는 구현 X -> 강의 자료에 나와있긴 한데, 정확한 구현이 없어서

```
PS C:\Users\egod1\proj\compiler\w4> gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c
-o parser.exe
c.y: In function 'yyparse':
c.y:308:52: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast]
  308 |       $$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL);
      |                                                    ^
c.y:308:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]
  308 |       $$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL);
      |            ^
c.y:310:22: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast]
  310 |       A_NODE *node = (A_NODE*)$$;
      |                      ^
c.y:311:55: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]
  311 |       fprintf(stderr, "[AST] N_EXP_INT_CONST: %ld\n", (long)node->clink);
      |                                                       ^
c.l: In function 'yylex':
c.l:105:18: error: 'yytest' undeclared (first use in this function); did you mean 'yytext'?
  105 |   yylval == atol(yytest);
      |                  ^~~~~~
      |                  yytext
c.l:105:18: note: each undeclared identifier is reported only once for each function it appears
in
lex.yy.c: At top level:
lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function]
 1572 | #else
```

```
    |            ^
lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1529 |
    |            ^
PS C:\Users\egod1\proj\compiler\w4>
```

오타

[2026-11-03] Lex/Yacc 통합 빌드 오류 및 경고 기록

실행 명령어 (실패)
gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c semantic.c syntax_tree.c -o parser.exe

1. 링크 오류 — makeNode 중복 정의
원문: multiple definition of `makeNode'; ... syntax_tree.c:4: first defined here
원문: collect2.exe: error: ld returned 1 exit status
원인: 빌드 명령에서 syntax_tree.c를 두 번 지정해 동일한 함수가 두 번 링크되었음.
수정 예정 명령어: gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c semantic.c -o parser.exe
상태: 수정 방법 확인, 재빌드 성공 여부 미확인.

2. 의미 분석 코드의 연산자 경고
원문: semantic.c:418:16: warning: suggest parentheses around comparison in operand of '|' [-Wparentheses]
문제 코드: if (id->kind != ID_FUNC | id->type == NULL || id->type->kind != T_FUNC)
원인: 논리 OR(||)를 사용해야 하는 조건에 비트 OR(|)를 사용한 것으로 보임. 단락 평가가 이뤄지지 않아 NULL 타입 역참조 위험도 있음.
수정 제안: if (id->kind != ID_FUNC || id->type == NULL || id->type->kind != T_FUNC)
상태: 수정 제안, 재실행 결과 미확인.

3. Yacc Semantic Value의 포인터 크기 경고 (4건)
원문: c.y:308:52: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast]
원문: c.y:308:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]
원문: c.y:310:22: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast]
원문: c.y:311:55: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]
관련 코드: $$ = (long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE *)$1, NULL);
관련 코드: A_NODE *node = (A_NODE *)$$;
관련 코드: fprintf(stderr, "[AST] N_EXP_INT_CONST: %ld\n", (long)node->clink);
원인 분석: 현재 Windows x64 MinGW 환경의 long은 일반적으로 4바이트이고 포인터는 8바이트이므로, YYSTYPE이 long이면 포인터 값이 손실될 수 있음.
조치 방안: Windows에서는 포인터 너비의 YYSTYPE을 사용하고, 최종 UNIX 환경에서 교수 지시의 typedef long YYSTYPE 및 sizeof 검사. Yacc/Lex 양쪽 선언과 재생성된 파일이 일치해야 함.
상태: 미해결 경고, 수정 후 재검증 필요.

4. Flex 생성 코드의 미사용 함수 경고 (2건)
원문: lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function]

원문: lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function]
원인: Flex가 생성한 보조 함수를 현재 Lex 규칙에서 사용하지 않음.
조치 방안: 프로그램 오류와 구분하여 기록. 필요한 경우 c.l에 %option noinput nounput 추가 후 재생성.
상태: 비치명적 경고.

종합: 이번 빌드는 링크 오류로 실패했으며 parser.exe 생성 성공은 확인되지 않았음. 위 항목의 원문·원인·수정 제안을 기록했으며, 변경 후 실제 빌드·실험 결과는 별도 기록해야 함.

```
PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/04_parentheses.c
lex.yy.c:1571:16: warning: 'input' defined but not used [-Wunused-function]
 1571 | #else
      |            ^
lex.yy.c:1528:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1528 |
      |             ^
[DEBUG] N_EXP_INT_CONST: 1
[DEBUG] N_EXP_INT_CONST: 2
[DEBUG] N_EXP_INT_CONST: 3
Parsing success
[ERROR] Semantic error: Expected N_EXP_ADD
[ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\04_parentheses.c"
PS C:\Users\egod1\proj\compiler\w4>

PS C:\Users\egod1\proj\compiler\w4> $env:AST_TEST_IDENTIFIER = "1"
PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/05_identifier.c
C:\\Users\\egod1\\proj\\compiler\\w4\\c.y:270.10-51: error: symbol
'parameter_ttestUpdateSpecifierype_list_opt' is used, but
is not defined as a token and has no rules
  270 |     | LP parameter_ttestUpdateSpecifierype_list_opt RP
      |       ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
[ERROR] bison failed. See "C:\Users\egod1\proj\compiler\w4\build.log".
PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/05_identifier.c
C:\Users\egod1\proj\compiler\w4\c.y: In function 'yyparse':
C:\Users\egod1\proj\compiler\w4\c.y:124:36: error: implicit declaration of function
'linkDeclarator'; did you mean 'linkDeclaratorList'? [-Wimplicit-function-declaration]
  124 |       $$ = (YYSTYPE) (intptr_t) linkDeclarator(
      |                                 ^~~~~~~~~~~~~
      |                                 linkDeclaratorList
lex.yy.c:1571:16: warning: 'input' defined but not used [-Wunused-function]
 1571 | #else
      |            ^
lex.yy.c:1528:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1528 |
      |             ^
[ERROR] gcc failed. See "C:\Users\egod1\proj\compiler\w4\build.log".
```

function_declaration 기존이랑 구조가 크게 바뀐듯?

```
C:\\Users\\egod1\\proj\\compiler\\w4\\c.y:418.1-17: error: unexpected identifier
  418 | labeled_statement
      | ^~~~~~~~~~~~~~~~
[ERROR] bison failed. See "C:\Users\egod1\proj\compiler\w4\build.log".
PS C:\Users\egod1\proj\compiler\w4>
```

```
[SUCCESS] Parser test passed:
"C:\Users\egod1\proj\compiler\w4\tests\ast\39_do_while_statement.c"
lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function]
 1593 | #else
      |               ^
lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1550 |
      |                 ^
[DEBUG] N_EXP_INT_CONST: 0
[DEBUG] N_EXP_INT_CONST: 3
Parsing success
[ERROR] Semantic error: Invalid for AST
[ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\40_for_statement.c"
Exception:
Line |
   7 |             throw "FAILED: (case[1])"
     |             ~~~~~~~~~~~~~~~~~~~~~~~~~~에서
     | FAILED: 40_for_statement.c
```

iteration_statment 에서 for 문법 규칙이 잘 못 적혀있었음

```
PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/26_function_call_args.c
>>
lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function]
 1593 | #else
      |               ^
lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1550 |
      |                 ^
C:\Users\egod1\proj\compiler\w4\syntax_tree.c: In function 'makeNodeList':
C:\Users\egod1\proj\compiler\w4\syntax_tree.c:28:13: warning: variable 'nil' set but not used
[-Wunused-but-set-variable=]
   28 |   NODE_NAME nil;
      |             ^~~
[DEBUG] N_EXP_INT_CONST: 1
[DEBUG] N_EXP_INT_CONST: 2
```

```
[ERROR] Semantic error: Invalid list terminator
[ERROR] Parser test failed:
"C:\Users\egod1\proj\compiler\w4\tests\ast\26_function_call_args.c"
PS C:\Users\egod1\proj\compiler\w4>
```

gdb로 디버깅 하기

```
PS C:\Users\egod1\proj\compiler\w4>
>>
PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/51_sizeof_type.c
>>
lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function]
 1593 | #else
      |              ^
lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1550 |
      |              ^
Parsing success
[ERROR] Semantic error: Invalid sizeof type AST
[ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\51_sizeof_type.c"
```

```
enum_specifier
    : ENUM_SYM IDENTIFIER
    {}
      LR enumerator_list RR
    | ENUM_SYM LR enumerator_list RR
    | ENUM_SYM IDENTIFIER
    ;
```

여기서 괄호 위치의 레벨이 동일해서 문법 중간에 액션이 들어가는 지 헷갈려서 에러 난 경우가 많음

```
C:\Users\egod1\proj\compiler\w4\c.y:459:61: error: too few arguments to function
'setTypeExpr'; expected 2, have 1
  459 |       $$ = YY(setTypeElementType(PTR(A_TYPE, $1),
setTypeExpr(makeType(T_ARRAY)),
      |                                                  ^~~~~~~~~~
C:\Users\egod1\proj\compiler\w4\parse_value.h:18:36: note: in definition of macro 'YY'
   18 | #define YY(x) ((YYSTYPE)(intptr_t)(x))
      |                                    ^
In file included from C:\Users\egod1\proj\compiler\w4\c.y:7:
C:\Users\egod1\proj\compiler\w4\semantic.h:33:9: note: declared here
   33 | A_TYPE *setTypeExpr(A_TYPE *type, A_NODE *expr);
```

```
     |        ^~~~~~~~~~
C:\Users\egod1\proj\compiler\w4\c.y:459:18: error: too many arguments to function
'setTypeElementType';
expected 2, have 3
  459 |       $$ = YY(setTypeElementType(PTR(A_TYPE, $1),
setTypeExpr(makeType(T_ARRAY)),
      |                  ^~~~~~~~~~~~~~~~~
C:\Users\egod1\proj\compiler\w4\parse_value.h:18:36: note: in definition of macro 'YY'
   18 | #define YY(x) ((YYSTYPE)(intptr_t)(x))
      |                                    ^
C:\Users\egod1\proj\compiler\w4\semantic.h:28:9: note: declared here
   28 | A_TYPE *setTypeElementType(A_TYPE *t1, A_TYPE *t2);
      |         ^~~~~~~~~~~~~~~~~
lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function]
 1625 | #else
      |                ^
lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1582 |
      |                 ^
[ERROR] gcc failed. See "C:\Users\egod1\proj\compiler\w4\build.log".
PS C:\Users\egod1\proj\compiler\w4>


PS C:\Users\egod1\proj\compiler\w4> Set-Content "$env:TEMP\w4_static.c" 'static int
counter; int main(void) { return counter; }'
PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd "$env:TEMP\w4_static.c"
lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function]
 1625 | #else
      |                ^
lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1582 |
      |                 ^
[ERROR] Semantic error: Duplicate type specifier
[ERROR] Parser test failed: "C:\Users\egod1\AppData\Local\Temp\w4_static.c"
PS C:\Users\egod1\proj\compiler\w4>

PS C:\Users\egod1\proj\compiler\w4> Set-Content "$env:TEMP\w4_storage_invalid.c" 'static
auto int x;'
PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd "$env:TEMP\w4_storage_invalid.c"
lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function]
 1625 | #else
      |                ^
lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1582 |
      |                 ^
[ERROR] Semantic error: Duplicate storage class specifier
[ERROR] Parser test failed: "C:\Users\egod1\AppData\Local\Temp\w4_storage_invalid.c"
PS C:\Users\egod1\proj\compiler\w4>
```

이건 의도된 오류

```
PS C:\Users\egod1\proj\compiler\w4> $env:AST_PRINT="1"; .\parser.cmd
tests/valid/02_functions.c
lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function]
 1625 | #else
      |               ^
lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function]
 1582 |
      |                 ^
[DEBUG] N_EXP_INT_CONST: 2
[DEBUG] N_EXP_INT_CONST: 3
Parsing success
=======  syntax tree  ==========
N_PROGRAM (0000000000000000,0)
| (ID="main") TYPE:000001fa9b2c1f80 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
| | TYPE
| | | FUNCTION
| | | | PARAMETER
| | | | | (ID="") TYPE:000001fa9b2c7e20 KIND:PARM SPEC=NULL LEV=0 VAL=0
ADDR=0
| | | | | | TYPE
| | | | | | | (void)| | | | TYPE
| | | | | (int)
| | | | BODY
| | | | | N_STMT_COMPOUND (0000000000000000,0)
| | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1 VAL=0
ADDR=0
| | | | | | | TYPE
| | | | | | | | (int)
| | | | | | N_STMT_LIST (0000000000000000,0)
| | | | | | | N_STMT_EXPRESSION (0000000000000000,0)
| | | | | | | | N_EXP_ASSIGN (0000000000000000,0)
| | | | | | | | | N_EXP_IDENT (0000000000000000,0)
| | | | | | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1
VAL=0 ADDR=0
| | | | | | | | | N_EXP_FUNCTION_CALL (0000000000000000,0)
| | | | | | | | | | N_EXP_IDENT (0000000000000000,0)
| | | | | | | | | | | (ID="add") TYPE:000001fa9b2c1690 KIND:FUNC SPEC=NULL LEV=0
VAL=0 ADDR=0

| | | | | | | | | N_ARG_LIST (0000000000000000,0)
| | | | | | | | | | N_EXP_INT_CONST (0000000000000000,0)
| | | | | | | | | | | 2
| | | | | | | | | N_ARG_LIST (0000000000000000,0)
```

| | | | | | | | | | | N_EXP_INT_CONST (0000000000000000,0)
| | | | | | | | | | | | 3
| | | | | | | | | | | N_ARG_LIST_NIL (0000000000000000,0)
| | | | | | N_STMT_LIST (0000000000000000,0)
| | | | | | | N_STMT_RETURN (0000000000000000,0)
| | | | | | | | N_EXP_IDENT (0000000000000000,0)
| | | | | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0
| | | | | | | N_STMT_LIST_NIL (0000000000000000,0)
[SUCCESS] Parser test passed:
"C:\Users\egod1\proj\compiler\w4\tests\valid\02_functions.c"
PS C:\Users\egod1\proj\compiler\w4>

처음 출력인데, add 함수가 누락되어 있음

w4 print.c 출력 검증, 2026-11-05 KST

검증 커밋: bbba5f5c030c44c26e909e7ebf908ca193ef7f54 (origin/w4, impl)
https://github.com/egod1537/compiler/commit/bbba5f5c030c44c26e909e7ebf908ca193ef7f54
기존 기록 뒤에 이번 검증을 추가했다. 아래 로그는 이 커밋을 Linux x86_64 / GCC로 직접 빌드해 실행한 결과다. 저장된 y.tab.c와 lex.yy.c를 사용했으며, 이 환경에는 bison/flex가 없어 재생성과 Windows parser.cmd 실행은 검증하지 않았다. 원본 소스 수정 및 커밋은 하지 않았다. 메모리 주소와 주소를 잘못 변환한 정수는 실행할 때마다 달라진다.

검증 결과

기존 AST 입력 63개: 모두 종료 코드 0, 출력 중 충돌·무한 반복·syntax tree error 없음. 이것은 출력 순회 성공이며 값의 정확성까지 모두 통과했다는 뜻은 아니다. 문자 상수 오류는 이 성공 사례 안에서도 발생했다.
기존 valid 입력 15개: 7개 성공, 8개는 현재 구현에서 명시적으로 거부한 기능으로 실패했다. 기존 invalid 입력 10개는 모두 구문 오류로 거부했다.
추가 입력 43개로 문자 escape, 함수 원형, 스코프, 재귀 구조체·공용체, 전방 참조, 다차원 배열, 함수 포인터, 빈 블록, for식 생략 등을 검사했다. 기존 AST 검증 옵션 48건 중 42건 통과, 6건은 아래의 오래된 검증 조건 때문에 실패했다. 실패한 지역 변수 검사 5건은 별도 임시 검사기로 실제 A_ID 포인터 연결과 스코프 복원까지 모두 확인했다.
앞서 기록된 add 누락은 이번 커밋에서 해결됨: add → main이 N_PROGRAM의 A_ID.link 목록에 소스 순서대로 출력된다. 구조체 멤버 접근, enum 타입/열거자 목록, switch/case/default, 재귀 타입의 DONE 표시는 점검한 입력에서 정상이다. DONE은 이미 출력한 타입의 순환 방지 표시다.

1. 문자 상수가 문자 코드 대신 주소의 일부로 출력됨

판정: 확정 오류, lexer의 토큰 값과 교수님 출력기가 기대하는 AST 자료형이 불일치한다.
```
int main(void) { return 'A'; }
```
실제 출력 전체, 종료 코드 0:

```
Parsing success
=======  syntax tree  ==========
N_PROGRAM ((nil),0)
|  (ID="main") TYPE:0x561e365c38b0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="") TYPE:0x561e365be360 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (void)|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND ((nil),0)
|  |  |  |  |  |  N_STMT_LIST ((nil),0)
|  |  |  |  |  |  |  N_STMT_RETURN ((nil),0)
|  |  |  |  |  |  |  |  N_EXP_CHAR_CONST ((nil),0)
|  |  |  |  |  |  |  |  |  912013584
|  |  |  |  |  |  |  N_STMT_LIST_NIL ((nil),0)
```

기대: N_EXP_CHAR_CONST의 값이 65. 추가 재현에서도 '\n'은 10, '\0'은 0이어야 하나 각각 주소에서 유래한 엉뚱한 정수가 출력됐다.

```
실제 '\n' 출력: |  |  |  |  |  |  |  |  7584016
실제 '\0' 출력: |  |  |  |  |  |  |  |  158300432
```

원인: c.l의 CHARACTER_CONSTANT는 따옴표를 포함한 문자열을 malloc하여 yylval에 저장한다. c.y는 이를 그대로 N_EXP_CHAR_CONST.clink에 넣는다. 반면 print.c:138~139는 clink를 정수값으로 해석한다. 강의노트 5-2장의 노드 구조도 N_EXP_CHAR_CONST의 중앙 링크를 INTEGER로 설명한다.

AST_checkCharString은 clink를 char*로 취급해 "'A'"와 비교하므로, 현재 잘못된 자료형을 오히려 통과시킨다. 문자 디코딩, clink 정수 저장, 해당 검증기의 기대값을 함께 맞춰야 한다. 교수님 print.c를 문자열 출력으로 바꾸는 것만으로 해결할 문제는 아니다.

```
Get-ChildItem Env:AST_TEST* | Remove-Item
$env:AST_PRINT="1"
.\parser.cmd tests/ast/62_char_string.c
```

2. 함수 원형의 매개변수 스코프 누출 및 잘못된 중복 판정

판정: 확정 심볼 테이블 오류. 출력기는 잘못 구성된 심볼 연결을 그대로 드러낸다.

재현 A, 원형과 정의가 같은 매개변수 이름을 사용:

```
int f(int x);
int f(int x) { return x; }
int main(void) { return f(1); }
[ERROR] Semantic error: Redeclaration of parameter
```

기대: 원형과 정의가 일치하므로 성공. 첫 번째 원형의 x가 다음 선언의 중복 검사에 남아 있어 잘못 거부된다. 원형을 int f(int);로 바꾸거나 원형의 이름을 a로 바꾸면 같은 정의가 성공했다.

재현 B, 선언되지 않은 이름이 다른 함수에서 허용됨:

```
int f(int x);
int main(void) { return x; }
```

실제 출력 전체, 종료 코드 0:

```
Parsing success
=======  syntax tree  ==========
N_PROGRAM ((nil),0)
|  (ID="f") TYPE:0x55e1ef94b8d0 KIND:FUNC SPEC=AUTO LEV=0 VAL=0 ADDR=0
|  |  TYPE
```

```
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="x") TYPE:0x55e1ef9462a0 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (int)
|  |  |  |  |  TYPE
|  |  |  |  |  (int)
|  (ID="main") TYPE:0x55e1ef94ba10 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0
|  |  TYPE
|  |  |  FUNCTION
|  |  |  |  PARAMETER
|  |  |  |  |  (ID="") TYPE:0x55e1ef946360 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0
|  |  |  |  |  |  TYPE
|  |  |  |  |  |  |  (void)|  |  |  |  TYPE
|  |  |  |  |  (int)
|  |  |  |  BODY
|  |  |  |  |  N_STMT_COMPOUND ((nil),0)
|  |  |  |  |  |  N_STMT_LIST ((nil),0)
|  |  |  |  |  |  |  N_STMT_RETURN ((nil),0)
|  |  |  |  |  |  |  |  N_EXP_IDENT ((nil),0)
|  |  |  |  |  |  |  |  |  (ID="x") TYPE:0x55e1ef9462a0 KIND:PARM SPEC=NULL LEV=0 VAL=0
ADDR=0
|  |  |  |  |  |  |  N_STMT_LIST_NIL ((nil),0)
```

기대: main에서 x는 선언되지 않았으므로 Undeclared identifier 오류. 실제로는 원형의 매개변수 x를 KIND:PARM LEV=0으로 참조한다. int (*p)(int x); 다음에 main에서 x를 참조하는 경우도 같은 증상이다.

재현 C, 전역 변수와 함수 매개변수의 정상적인 가림도 거부됨:

```
int x;
int f(int x) { return x; }
[ERROR] Semantic error: Redeclaration of parameter
```

기대: 매개변수 x가 함수 안에서 전역 x를 가리므로 성공.

원인: direct_declarator가 LP~RP 매개변수를 읽는 동안 별도 스코프를 열지 않는다. makeIdentifier는 현재 수준인 0에 매개변수를 등록하고, setParameterDeclaratorSpecifier가 같은 수준의 전역 이름과 중복 검사한다. 함수 정의의 enterScope는 이 과정이 끝난 뒤에 실행되어 거부를 막지 못한다. 원형만 선언한 경우 매개변수 심볼이 current_id/prev 검색 목록에도 남는다. A_TYPE.field/link의 매개변수 목록은 유지하면서 검색 스코프만 올바르게 분리해야 한다.

빌드 후 PowerShell에서 최소 입력을 바로 재현:

```
'int f(int x); int f(int x) { return x; }' | .\parser.exe
'int f(int x); int main(void) { return x; }' | .\parser.exe
'int x; int f(int x) { return x; }' | .\parser.exe
```

3. AST 검증기 6건이 현재 자료구조와 스코프 규칙을 따르지 않음

판정: 검증기 오류. 파서 수정과 구분해야 한다.

전역 선언 목록 재현:

```
int a; int b; int c;
Parsing success
[ERROR] Semantic error: Incorrect global declaration order
```

AST_TEST_PROGRAM을 끄면 a → b → c가 정상 출력된다. AST_checkProgram은 아직 {c,b,a} 순서와 prev 순회를 기대한다. 현재 N_PROGRAM.clink의 선언 목록 규약에 맞게 {a,b,c}와 link 순회로 바꿔야 한다.
지역 변수 관련 재현: AST_TEST_ADDRESS_OF / 15_address_of.c, AST_TEST_PRE_INC / 18_pre_increment.c, AST_TEST_PRE_DEC / 19_pre_decrement.c, AST_TEST_POST_INC / 20_post_increment.c, AST_TEST_POST_DEC / 21_post_decrement.c.

```
Parsing success
[ERROR] Semantic error: Identifier mismatch
```

위 5건은 모두 Parsing success 뒤 Identifier mismatch로 실패한다. 검사 함수는 파싱 완료 후 searchIdentifier("x")와 AST의 지역 변수 포인터를 비교하지만, 함수 스코프가 닫힌 뒤 지역 x가 전역 검색에서 사라지는 것이 정상이다.
별도 임시 검사 결과: 5건 모두 함수 본문의 선언 목록에 있는 local x와 단항 연산자의 N_EXP_IDENT가 동일한 A_ID*를 참조했다. local.level == 1이고 전역 searchIdentifier("x") == NULL인 것도 확인했다. 기존 검사기는 전역 검색 대신 main->type->expr의 지역 선언 목록과 비교해야 한다.

```
Get-ChildItem Env:AST_TEST* | Remove-Item
$env:AST_TEST_ADDRESS_OF="1"
.\parser.cmd tests/ast/15_address_of.c
Remove-Item Env:AST_TEST_ADDRESS_OF
$env:AST_TEST_PROGRAM="1"
'int a; int b; int c;' | .\parser.exe
Remove-Item Env:AST_TEST_PROGRAM
```

4. void 타입 출력 뒤 줄바꿈이 빠짐

판정: 교수님 제공 print.c의 경미한 출력 서식 문제. AST 연결 문제는 아니다.

```
void f(void) { return; }
int main(void) { f(); return 0; }
```

실제 출력 발췌:

```
|  |  |  |  |  |  |  (void)|  |  |  |  TYPE
|  |  |  |  |  (void)|  |  |  |  BODY
|  |  |  |  |  |  |  (void)|  |  |  |  TYPE
```

기대: (void) 뒤에 줄바꿈하고 다음 TYPE 또는 BODY를 독립된 줄에 출력. print.c:320~321이 printf("(void)")만 호출하기 때문이다. 교수님 원본 보존 방침에 따라 이번에는 수정하지 않았다.

5. 추가 호환성 관찰: typedef 이름의 내부 스코프 재선언

아래는 C 규칙상 허용되는 코드이나 현재 파서는 거부한다. 과제에서 요구하는 typedef 가림 범위에 포함되는지는 별도 확인이 필요하며 print.c 결함으로 분류하지 않는다.

```
typedef int I;
int main(void) { typedef float I; I y; return 0; }
[ERROR] Semantic error: Duplicate type specifier
```

원인: 기존 typedef 이름 I가 lexer에서 TYPE_IDENTIFIER로 분류되어 새 선언자 대신 선언 지정자로 읽힌다. 새 이름 F를 사용하는 typedef float F;는 성공했다.

6. valid 테스트 폴더의 8개 실패는 별도 분류

01_declarations, 06_storage_qualifiers, 07_arrays_and_calls: Initializers are not supported.

04_struct_enum: Enumerator initializers are not supported.
05_operators: Left shift is not supported.
09_bitwise: Bitwise AND is not supported.
13_variadic_function: Variadic parameters are not supported.
15_comma_and_conditional: Comma expressions are not supported.
위 사례는 AST 출력 전에 거부되므로 print.c의 출력 오류가 아니다. 현재 c.y가 명시한 미지원 기능과 테스트 폴더의 valid 분류가 맞지 않는다. 지원 범위 내 회귀 테스트와 미지원 입력 테스트를 구분해 결과를 읽어야 하며, 전체 valid 테스트가 통과했다고 보고하면 안 된다.

이번 검증 후 남은 수정 우선순위

우선 수정: 문자 상수의 정수 표현 통일, 함수 선언자 매개변수 스코프 분리.
검증 보완: 전역 선언 검사 1건과 지역 변수 검사 5건 수정, AST_checkCharString의 숫자 기대값 검사.
낮은 우선순위: void 줄바꿈, typedef 가림의 과제 지원 범위 확인, valid 테스트 분류 정리.
이번 작업은 검증 및 로그 기록까지이며 저장소 소스는 변경하지 않았다.

컴파일러 **W4** 테스트 케이스

[검증 시점 구분] 이 탭의 AST/ASTEXP '미실행'은 이전 Linux 환경의 기록입니다. 2026.11.07 Windows 전체 회귀 결과는 CH5 15, AST 17, ASTEXP 7, SEM 17, SEMERR 16건 모두 통과입니다(w4/TEST_RESULTS.md). 기존 콘솔 출력은 소급 변경하지 않았습니다.

번호로 선택하는 자동 테스트 41건의 기록입니다.

SEM 17건은 전체·개별 실행 모두 통과입니다. AST 17건과 ASTEXP 7건은 미실행입니다.
실행 환경은 Linux x86_64, GCC 14.2.0입니다. AST 실행에는 Bison·Flex 설치가 필요한 상태입니다.
코드와 확보된 콘솔 출력은 원문입니다. 명령은 w4 디렉터리 기준입니다.

1. SEM, 의미 처리 단위 테스트

SEM 01. 타입 생성

의도 / 기준	타입 객체 생성과 T_POINTER 설정, element_type의 NULL 초기화를 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_type_table.c make test-sem TEST=1

테스트 코드	`void SEM_01_MakeType(void) {`
	`  A_TYPE *t = makeType(T_POINTER);`
	
	`  assert(t != NULL);`
	`  assert(t->kind == T_POINTER);`
	`  assert(t->element_type == NULL);`
	
	`  free(t);`
	`}`

실행 결과 종료 코드 0	`PASS [SEM:01] SEM_01_MakeType  make type` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 02. 식별자 생성

의도 / 기준	식별자 이름 저장, prev 연결, current_id 갱신과 link 초기화를 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_symbol_table.c make test-sem TEST=2

테스트 코드	`void SEM_02_MakeIdentifier(void) {`
	`  A_ID *a = makeIdentifier("a");`
	`  A_ID *b = makeIdentifier("b");`
	
	`  assert(strcmp(a->name, "a") == 0);`
	`  assert(strcmp(b->name, "b") == 0);`
	`  assert(b->prev == a);`
	`  assert(current_id == b);`
	`  assert(a->prev == NULL);`
	`  assert(a->link == NULL);`
	
	`  free(a);`
	`  free(b);`
	`}`

실행 결과 종료 코드 0	`PASS [SEM:02] SEM_02_MakeIdentifier  make identifier` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 03. 식별자 검색

의도 / 기준	전역 검색, 없는 이름의 NULL 반환, 지역 이름 가림과 동일 레벨 중복 검색을 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_symbol_table.c make test-sem TEST=3

테스트 코드	`void SEM_03_SearchIdentifier(void) {`
	`  current_id = NULL;`
	`  current_level = 0;`
	
	`  A_ID *global_x = makeIdentifier("x");`
	`  A_ID *global_y = makeIdentifier("y");`
	`  assert(searchIdentifier("x") == global_x);`
	`  assert(searchIdentifier("y") == global_y);`
	`  assert(searchIdentifier("z") == NULL);`
	
	`  current_level = 1;`
	
	`  A_ID *local_x = makeIdentifier("x");`
	`  assert(searchIdentifier("x") == local_x);`
	`  assert(searchIdentifierAtCurrentLevel("x", local_x->prev) == NULL);`
	
	`  A_ID *duplicate_x = makeIdentifier("x");`
	`  assert(searchIdentifierAtCurrentLevel("x", duplicate_x->prev) == local_x);`
	
	`  A_ID *id = current_id;`
	`  while (id != NULL) {`
	`  A_ID *prev = id->prev;`
	`  free(id->name);`
	`  free(id);`
	`  id = prev;`
	`  }`
	
	`  current_id = NULL;`
	`  current_level = 0;`
	`}`

실행 결과 종료 코드 0	`PASS [SEM:03] SEM_03_SearchIdentifier  search identifier` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 04. 스코프 진입·종료

의도 / 기준	스코프 레벨 0→1→2→1→0과 종료 후 식별자 검색 범위 복원을 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_scope.c make test-sem TEST=4
테스트 코드	

```c
void SEM_04_Scope(void) {
  assert(current_level == 0);
  assert(current_id == NULL);

  A_ID *a = setDeclaratorKind(makeIdentifier("a"), ID_VAR);

  enterScope();
  assert(current_level == 1);

  A_ID *b = setDeclaratorKind(makeIdentifier("b"), ID_VAR);

  assert(current_id == b);
  assert(searchIdentifier("a") == a);
  assert(searchIdentifier("b") == b);

  enterScope();
  assert(current_level == 2);

  A_ID *c = setDeclaratorKind(makeIdentifier("c"), ID_VAR);

  assert(searchIdentifier("c") == c);

  leaveScope();

  assert(current_level == 1);
  assert(current_id == b);
  assert(searchIdentifier("c") == NULL);
  assert(searchIdentifier("b") == b);

  leaveScope();

  assert(current_level == 0);
  assert(current_id == a);
  assert(searchIdentifier("b") == NULL);
  assert(searchIdentifier("a") == a);

  free(c->name);
  free(c);
  free(b->name);
  free(b);
  free(a->name);
  free(a);

  current_id = NULL;

}
```

실행 결과
종료 코드 0

```
PASS [SEM:04] SEM_04_Scope                          scope
SUMMARY [SEM] 1 passed, 0 failed, 1 total
```

SEM 05. 선언자 목록 연결

의도 / 기준	선언자 목록의 a→b→c 순서, prev 체인 보존과 NULL 목록 처리를 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_symbol_table.c make test-sem TEST=5

테스트 코드	`void SEM_05_LinkDeclaratorList(void) {`
	`  assert(current_id == NULL);`
	`  assert(current_level == 0);`
	
	`  A_ID *a = makeIdentifier("a");`
	`  A_ID *b = makeIdentifier("b");`
	`  A_ID *c = makeIdentifier("c");`
	
	`  A_ID *list = linkDeclaratorList(a, b);`
	`  list = linkDeclaratorList(list, c);`
	
	`  // 선언자 목록 연결 검증`
	`  assert(list == a);`
	`  assert(a->link == b);`
	`  assert(b->link == c);`
	`  assert(c->link == NULL);`
	
	`  // 심볼 테이블 연결은 변경되지 않아야 함`
	`  assert(current_id == c);`
	`  assert(c->prev == b);`
	`  assert(b->prev == a);`
	`  assert(a->prev == NULL);`
	
	`  // 빈 리스트 처리 검증`
	`  assert(linkDeclaratorList(NULL, NULL) == NULL);`
	`  assert(linkDeclaratorList(NULL, a) == a);`
	`  assert(linkDeclaratorList(a, NULL) == a);`
	
	`  // 메모리 해제`
	`  free(c->name);`
	`  free(c);`
	`  free(b->name);`
	`  free(b);`
	`  free(a->name);`
	`  free(a);`
	
	`  current_id = NULL;`
	
	`}`

실행 결과 종료 코드 0	`PASS [SEM:05] SEM_05_LinkDeclaratorList  link declarator list` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 06. 타입 체인 연결

의도 / 기준	포인터·배열 타입 체인의 끝에 element_type이 연결되는지 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_type_table.c make test-sem TEST=6

테스트 코드	`void SEM_06_SetTypeElementType(void) {`
	`  A_TYPE *int_type = makeType(T_ENUM);`
	`  A_TYPE *pointer1 = makeType(T_POINTER);`
	`  A_TYPE *pointer2 = makeType(T_POINTER);`
	
	`  // T_POINTER -> int_type`
	`  A_TYPE *result = setTypeElementType(pointer1, int_type);`
	
	`  assert(result == pointer1);`
	`  assert(pointer1->element_type == int_type);`
	
	`  // T_POINTER -> T_POINTER -> int_type`
	`  result = setTypeElementType(pointer2, pointer1);`
	
	`  assert(result == pointer2);`
	`  assert(pointer2->element_type == pointer1);`
	`  assert(pointer1->element_type == int_type);`
	
	`  // 기존 체인의 마지막에 다른 타입 연결`
	`  A_TYPE *array = makeType(T_ARRAY);`
	`  A_TYPE *outer = makeType(T_POINTER);`
	
	`  setTypeElementType(outer, array);`
	`  setTypeElementType(outer, pointer2);`
	
	`  assert(outer->element_type == array);`
	`  assert(array->element_type == pointer2);`
	`  assert(pointer2->element_type == pointer1);`
	
	`  // 만들어진 타입 체인의 메모리 해제`
	`  free(outer);`
	`  free(array);`
	`  free(pointer2);`
	`  free(pointer1);`
	`  free(int_type);`
	
	`}`

실행 결과 종료 코드 0	`PASS [SEM:06] SEM_06_SetTypeElementType  set type element type` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 07. 선언자 타입 연결

의도 / 기준	선언자의 최초 타입 설정, 포인터→포인터→int 연결과 NULL 타입 입력의 무변경을 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=7

테스트 코드	`void SEM_07_SetDeclaratorElementType(void) {`
	`  assert(current_id == NULL);`
	`  assert(current_level == 0);`
	
	`  A_ID *p = makeIdentifier("p");`
	
	`  A_TYPE *pointer1 = makeType(T_POINTER);`
	`  A_TYPE *pointer2 = makeType(T_POINTER);`
	`  A_TYPE *int_type = makeType(T_ENUM);`
	
	`  // 1. 처음 타입 연결`
	`  A_ID *result = setDeclaratorElementType(p, pointer1);`
	
	`  assert(result == p);`
	`  assert(p->type == pointer1);`
	`  assert(pointer1->element_type == NULL);`
	
	`  // 2. 기존 타입 체인의 마지막에 연결`
	`  setDeclaratorElementType(p, pointer2);`
	`  setDeclaratorElementType(p, int_type);`
	
	`  assert(p->type == pointer1);`
	`  assert(pointer1->element_type == pointer2);`
	`  assert(pointer2->element_type == int_type);`
	`  assert(int_type->element_type == NULL);`
	
	`  // 3. NULL 타입 처리`
	`  result = setDeclaratorElementType(p, NULL);`
	
	`  assert(result == p);`
	`  assert(p->type == pointer1);`
	`  assert(pointer2->element_type == int_type);`
	
	`  // 메모리 해제`
	`  free(pointer1);`
	`  free(pointer2);`
	`  free(int_type);`
	`  free(p->name);`
	`  free(p);`
	
	`  current_id = NULL;`
	
	`}`

실행 결과 종료 코드 0	`PASS [SEM:07] SEM_07_SetDeclaratorElementType  set declarator element type` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 08. 배열 크기 수식

의도 / 기준	배열 크기 10의 수식 연결, 원소 타입 설정 후 수식 보존과 크기 생략 배열을 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_type_table.c make test-sem TEST=8

테스트 코드	`void SEM_08_SetTypeExpr(void) {`
	`  A_TYPE *array = makeType(T_ARRAY);`
	
	`  A_NODE size_expr = {0};`
	`  size_expr.name = N_EXP_INT_CONST;`
	`  size_expr.clink = (A_NODE *)(long)10;`
	
	`  // 1. 배열 크기 수식 연결`
	`  A_TYPE *result = setTypeExpr(array, &size_expr);`
	
	`  assert(result == array);`
	`  assert(array->kind == T_ARRAY);`
	`  assert(array->expr == &size_expr);`
	
	`  // 2. 원소 타입 연결`
	`  A_TYPE *int_type = makeType(T_ENUM);`
	
	`  setTypeElementType(array, int_type);`
	
	`  assert(array->element_type == int_type);`
	`  assert(array->expr == &size_expr);`
	
	`  // 3. 크기가 생략된 배열`
	`  A_TYPE *unsized_array = makeType(T_ARRAY);`
	
	`  result = setTypeExpr(unsized_array, NULL);`
	
	`  assert(result == unsized_array);`
	`  assert(unsized_array->expr == NULL);`
	
	`  // 메모리 해제`
	`  free(unsized_array);`
	`  free(int_type);`
	`  free(array);`
	
	`}`

실행 결과 종료 코드 0	`PASS [SEM:08] SEM_08_SetTypeExpr  set type expression` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 09. 필드·매개변수 목록

의도 / 기준	구조체 필드와 함수 매개변수 목록 연결, 빈 목록과 element_type 보존을 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_type_table.c make test-sem TEST=9
테스트 코드	

```c
void SEM_09_SetTypeField(void) {
  assert(current_id == NULL);
  assert(current_level == 0);

  // 1. 구조체 타입 생성
  A_TYPE *struct_type = makeType(T_STRUCT);

  A_ID *x = makeIdentifier("x");
  A_ID *y = makeIdentifier("y");

  A_ID *fields = linkDeclaratorList(x, y);

  A_TYPE *result = setTypeField(struct_type, fields);

  assert(result == struct_type);
  assert(struct_type->kind == T_STRUCT);
  assert(struct_type->field == x);
  assert(struct_type->field->link == y);
  assert(y->link == NULL);

  // 2. 함수 타입의 파라미터 목록
  A_TYPE *func_type = makeType(T_FUNC);

  A_ID *param = makeIdentifier("param");

  result = setTypeField(func_type, param);

  assert(result == func_type);
  assert(func_type->field == param);

  // 3. 빈 파라미터 목록
  A_TYPE *empty_func = makeType(T_FUNC);

  result = setTypeField(empty_func, NULL);

  assert(result == empty_func);
  assert(empty_func->field == NULL);

  // 4. 기존 타입 체인에는 영향이 없는지 확인
  assert(struct_type->element_type == NULL);
  assert(func_type->element_type == NULL);

  // 메모리 해제
  free(empty_func);
  free(func_type);
  free(struct_type);

  free(param->name);
  free(param);
  free(y->name);
  free(y);
  free(x->name);
  free(x);

  current_id = NULL;
  current_level = 0;

}
```

실행 결과 종료 코드 0	PASS [SEM:09] SEM_09_SetTypeField set type field SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 10. 선언 지정자 생성

의도 / 기준	타입·저장 클래스 조합, NULL 조합, 독립된 지정자 객체와 타입 공유를 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=10
테스트 코드	

```c
void SEM_10_MakeSpecifier(void) {
  A_TYPE *int_type = makeType(T_ENUM);

  // 1. 타입만 있는 선언: int a;
  A_SPECIFIER *s1 = makeSpecifier(int_type, S_NULL);

  assert(s1 != NULL);
  assert(s1->type == int_type);
  assert(s1->stor == S_NULL);
  assert(s1->line == 0);

  // 2. 타입과 저장 클래스: static int b;
  A_SPECIFIER *s2 = makeSpecifier(int_type, S_STATIC);

  assert(s2 != NULL);
  assert(s2->type == int_type);
  assert(s2->stor == S_STATIC);

  // 3. 저장 클래스만 있는 상태
  A_SPECIFIER *s3 = makeSpecifier(NULL, S_TYPEDEF);

  assert(s3 != NULL);
  assert(s3->type == NULL);
  assert(s3->stor == S_TYPEDEF);

  // 4. 둘 다 지정되지 않은 상태
  A_SPECIFIER *s4 = makeSpecifier(NULL, S_NULL);

  assert(s4 != NULL);
  assert(s4->type == NULL);
  assert(s4->stor == S_NULL);

  // 5. 서로 독립적인 객체인지 확인
  assert(s1 != s2);
  assert(s2 != s3);
  assert(s3 != s4);

  // 6. 타입 객체는 공유되는지 확인
  assert(s1->type == s2->type);

  free(s1);
  free(s2);
  free(s3);
  free(s4);
  free(int_type);

}
```

실행 결과 종료 코드 0	PASS [SEM:10] SEM_10_MakeSpecifier make specifier SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 11. 선언 지정자 갱신

의도 / 기준	지정자에 타입 또는 저장 클래스를 추가하고 기존 객체를 재사용하는지 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=11

테스트 코드	`void SEM_11_UpdateSpecifier(void) {`
	`  A_TYPE *int_type = makeType(T_ENUM);`
	
	`  // 1. 타입만 있는 명시자에 static 추가`
	`  A_SPECIFIER *s1 = makeSpecifier(int_type, S_NULL);`
	
	`  A_SPECIFIER *result = updateSpecifier(s1, NULL, S_STATIC);`
	
	`  assert(result == s1);`
	`  assert(s1->type == int_type);`
	`  assert(s1->stor == S_STATIC);`
	
	`  // 2. 저장 클래스만 있는 명시자에 타입 추가`
	`  A_SPECIFIER *s2 = makeSpecifier(NULL, S_TYPEDEF);`
	
	`  result = updateSpecifier(s2, int_type, S_NULL);`
	
	`  assert(result == s2);`
	`  assert(s2->type == int_type);`
	`  assert(s2->stor == S_TYPEDEF);`
	
	`  // 3. 아무 정보가 없는 명시자에 타입 추가`
	`  A_SPECIFIER *s3 = makeSpecifier(NULL, S_NULL);`
	
	`  updateSpecifier(s3, int_type, S_NULL);`
	
	`  assert(s3->type == int_type);`
	`  assert(s3->stor == S_NULL);`
	
	`  // 4. 아무 정보가 없는 명시자에 static 추가`
	`  A_SPECIFIER *s4 = makeSpecifier(NULL, S_NULL);`
	
	`  updateSpecifier(s4, NULL, S_STATIC);`
	
	`  assert(s4->type == NULL);`
	`  assert(s4->stor == S_STATIC);`
	
	`  // 5. 기존 명시자 객체를 재사용하는지 검증`
	`  assert(s1 != s2);`
	`  assert(s2 != s3);`
	`  assert(s3 != s4);`
	
	`  free(s1);`
	`  free(s2);`
	`  free(s3);`
	`  free(s4);`
	`  free(int_type);`
	
	`}`

실행 결과 종료 코드 0	`PASS [SEM:11] SEM_11_UpdateSpecifier  update specifier` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 12. 기본 타입 설정

의도 / 기준	타입 생략 시 기본 int를 적용하고 기존 타입·저장 클래스를 유지하는지 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=12

테스트 코드	`void SEM_12_SetDefaultSpecifier(void) {`
	`  // 아직 initialize()를 구현하지 않았으므로`
	`  // 테스트에서 기본 int 타입을 임시 생성`
	`  int_type = makeType(T_ENUM);`
	
	`  // 1. 타입이 없는 경우 기본 int 적용`
	`  A_SPECIFIER *s1 = makeSpecifier(NULL, S_STATIC);`
	
	`  setDefaultSpecifier(s1);`
	
	`  assert(s1->type == int_type);`
	`  assert(s1->stor == S_STATIC);`
	
	`  // 2. 타입이 이미 지정된 경우 유지`
	`  A_TYPE *char_type = makeType(T_ENUM);`
	`  A_SPECIFIER *s2 = makeSpecifier(char_type, S_NULL);`
	
	`  setDefaultSpecifier(s2);`
	
	`  assert(s2->type == char_type);`
	`  assert(s2->type != int_type);`
	
	`  // 3. 타입과 저장 클래스 모두 없는 경우`
	`  A_SPECIFIER *s3 = makeSpecifier(NULL, S_NULL);`
	
	`  setDefaultSpecifier(s3);`
	
	`  assert(s3->type == int_type);`
	`  assert(s3->stor == S_NULL);`
	
	`  // 메모리 해제`
	`  free(s1);`
	`  free(s2);`
	`  free(s3);`
	`  free(char_type);`
	`  free(int_type);`
	
	`  int_type = NULL;`
	
	`}`

실행 결과 종료 코드 0	`PASS [SEM:12] SEM_12_SetDefaultSpecifier  set default specifier` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 13. 의미 분석 환경 초기화

의도 / 기준	기본 타입, char 포인터, printf·scanf와 전역 식별자 등록을 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_symbol_table.c make test-sem TEST=13

테스트 코드	`void SEM_13_Initialize(void) {`
	`  assert(current_id == NULL);`
	`  assert(current_level == 0);`
	`  assert(int_type == NULL);`
	`  initialize();`
	`  // 기본 타입 생성 확인`
	`  assert(int_type != NULL);`
	`  assert(float_type != NULL);`
	`  assert(char_type != NULL);`
	`  assert(void_type != NULL);`
	`  assert(string_type != NULL);`
	`  assert(int_type->kind == T_ENUM);`
	`  assert(float_type->kind == T_ENUM);`
	`  assert(char_type->kind == T_ENUM);`
	`  assert(void_type->kind == T_VOID);`
	`  assert(string_type->kind == T_POINTER);`
	`  assert(string_type->element_type == char_type);`
	`  // 기본 타입 명칭 검색`
	`  A_ID *id = searchIdentifier("int");`
	`  assert(id != NULL);`
	`  assert(id->kind == ID_TYPE);`
	`  assert(id->type == int_type);`
	`  id = searchIdentifier("float");`
	`  assert(id != NULL);`
	`  assert(id->type == float_type);`
	`  id = searchIdentifier("char");`
	`  assert(id != NULL);`
	`  assert(id->type == char_type);`
	`  id = searchIdentifier("void");`
	`  assert(id != NULL);`
	`  assert(id->type == void_type);`
	`  // 기본 함수 검색`
	`  id = searchIdentifier("printf");`
	`  assert(id != NULL);`
	`  assert(id->kind == ID_FUNC);`
	`  assert(id->type->kind == T_FUNC);`
	`  assert(id->type->element_type == int_type);`
	`  id = searchIdentifier("scanf");`
	`  assert(id != NULL);`
	`  assert(id->kind == ID_FUNC);`
	`  // 모든 기본 명칭은 전역 레벨에 등록됨`
	`  assert(id->level == 0);`
	`  // 테스트 전역 상태 정리`
	`  A_ID *current = current_id;`
	`  while (current != NULL) {`
	`  A_ID *prev = current->prev;`
	`  if (current->kind == ID_FUNC)`
	`  free(current->type);`
	`  free(current->name);`
	`  free(current);`
	`  current = prev;`
	`  }`
	`  current_id = NULL;`
	`  free(string_type);`
	`  free(void_type);`
	`  free(char_type);`
	`  free(float_type);`
	`  free(int_type);`
	`  string_type = NULL;`
	`  void_type = NULL;`
	`  char_type = NULL;`
	`  float_type = NULL;`
	`  int_type = NULL;`
	`}`

실행 결과 종료 코드 0	`PASS [SEM:13] SEM_13_Initialize  initialize` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 14. 선언 목록 지정자 적용

의도 / 기준	static 선언, 포인터, typedef, 기본 S_AUTO와 내부 스코프 이름 가림을 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=14
테스트 코드	

```c
void SEM_14_SetDeclaratorListSpecifier(void) {
  assert(current_id == NULL);
  assert(current_level == 0);
  assert(int_type == NULL);
  initialize();
  // 1. static int a, *p;
  A_ID *a = makeIdentifier("a");
  A_ID *p = makeIdentifier("p");
  setDeclaratorElementType(p, makeType(T_POINTER));
  A_ID *list = linkDeclaratorList(a, p);
  A_SPECIFIER *specifier = makeSpecifier(int_type, S_STATIC);
  A_ID *result = setDeclaratorListSpecifier(list, specifier);
  assert(result == a);
  assert(a->kind == ID_VAR);
  assert(a->type == int_type);
  assert(a->specifier == S_STATIC);
  assert(p->kind == ID_VAR);
  assert(p->type->kind == T_POINTER);
  assert(p->type->element_type == int_type);
  assert(p->specifier == S_STATIC);
  // 2. typedef int NUMBER;
  A_ID *number = makeIdentifier("NUMBER");
  A_SPECIFIER *typedef_spec = makeSpecifier(int_type, S_TYPEDEF);
  setDeclaratorListSpecifier(number, typedef_spec);
  assert(number->kind == ID_TYPE);
  assert(number->type == int_type);
  assert(number->specifier == S_TYPEDEF);
  // 3. int value; (저장 클래스 기본값)
  A_ID *value = makeIdentifier("value");
  A_SPECIFIER *default_spec = makeSpecifier(int_type, S_NULL);
  setDeclaratorListSpecifier(value, default_spec);
  assert(value->kind == ID_VAR);
  assert(value->type == int_type);
  assert(value->specifier == S_AUTO);
  // 4. 다른 스코프에서 동일 명칭 선언
  enterScope();
  A_ID *local_a = makeIdentifier("a");
  A_SPECIFIER *local_spec = makeSpecifier(int_type, S_NULL);
  setDeclaratorListSpecifier(local_a, local_spec);
  assert(local_a->kind == ID_VAR);
  assert(local_a->type == int_type);
  assert(local_a->level == 1);
  assert(searchIdentifier("a") == local_a);
  leaveScope();
  assert(searchIdentifier("a") == a);
  // 5. 테스트에서 생성한 객체 해제
  free(specifier);
  free(typedef_spec);
  free(default_spec);
  free(local_spec);
  free(local_a->name);
  free(local_a);
  // 기본 타입은 공유되므로 여기서 해제하지 않음
  free(p->type);
  // 전역 심볼 체인 해제
  A_ID *current = current_id;
  while (current != NULL) {
    A_ID *prev = current->prev;
    if (current->kind == ID_FUNC)
      free(current->type);
    free(current->name);
    free(current);
    current = prev;
  }
  current_id = NULL;
  free(string_type);
  free(void_type);
  free(char_type);
  free(float_type);
  free(int_type);
  string_type = NULL;
  void_type = NULL;
  char_type = NULL;
  float_type = NULL;
  int_type = NULL;
}
```

실행 결과 종료 코드 0	PASS [SEM:14] SEM_14_SetDeclaratorListSpecifier set declarator list specifier SUMMARY [SEM] 1 passed, 0 failed, 1 total

SEM 15. 함수 선언 지정자

의도 / 기준	함수 원형과 정의의 반환 타입, 매개변수 목록과 ID_FUNC 설정을 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=15

테스트 코드	`void SEM_15_SetFunctionDeclaratorSpecifier(void) {`
	`  assert(current_id == NULL);`
	`  assert(current_level == 0);`
	`  assert(int_type == NULL);`
	`  /* 테스트용 기본 타입 */`
	`  int_type = makeType(T_ENUM);`
	`  /* 1. int foo(); 프로토타입 등록 */`
	`  A_ID *prototype = makeIdentifier("foo");`
	`  setDeclaratorElementType(prototype, makeType(T_FUNC));`
	`  A_SPECIFIER *proto_spec = makeSpecifier(int_type, S_NULL);`
	`  setDeclaratorListSpecifier(prototype, proto_spec);`
	`  assert(prototype->kind == ID_FUNC);`
	`  assert(prototype->type->element_type == int_type);`
	`  /* 2. int foo() { ... } 함수 정의 분석 */`
	`  A_ID *definition = makeIdentifier("foo");`
	`  setDeclaratorElementType(definition, makeType(T_FUNC));`
	`  A_SPECIFIER *def_spec = makeSpecifier(int_type, S_NULL);`
	`  A_ID *result = setFunctionDeclaratorSpecifier(definition, def_spec);`
	`  assert(result == definition);`
	`  assert(definition->kind == ID_FUNC);`
	`  assert(definition->type->kind == T_FUNC);`
	`  assert(definition->type->element_type == int_type);`
	`  /* 3. int bar(int x) { ... } */`
	`  A_ID *bar = makeIdentifier("bar");`
	`  A_TYPE *bar_type = makeType(T_FUNC);`
	`  setDeclaratorElementType(bar, bar_type);`
	`  A_ID *param = makeIdentifier("x");`
	`  param->kind = ID_PARM;`
	`  param->type = int_type;`
	`  setTypeField(bar_type, param);`
	`  A_SPECIFIER *bar_spec = makeSpecifier(int_type, S_NULL);`
	`  result = setFunctionDeclaratorSpecifier(bar, bar_spec);`
	`  assert(result == bar);`
	`  assert(bar->kind == ID_FUNC);`
	`  assert(bar->type->field == param);`
	`  assert(bar->type->element_type == int_type);`
	`  assert(searchIdentifier("x") == param);`
	`  /* 테스트에서 생성한 객체 해제 */`
	`  free(proto_spec);`
	`  free(def_spec);`
	`  free(bar_spec);`
	`  A_ID *id = current_id;`
	`  while (id != NULL) {`
	`  A_ID *prev = id->prev;`
	`  if (id->kind == ID_FUNC)`
	`  free(id->type);`
	`  free(id->name);`
	`  free(id);`
	`  id = prev;`
	`  }`
	`  current_id = NULL;`
	`  free(int_type);`
	`  int_type = NULL;`
	`}`

실행 결과 종료 코드 0	`PASS [SEM:15] SEM_15_SetFunctionDeclaratorSpecifier  set function declarator specifier` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 16. 함수 본문 연결

의도 / 기준	함수 타입의 expr에 본문 AST를 연결하고 노드·함수 종류를 유지하는지 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=16

테스트 코드	`void SEM_16_SetFunctionDeclaratorBody(void) {`
	`  assert(current_id == NULL);`
	`  assert(current_level == 0);`
	
	`  A_ID *fun = makeIdentifier("fun");`
	
	`  A_TYPE *func_type = makeType(T_FUNC);`
	`  setDeclaratorElementType(fun, func_type);`
	
	`  // 함수 정의라고 가정`
	`  fun->kind = ID_FUNC;`
	
	`  // { return ...; } 에 해당하는 AST 루트`
	`  A_NODE *body = calloc(1, sizeof(A_NODE));`
	`  assert(body != NULL);`
	
	`  body->name = N_STMT_COMPOUND;`
	
	`  // 1. 몸체 연결`
	`  A_ID *result = setFunctionDeclaratorBody(fun, body);`
	
	`  assert(result == fun);`
	`  assert(fun->type == func_type);`
	`  assert(func_type->expr == body);`
	
	`  // 2. AST가 변경되지 않았는지 확인`
	`  assert(func_type->expr->name == N_STMT_COMPOUND);`
	
	`  // 3. 함수 타입 유지 확인`
	`  assert(fun->kind == ID_FUNC);`
	`  assert(fun->type->kind == T_FUNC);`
	
	`  // 메모리 해제`
	`  free(body);`
	`  free(func_type);`
	`  free(fun->name);`
	`  free(fun);`
	
	`  current_id = NULL;`
	
	`}`

실행 결과 종료 코드 0	`PASS [SEM:16] SEM_16_SetFunctionDeclaratorBody  set function declarator body` `SUMMARY [SEM] 1 passed, 0 failed, 1 total`

SEM 17. 매개변수 지정자

의도 / 기준	일반·포인터·익명 void 매개변수, 목록 연결과 스코프 종료 후 검색 해제를 확인하는 테스트입니다.
소스 / 명령	tests/sem/sem_declarator.c make test-sem TEST=17
테스트 코드	

```c
void SEM_17_SetParameterDeclaratorSpecifier(void) {
  assert(current_id == NULL);
  assert(current_level == 0);
  assert(int_type == NULL);
  initialize();
  enterScope();
  // 1. int a
  A_ID *a = makeIdentifier("a");
  A_SPECIFIER *s1 = makeSpecifier(int_type, S_NULL);
  A_ID *result = setParameterDeclaratorSpecifier(a, s1);
  assert(result == a);
  assert(a->kind == ID_PARM);
  assert(a->type == int_type);
  assert(a->level == 1);
  // 2. int *b
  A_ID *b = makeIdentifier("b");
  setDeclaratorElementType(b, makeType(T_POINTER));
  A_SPECIFIER *s2 = makeSpecifier(int_type, S_NULL);
  result = setParameterDeclaratorSpecifier(b, s2);
  assert(result == b);
  assert(b->kind == ID_PARM);
  assert(b->type->kind == T_POINTER);
  assert(b->type->element_type == int_type);
  // 3. 이름 없는 void 파라미터
  A_ID *anonymous = makeIdentifier("");
  A_SPECIFIER *s3 = makeSpecifier(void_type, S_NULL);
  result = setParameterDeclaratorSpecifier(anonymous, s3);
  assert(result == anonymous);
  assert(anonymous->kind == ID_PARM);
  assert(anonymous->type == void_type);
  // 4. 이름 검색
  assert(searchIdentifier("a") == a);
  assert(searchIdentifier("b") == b);
  // 5. 파라미터 목록 연결
  A_ID *params = linkDeclaratorList(a, b);
  assert(params == a);
  assert(a->link == b);
  // 스코프 종료
  leaveScope();
  assert(current_level == 0);
  assert(searchIdentifier("a") == NULL);
  assert(searchIdentifier("b") == NULL);
  // 테스트 객체 해제
  free(s1);
  free(s2);
  free(s3);
  free(b->type);
  free(anonymous->name);
  free(anonymous);
  free(b->name);
  free(b);
  free(a->name);
  free(a);
  // initialize()에서 생성한 전역 심볼 해제
  A_ID *current = current_id;
  while (current != NULL) {
    A_ID *prev = current->prev;
    if (current->kind == ID_FUNC)
      free(current->type);
    free(current->name);
    free(current);
    current = prev;
  }
  current_id = NULL;
  free(string_type);
  free(void_type);
  free(char_type);
  free(float_type);
  free(int_type);
  string_type = NULL;
  void_type = NULL;
  char_type = NULL;
  float_type = NULL;
  int_type = NULL;
}
```

실행 결과 종료 코드 0	PASS [SEM:17] SEM_17_SetParameterDeclaratorSpecifier set parameter declarator specifier SUMMARY [SEM] 1 passed, 0 failed, 1 total

2. AST, 파서 통합 테스트

AST 01. 함수 정의와 호출

의도 / 기준	함수 정의, 두 인자의 호출과 반환값 대입을 수용하는지 확인하는 테스트입니다. 기대 종료 코드는 0입니다.
소스 / 명령	tests/ast/valid/02_functions.c make test-ast TEST=1

입력 코드	`int add(int left, int right)`
	`{`
	`  return left + right;`
	`}`
	
	`int main(void)`
	`{`
	`  int result;`
	`  result = add(2, 3);`
	`  return result;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 02. 제어 흐름 통합

의도 / 기준	while·do-while·for, if-else, switch와 break·continue를 수용하는지 확인하는 테스트입니다. 기대 종료 코드는 0입니다.
소스 / 명령	tests/ast/valid/03_control_flow.c make test-ast TEST=2

입력 코드	`int main(void)`
	`{`
	`  int i;`
	`  int sum;`
	`  i = 0;`
	`  sum = 0;`
	
	`  while (i < 3) {`
	`  sum = sum + i;`
	`  i++;`
	`  }`
	
	`  do {`
	`  sum = sum - 1;`
	`  } while (sum > 10);`
	
	`  for (i = 0; i < 5; i++) {`
	`  if (i == 2)`
	`  continue;`
	`  else`
	`  sum = sum + i;`
	`  }`
	
	`  switch (sum) {`
	`  case 0:`
	`  sum = 1;`
	`  break;`
	`  default:`
	`  break;`
	`  }`
	
	`  return sum;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 03. 재귀 함수

의도 / 기준	factorial의 자기 호출과 중첩 수식을 수용하는지 확인하는 테스트입니다. 기대 종료 코드는 0입니다.
소스 / 명령	tests/ast/valid/08_recursion.c make test-ast TEST=3

입력 코드	`int factorial(int value)`
	`{`
	`    if (value <= 1)`
	`        return 1;`
	`    return value * factorial(value - 1);`
	`}`
	
	`int main(void)`
	`{`
	`    return factorial(5);`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 04. 공용체와 멤버 접근

의도 / 기준	공용체 정의, 전역·지역 변수와 멤버 대입·참조를 수용하는지 확인하는 테스트입니다. 기대 종료 코드는 0입니다.
소스 / 명령	tests/ast/valid/10_union.c make test-ast TEST=4

입력 코드	`union Number {`
	`    int integer_value;`
	`    float float_value;`
	`} number;`
	
	`int main(void)`
	`{`
	`    union Number local;`
	`    local.integer_value = 42;`
	`    number.float_value = 1.5;`
	`    return local.integer_value;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 05. 포인터 연산과 매개변수

의도 / 기준	포인터 매개변수, 역참조 대입과 주소 전달을 수용하는지 확인하는 테스트입니다. 기대 종료 코드는 0입니다.
소스 / 명령	tests/ast/valid/11_pointers.c make test-ast TEST=5

입력 코드	`void swap(int *left, int *right)`
	`{`
	`    int temporary;`
	`    temporary = *left;`
	`    *left = *right;`
	`    *right = temporary;`
	`    return;`
	`}`
	
	`int main(void)`
	`{`
	`    int first;`
	`    int second;`
	`    first = 10;`
	`    second = 20;`
	`    swap(&first, &second);`
	`    return first;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 06. 중첩 조건문

의도 / 기준	중첩 if-else와 여러 매개변수의 함수 호출을 수용하는지 확인하는 테스트입니다. 기대 종료 코드는 0입니다.
소스 / 명령	tests/ast/valid/12_nested_conditionals.c make test-ast TEST=6

입력 코드	`int choose(int first, int second, int enabled)`
	`{`
	`  if (enabled)`
	`  if (first > second)`
	`  return first;`
	`  else`
	`  return second;`
	`  else`
	`  return 0;`
	`}`
	
	`int main(void)`
	`{`
	`  return choose(4, 7, 1);`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 07. 형 변환과 **sizeof**

의도 / 기준	형 변환과 sizeof 수식이 포함된 입력을 수용하는지 확인하는 테스트입니다. 기대 종료 코드는 0입니다.
소스 / 명령	tests/ast/valid/14_cast_and_sizeof.c make test-ast TEST=7

입력 코드	`int main(void)`
	`{`
	`  double value;`
	`  int result;`
	`  value = 4.5;`
	`  result = (int)value + sizeof(value);`
	`  return result;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 08. 세미콜론 누락

의도 / 기준	변수 선언 뒤 세미콜론 누락을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/01_missing_semicolon.c make test-ast TEST=8

입력 코드	`int main(void)`
	`{`
	`  int value`
	`  value = 1;`
	`  return value;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 09. 불완전한 수식

의도 / 기준	덧셈 연산자의 오른쪽 피연산자 누락을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/02_incomplete_expression.c make test-ast TEST=9

입력 코드	`int main(void)`
	`{`
	`    return 1 + ;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 10. 블록 닫기 누락

의도 / 기준	함수 본문의 닫는 중괄호 누락을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/03_unclosed_block.c make test-ast TEST=10

입력 코드	`int main(void)`
	`{`
	`    int value;`
	`    value = 1;`
	`    return value;`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 11. 문장 뒤 선언

의도 / 기준	이 파서의 선언부·문장부 순서에 맞지 않는 문장 뒤 선언을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/04_declaration_after_statement.c make test-ast TEST=11

입력 코드	`int main(void)`
	`{`
	`    int first;`
	`    first = 1;`
	`    int second;`
	`    return first;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 12. 인자 목록 끝 쉼표

의도 / 기준	함수 호출의 쉼표 뒤 인자 누락을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/05_trailing_argument_comma.c make test-ast TEST=12

입력 코드	`int add(int left, int right)`
	`{`
	`    return left + right;`
	`}`
	
	`int main(void)`
	`{`
	`    return add(1, );`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 13. if 조건 괄호 누락

의도 / 기준	if 조건을 감싸는 괄호 누락을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/06_if_without_parentheses.c make test-ast TEST=13

입력 코드	`int main(void)`
	`{`
	`    int value;`
	`    value = 1;`
	`    if value`
	`        return 1;`
	`    return 0;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 14. 수식 괄호 닫기 누락

의도 / 기준	수식의 닫는 소괄호 누락을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/07_unclosed_parenthesis.c make test-ast TEST=14

입력 코드	`int main(void)`
	`{`
	`    int value;`
	`    value = (1 + 2;`
	`    return value;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 15. 배열 선언자 오류

의도 / 기준	배열 선언자의 닫는 대괄호 누락을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/08_bad_array_declarator.c make test-ast TEST=15

입력 코드	`int values[10;`
	
	`int main(void)`
	`{`
	`    return 0;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 16. 매개변수 쉼표 누락

의도 / 기준	함수 매개변수 사이 쉼표 누락을 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/09_missing_parameter_comma.c make test-ast TEST=16

입력 코드	`int add(int left int right)`
	`{`
	`    return left + right;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

AST 17. if 없는 **else**

의도 / 기준	대응하는 if가 없는 else를 거부하는지 확인하는 테스트입니다. 기대 종료 코드는 0이 아닌 값이며, 오류 거부가 테스트의 통과 조건입니다.
소스 / 명령	tests/ast/invalid/10_else_without_if.c make test-ast TEST=17

입력 코드	`int main(void)`
	`{`
	`    else`
	`        return 1;`
	`    return 0;`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

3. ASTEXP, 기대 AST 검증

ASTEXP 01. 연산자 우선순위

의도 / 기준	덧셈 루트의 왼쪽이 1, 오른쪽이 2×3인 AST와 정수값을 검사하는 테스트입니다.
소스 / 명령	tests/ast/cases/03_precedence.c make test-ast-exp TEST=1

입력 코드	`int main(void)`
	`{`
	`    return 1 + 2 * 3;`
	`}`

검증 함수	`void ASTEXP_01_Precedence(void) {`
	`  AST_03_Precedence(expression_root);`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

ASTEXP 02. if-else 구조

의도 / 기준	N_STMT_IF_ELSE의 조건이 N_EXP_GTR이고 두 분기가 N_STMT_RETURN인지 검사하는 테스트입니다.
소스 / 명령	tests/ast/cases/07_if_else.c make test-ast-exp TEST=2

입력 코드	`int main(void)`
	`{`
	`    int x;`
	`    x = 1;`
	`    if (x > 0)`
	`        return 1;`
	`    else`
	`        return 0;`
	`}`

검증 함수	`void ASTEXP_02_IfElse(void) {`
	`  AST_36_Selection(2);`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

ASTEXP 03. for 구조

의도 / 기준	N_STMT_FOR, N_FOR_EXP와 초기식·조건식·증감식 연결을 검사하는 테스트입니다. 정적 점검상 입력의 선행 대입문·수식 본문과 검사기의 첫 문장·빈 본문 요구가 불일치입니다.
소스 / 명령	tests/ast/cases/09_for.c make test-ast-exp TEST=3

입력 코드	`int main(void)`
	`{`
	`  int i;`
	`  int sum;`
	`  i = 0;`
	`  sum = 0;`
	`  for (i = 0; i < 10; i++)`
	`  sum = sum + i;`
	`  return sum;`
	`}`

검증 함수	`void ASTEXP_03_For(void) {`
	`  AST_38_Iteration(3);`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

ASTEXP 04. 함수 원형

의도 / 기준	add→main 전역 목록, 반환 타입, 원형 매개변수 a·b의 레벨 1과 main 본문을 검사하는 테스트입니다.
소스 / 명령	tests/ast/cases/24_function_prototype.c make test-ast-exp TEST=4

입력 코드	`int add(int a, int b);`
	`int main(void) {`
	`  return 0;`
	`}`

검증 함수	`void ASTEXP_04_FunctionPrototype(void) {`
	`  AST_24_FunctionPrototype();`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

ASTEXP 05. 함수 매개변수와 스코프

의도 / 기준	a·b의 이름·타입·목록, 반환 수식의 식별자 참조와 함수 종료 후 매개변수 검색 해제를 검사하는 테스트입니다.
소스 / 명령	tests/ast/cases/25_function_parameters.c make test-ast-exp TEST=5

입력 코드	`int add(int a, int b) {`
	`  return a + b;`
	`}`

검증 함수	`void ASTEXP_05_FunctionParameters(void) {`
	`  AST_25_FunctionParameters();`
	`  AST_25_FunctionScope();`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

ASTEXP 06. 중첩 스코프의 이름 가림

의도 / 기준	서로 다른 x의 레벨 0·1·2, 전역 검색 복원과 return의 함수 지역 x 참조를 검사하는 테스트입니다.
소스 / 명령	tests/ast/cases/44_scope_shadowing.c make test-ast-exp TEST=6

입력 코드	`int x;`
	`int main(void) {`
	`  int x;`
	`  x = 1;`
	`  {`
	`  int x;`
	`  x = 2;`
	`  }`
	`  return x;`
	`}`

검증 함수	`void ASTEXP_06_ScopeShadowing(void) {`
	`  AST_44_BlockShadowing();`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

ASTEXP 07. 함수 선언자

의도 / 기준	f의 ID_FUNC·T_FUNC, int 반환 타입과 빈 매개변수 목록을 검사하는 테스트입니다.
소스 / 명령	tests/ast/cases/59_function_declarator.c make test-ast-exp TEST=7
입력 코드	`int f();`

검증 함수	`void ASTEXP_07_FunctionDeclarator(void) {`
	`  AST_59_FunctionDeclarator();`
	`}`

실행 결과	미실행입니다. Bison·Flex 미설치로 현재 파서를 빌드하지 못한 상태입니다.

컴파일러 W4 오류 및 해결 기록

[기록 시점 구분] 아래는 개발 당시 오류와 수정안을 보존한 기록입니다. 당시의 '미해결' 표기는 최신 판정과 구별해야 합니다. 2026.11.07 Windows 전체 회귀는 별도로 72건 통과, 0건 실패로 기록되어 있습니다.

콘솔 로그는 원문입니다. 해결 기록이 없는 항목은 수정안입니다.

---

01. 초기 빌드**:** 미구현 함수와 링크 오류

문제 / 원인	함수 구현 누락과 makeType·int_type의 정의를 찾지 못한 빌드 오류입니다.
문제 코드 실행 출력	In file included from C:\Users\egod1\proj\compiler\w4\test.c:3: C:\Users\egod1\proj\compiler\w4\test.c: In function 'main': C:\Users\egod1\proj\compiler\w4\test.c:11:18: warning: comparison between pointer and integer 11 | assert(t->kind != NULL); | ^~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c: In function 'makeNode': C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:28: warning: unused parameter 'name' [-Wunused-paramete] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ~~~~~~~~~~^~~~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:42: warning: unused parameter 'left' [-Wunused-paramete] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ~~~~~~~~^~~~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:56: warning: unused parameter 'center' [-Wunused-parameter] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ~~~~~~~~^~~~~~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:72: warning: unused parameter 'right' [-Wunused-parameter] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ~~~~~~~~^~~~~ C:\Users\egod1\proj\compiler\w4\syntax_tree.c:4:80: warning: control reaches end of non-void function [-Wreturn-type] 4 | A_NODE *makeNode(NODE_NAME name, A_NODE *left, A_NODE *center, A_NODE *right) {} | ^ C:/msys64/mingw64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\cc6VZ5qo.o:test.c:(.text+0x13): undefined reference to `makeType' collect2.exe: error: ld returned 1 exit status PS C:\Users\egod1\proj\compiler\w4> PS C:\Users\egod1\proj\compiler\w4> ./run C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe: C:\Users\egod1\AppData\Local\Temp\ccuBiCXf.o:test.c:(.rdata$.refptr.int_type[.refptr.int_type]+0x0): undefined reference to `int_type' collect2.exe: error: ld returned 1 exit status
해결 / 상태	함수 정의·링크 대상과 **int_type**의 정의·초기화를 확인할 항목입니다.

02. Specifier 처리: NULL 타입 오류를 GDB로 추적

문제 / 원인	Specifier 처리 중 **NULL** 타입이 전달된 오류입니다.
문제 코드 실행 출력	

```
PS C:\Users\egod1\proj\compiler\w4> ./run
makeType test passed
makeIdentifier test passed
searchIdentifier test passed
testScope passed
testLinkDeclaratorList passed
testSetTypeElementType passed
testSetDeclaratorElementType passed
testSetTypeExpr passed
testSetTypeField passed
Semantic error: NULL type in setTypeField

PS C:\Users\egod1\proj\compiler\w4> gdb ./test.exe
GNU gdb (GDB) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-w64-mingw32".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test.exe...
(gdb) break semanticError
Breakpoint 1 at 0x14000342e: file C:\Users\egod1\proj\compiler\w4\semantic.c, line 111.
(gdb) run
Starting program: C:\Users\egod1\proj\compiler\w4\test.exe
[New Thread 13820.0x657c]
makeType test passed
makeIdentifier test passed
searchIdentifier test passed
testScope passed
testLinkDeclaratorList passed
testSetTypeElementType passed
testSetDeclaratorElementType passed
testSetTypeExpr passed
testSetTypeField passed

Thread 1 hit Breakpoint 1, semanticError (message=0x7ff767b16a3d "NULL type in setTypeField")
    at C:\Users\egod1\proj\compiler\w4\semantic.c:111
111       fprintf(stderr, "Semantic error: %s\n", message);
(gdb) bt
#0  semanticError (message=0x7ff767b16a3d "NULL type in setTypeField") at
C:\Users\egod1\proj\compiler\w4\semantic.c:111
#1  0x00007ff767b1363e in makeSpecifier (type=0x0, stor=S_TYPEDEF) at
C:\Users\egod1\proj\compiler\w4\semantic.c:186
#2  0x00007ff767b12b99 in testMakeSpecifier () at C:\Users\egod1\proj\compiler\w4\test.c:360
#3  0x00007ff767b130df in main () at C:\Users\egod1\proj\compiler\w4\test.c:451
(gdb) quit
A debugging session is active.

        Inferior 1 [process 13820] will be killed.
```

해결 / 상태	GDB의 중단점과 호출 스택을 확인하여 해결한 오류입니다.

03. 이름 없는 식별자 객체 미구현

문제 / 원인	이름 없는 식별자 객체가 구현되지 않은 상태입니다.
문제 코드 실행 출력	이름이 없는 식별자 객체는 구현 X -> 강의 자료에 나와있긴 한데, 정확한 구현이 없어서
해결 / 상태	이름 없는 식별자를 생성하는 기능의 구현이 필요한 상태입니다.

04. Lex 오타**: yytest**와 잘못된 대입 연산자

문제 / 원인	**yytext**의 이름과 대입 연산자를 잘못 작성한 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c -o parser.exe c.y: In function 'yyparse': c.y:308:52: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] 308	$$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL); | ^ c.y:308:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] 308 | $$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL); 	^ c.y:310:22: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast] 310	A_NODE *node = (A_NODE*)$$; 	^ c.y:311:55: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast] 311	fprintf(stderr, "[AST] N_EXP_INT_CONST: %ld\n", (long)node->clink); 	^ c.l: In function 'yylex': c.l:105:18: error: 'yytest' undeclared (first use in this function); did you mean 'yytext'? 105	yylval == atol(yytest); 	^~~~~ 	yytext c.l:105:18: note: each undeclared identifier is reported only once for each function it appears in lex.yy.c: At top level: lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function] 1572	#else 	^ lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function] 1529 | 	^ PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	**yytest**를 **yytext**로, **==**를 **=**로 변경하는 수정안입니다.
수정안	`yylval = atol(yytext);`

05. makeNode 중복 정의: 빌드 목록 중복

문제 / 원인	syntax_tree.c를 두 번 링크하여 발생한 중복 정의 오류입니다.
문제 코드 실행 출력	`gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c semantic.c syntax_tree.c -o parser.exe` 원문: multiple definition of `makeNode'; ... syntax_tree.c:4: first defined here 원문: collect2.exe: error: ld returned 1 exit status
해결 / 상태	빌드 명령에서 중복된 **syntax_tree.c** 하나를 제거하는 수정안입니다.
수정안	`gcc -Wall -Wextra -g -O0 y.tab.c lex.yy.c syntax_tree.c semantic.c -o parser.exe`

06. 조건식의 비트 OR와 논리 OR 혼동

문제 / 원인	논리 **OR** 대신 비트 OR를 사용한 오류입니다.
문제 코드 실행 출력	원문: semantic.c:418:16: warning: suggest parentheses around comparison in operand of '|' [-Wparentheses] 문제 코드: `if (id->kind != ID_FUNC | id->type == NULL || id->type->kind != T_FUNC)`
해결 / 상태	**|**를 **||**로 변경하는 수정안입니다.
수정안	`if (id->kind != ID_FUNC || id->type == NULL || id->type->kind != T_FUNC)`

07. YYSTYPE: 포인터와 정수 크기 불일치

문제 / 원인	**long**과 포인터의 크기 차이로 발생한 형 변환 경고입니다.

문제 코드
실행 출력
	`c.y: In function 'yyparse':`
	`c.y:308:52: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast]`
	`  308 |  $$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL);`
	`  \|  ^`
	`c.y:308:14: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]`
	`  308 \|  $$ =(long)makeNode(N_EXP_INT_CONST, NULL, (A_NODE*)$1, NULL);`
	`  |  ^`
	`c.y:310:22: warning: cast to pointer from integer of different size [-Wint-to-pointer-cast]`
	`  310 |  A_NODE *node = (A_NODE*)$$;`
	`  |  ^`
	`c.y:311:55: warning: cast from pointer to integer of different size [-Wpointer-to-int-cast]`
	`  311 |  fprintf(stderr, "[AST] N_EXP_INT_CONST: %ld\n", (long)node->clink);`
	`  |  ^`

해결 / 상태	포인터 너비의 **YYSTYPE**을 사용하고 **Lex/Yacc** 선언을 일치시키는 수정안입니다.

수정안	`#include <stdint.h>`
	`typedef intptr_t YYSTYPE;`
	`#define YY(x) ((YYSTYPE)(intptr_t)(x))`
	`#define PTR(T, x) ((T *)(intptr_t)(x))`

08. Flex 생성 코드의 미사용 함수 경고

문제 / 원인	Flex에서 생성된 **input·yyunput**이 사용되지 않은 경고입니다.

문제 코드
실행 출력
	`lex.yy.c: At top level:`
	`lex.yy.c:1572:16: warning: 'input' defined but not used [-Wunused-function]`
	` 1572 | #else`
	`      |                ^`
	`lex.yy.c:1529:17: warning: 'yyunput' defined but not used [-Wunused-function]`
	` 1529 |`
	`      |                 ^`

해결 / 상태	비치명적 경고이며, 아래 옵션으로 미사용 함수 생성을 끄는 방안입니다.
수정안	`%option noinput nounput`

09. Yacc 기호명·함수명 오타와 규칙 경계 오류

문제 / 원인	Yacc 기호명·함수명 오타와 문법 규칙 경계의 오류입니다.

문제 코드
실행 출력
	`PS C:\Users\egod1\proj\compiler\w4> $env:AST_TEST_IDENTIFIER = "1"`
	`PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/05_identifier.c`
	`C:\\Users\\egod1\\proj\\compiler\\w4\\c.y:270.10-51: error: symbol`
	`'parameter_ttestUpdateSpecifierype_list_opt' is used, but`
	`is not defined as a token and has no rules`
	`  270 |      | LP parameter_ttestUpdateSpecifierype_list_opt RP`
	`      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~`
	`[ERROR] bison failed. See "C:\Users\egod1\proj\compiler\w4\build.log".`
	`PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/05_identifier.c`
	`C:\Users\egod1\proj\compiler\w4\c.y: In function 'yyparse':`
	`C:\Users\egod1\proj\compiler\w4\c.y:124:36: error: implicit declaration of function 'linkDeclarator'; did`
	`you mean 'linkDeclaratorList'? [-Wimplicit-function-declaration]`
	`  124 |       $$ = (YYSTYPE) (intptr_t) linkDeclarator(`
	`      |                                    ^~~~~~~~~~~~`
	`      |                                    linkDeclaratorList`
	`lex.yy.c:1571:16: warning: 'input' defined but not used [-Wunused-function]`
	` 1571 | #else`
	`      |                ^`
	`lex.yy.c:1528:17: warning: 'yyunput' defined but not used [-Wunused-function]`
	` 1528 |`
	`      |                 ^`
	`[ERROR] gcc failed. See "C:\Users\egod1\proj\compiler\w4\build.log".`
	
	`C:\\Users\\egod1\\proj\\compiler\\w4\\c.y:418.1-17: error: unexpected identifier`
	`  418 | labeled_statement`
	`      | ^~~~~~~~~~~~~~~~~`
	`[ERROR] bison failed. See "C:\Users\egod1\proj\compiler\w4\build.log".`
	`PS C:\Users\egod1\proj\compiler\w4>`

해결 / 상태	기호명·함수명을 수정하고 앞 규칙의 **;** 및 **{}**를 점검할 항목입니다.

10. 중간 액션과 중첩 함수 호출의 괄호 위치

문제 / 원인	중간 액션과 중첩 함수 호출의 괄호 위치를 혼동한 오류입니다.
문제 코드 실행 출력	enum_specifier : ENUM_SYM IDENTIFIER {} LR enumerator_list RR | ENUM_SYM LR enumerator_list RR | ENUM_SYM IDENTIFIER ; C:\Users\egod1\proj\compiler\w4\c.y:459:61: error: too few arguments to function 'setTypeExpr'; expected 2, have 1 459 | $$ = YY(setTypeElementType(PTR(A_TYPE, $1), setTypeExpr(makeType(T_ARRAY)), \| ^~~~~~~~~~ C:\Users\egod1\proj\compiler\w4\parse_value.h:18:36: note: in definition of macro 'YY' 18 \| #define YY(x) ((YYSTYPE)(intptr_t)(x)) \| ^ In file included from C:\Users\egod1\proj\compiler\w4\c.y:7: C:\Users\egod1\proj\compiler\w4\semantic.h:33:9: note: declared here 33 \| A_TYPE *setTypeExpr(A_TYPE *type, A_NODE *expr); \| ^~~~~~~~~~ C:\Users\egod1\proj\compiler\w4\c.y:459:18: error: too many arguments to function 'setTypeElementType'; expected 2, have 3 459 \| $$ = YY(setTypeElementType(PTR(A_TYPE, $1), setTypeExpr(makeType(T_ARRAY)), | ^~~~~~~~~~~~~~~~~ C:\Users\egod1\proj\compiler\w4\parse_value.h:18:36: note: in definition of macro 'YY' 18 | #define YY(x) ((YYSTYPE)(intptr_t)(x)) | ^ C:\Users\egod1\proj\compiler\w4\semantic.h:28:9: note: declared here 28 | A_TYPE *setTypeElementType(A_TYPE *t1, A_TYPE *t2); | ^~~~~~~~~~~~~~~~~ lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function] 1625 | #else | ^ lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function] 1582 | | ^ [ERROR] gcc failed. See "C:\Users\egod1\proj\compiler\w4\build.log". PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	중간 액션을 구분하고, 함수별 인수 두 개를 정확히 묶는 수정안입니다.
수정안	setTypeExpr(type, expr); setTypeElementType(t1, t2);

11. 괄호 수식**:** 파싱 성공 후 **AST** 검증 실패

문제 / 원인	괄호 수식의 **AST**가 검증 조건과 일치하지 않은 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/04_parentheses.c lex.yy.c:1571:16: warning: 'input' defined but not used [-Wunused-function] 1571 | #else | ^ lex.yy.c:1528:17: warning: 'yyunput' defined but not used [-Wunused-function] 1528 | | ^ [DEBUG] N_EXP_INT_CONST: 1 [DEBUG] N_EXP_INT_CONST: 2 [DEBUG] N_EXP_INT_CONST: 3 Parsing success [ERROR] Semantic error: Expected N_EXP_ADD [ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\04_parentheses.c" PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	괄호 규칙과 **AST** 검증 조건을 대조할 항목입니다. 해결 여부는 미확인입니다.

12. for문: 잘못된 문법 규칙과 AST 연결

문제 / 원인	iteration_statement의 **for** 문법 규칙 작성 오류입니다.
문제 코드 실행 출력	[SUCCESS] Parser test passed: "C:\Users\egod1\proj\compiler\w4\tests\ast\39_do_while_statement.c" lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function] 1593 | #else | ^ lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function] 1550 | | ^ [DEBUG] N_EXP_INT_CONST: 0 [DEBUG] N_EXP_INT_CONST: 3 Parsing success [ERROR] Semantic error: Invalid for AST [ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\40_for_statement.c" Exception: Line | 7 | throw "FAILED: (case[1])" | ~~~~~~~~~~~~~~~~~~~~~~~~~~에서 | FAILED: 40_for_statement.c
해결 / 상태	**for** 규칙과 **AST** 연결을 수정할 항목입니다. 수정 후 통과 여부는 미기록입니다.

13. 함수 인자 목록: 종료 노드 불일치

문제 / 원인	함수 인자 목록의 종료 노드가 예상과 다른 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/26_function_call_args.c >> lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function] 1593 | #else | ^ lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function] 1550 | | ^ C:\Users\egod1\proj\compiler\w4\syntax_tree.c: In function 'makeNodeList': C:\Users\egod1\proj\compiler\w4\syntax_tree.c:28:13: warning: variable 'nil' set but not used [-Wunused-but-set-variable=] 28 | NODE_NAME nil; | ^~~ [DEBUG] N_EXP_INT_CONST: 1 [DEBUG] N_EXP_INT_CONST: 2 [ERROR] Semantic error: Invalid list terminator [ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\26_function_call_args.c" PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	GDB로 리스트 종료 노드를 확인할 항목입니다. 해결 여부는 미확인입니다.

14. sizeof(type): 타입 노드 연결 검증 실패

문제 / 원인	**sizeof(type)**의 **AST** 구조가 검증 조건과 다른 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> >> PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd tests/ast/51_sizeof_type.c >> lex.yy.c:1593:16: warning: 'input' defined but not used [-Wunused-function] 1593 | #else | ^ lex.yy.c:1550:17: warning: 'yyunput' defined but not used [-Wunused-function] 1550 | | ^ Parsing success [ERROR] Semantic error: Invalid sizeof type AST [ERROR] Parser test failed: "C:\Users\egod1\proj\compiler\w4\tests\ast\51_sizeof_type.c"
해결 / 상태	타입 정보와 **N_EXP_SIZE_TYPE** 연결을 확인할 항목입니다. 해결 여부는 미확인입니다.

15. static 처리 오류와 의도된 중복 지정자 오류

문제 / 원인	**static int**가 잘못 거부된 오류입니다. static auto의 거부는 의도된 결과입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> Set-Content "$env:TEMP\w4_static.c" 'static int counter; int main(void) { return counter; }' PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd "$env:TEMP\w4_static.c" lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function] 1625 | #else | ^ lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function] 1582 | | ^ [ERROR] Semantic error: Duplicate type specifier [ERROR] Parser test failed: "C:\Users\egod1\AppData\Local\Temp\w4_static.c" PS C:\Users\egod1\proj\compiler\w4> PS C:\Users\egod1\proj\compiler\w4> Set-Content "$env:TEMP\w4_storage_invalid.c" 'static auto int x;' PS C:\Users\egod1\proj\compiler\w4> .\parser.cmd "$env:TEMP\w4_storage_invalid.c" lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function] 1625 | #else | ^ lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function] 1582 | | ^ [ERROR] Semantic error: Duplicate storage class specifier [ERROR] Parser test failed: "C:\Users\egod1\AppData\Local\Temp\w4_storage_invalid.c" PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	타입 지정자와 저장 클래스 지정자의 중복 검사를 분리할 항목입니다.

16. 전역 선언 목록**: add** 함수 출력 누락

문제 / 원인	전역 선언 목록에서 **add** 함수가 출력되지 않은 오류입니다.
문제 코드 실행 출력	PS C:\Users\egod1\proj\compiler\w4> $env:AST_PRINT="1"; .\parser.cmd tests/valid/02_functions.c lex.yy.c:1625:16: warning: 'input' defined but not used [-Wunused-function] 1625 | #else | ^ lex.yy.c:1582:17: warning: 'yyunput' defined but not used [-Wunused-function] 1582 | | ^ [DEBUG] N_EXP_INT_CONST: 2 [DEBUG] N_EXP_INT_CONST: 3 Parsing success ======= syntax tree ========== N_PROGRAM (0000000000000000,0) | (ID="main") TYPE:000001fa9b2c1f80 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="") TYPE:000001fa9b2c7e20 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (void)| | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND (0000000000000000,0) | | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0 | | | | | | | TYPE | | | | | | | | (int) | | | | | | N_STMT_LIST (0000000000000000,0) | | | | | | | N_STMT_EXPRESSION (0000000000000000,0) | | | | | | | | N_EXP_ASSIGN (0000000000000000,0) | | | | | | | | | N_EXP_IDENT (0000000000000000,0) | | | | | | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0 | | | | | | | | | N_EXP_FUNCTION_CALL (0000000000000000,0) | | | | | | | | | | N_EXP_IDENT (0000000000000000,0) | | | | | | | | | | | (ID="add") TYPE:000001fa9b2c1690 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | | | | N_ARG_LIST (0000000000000000,0) | | | | | | | | | | N_EXP_INT_CONST (0000000000000000,0) | | | | | | | | | | | 2 | | | | | | | | | N_ARG_LIST (0000000000000000,0) | | | | | | | | | | N_EXP_INT_CONST (0000000000000000,0) | | | | | | | | | | | 3 | | | | | | | | | N_ARG_LIST_NIL (0000000000000000,0) | | | | | | | N_STMT_LIST (0000000000000000,0) | | | | | | | | N_STMT_RETURN (0000000000000000,0) | | | | | | | | | N_EXP_IDENT (0000000000000000,0) | | | | | | | | | | (ID="result") TYPE:000001fa9b2c7d60 KIND:VAR SPEC=AUTO LEV=1 VAL=0 ADDR=0 | | | | | | | N_STMT_LIST_NIL (0000000000000000,0) [SUCCESS] Parser test passed: "C:\Users\egod1\proj\compiler\w4\tests\valid\02_functions.c" PS C:\Users\egod1\proj\compiler\w4>
해결 / 상태	**A_ID.link**로 선언 목록을 연결하여 **add** → **main** 순서의 출력이 확인된 상태입니다.

17. 문자 상수**:** 문자열 주소를 문자 코드로 출력

문제 **/** 원인	문자 코드 대신 문자열 주소의 일부가 출력된 오류입니다.

문제 코드
실행 출력
	`int main(void) { return 'A'; }`
	`실제 출력 전체, 종료 코드 0:`
	`Parsing success`
	`=======  syntax tree  ==========`
	`N_PROGRAM ((nil),0)`
	`|  (ID="main") TYPE:0x561e365c38b0 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0`
	`|  |  TYPE`
	`|  |  |  FUNCTION`
	`|  |  |  |  PARAMETER`
	`|  |  |  |  |  (ID="") TYPE:0x561e365be360 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0`
	`|  |  |  |  |  |  TYPE`
	`|  |  |  |  |  |  |  (void)|  |  |  |  TYPE`
	`|  |  |  |  |  (int)`
	`|  |  |  |  BODY`
	`|  |  |  |  |  N_STMT_COMPOUND ((nil),0)`
	`|  |  |  |  |  |  N_STMT_LIST ((nil),0)`
	`|  |  |  |  |  |  |  N_STMT_RETURN ((nil),0)`
	`|  |  |  |  |  |  |  |  N_EXP_CHAR_CONST ((nil),0)`
	`|  |  |  |  |  |  |  |  |  912013584`
	`|  |  |  |  |  |  |  N_STMT_LIST_NIL ((nil),0)`
	
	`실제 '\n' 출력: |  |  |  |  |  |  |  |  |  7584016`
	`실제 '\0' 출력: |  |  |  |  |  |  |  |  |  158300432`
	
	`Get-ChildItem Env:AST_TEST* | Remove-Item`
	`$env:AST_PRINT="1"`
	`.\parser.cmd tests/ast/62_char_string.c`

해결 **/** 상태	문자 코드를 정수로 저장하고 검증기의 기대값도 바꾸어야 하는 오류입니다.

18. 함수 원형 매개변수 스코프 누출

문제 / 원인	함수 매개변수가 전역 스코프에 남아 잘못 검색되는 오류입니다.
문제 코드 실행 출력	재현 A, 원형과 정의가 같은 매개변수 이름을 사용: int f(int x); int f(int x) { return x; } int main(void) { return f(1); } [ERROR] Semantic error: Redeclaration of parameter 재현 B, 선언되지 않은 이름이 다른 함수에서 허용됨: int f(int x); int main(void) { return x; } 실제 출력 전체, 종료 코드 0: Parsing success ======= syntax tree ========== N_PROGRAM ((nil),0) | (ID="f") TYPE:0x55e1ef94b8d0 KIND:FUNC SPEC=AUTO LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="x") TYPE:0x55e1ef9462a0 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (int) | | | | TYPE | | | | | (int) | (ID="main") TYPE:0x55e1ef94ba10 KIND:FUNC SPEC=NULL LEV=0 VAL=0 ADDR=0 | | TYPE | | | FUNCTION | | | | PARAMETER | | | | | (ID="") TYPE:0x55e1ef946360 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | TYPE | | | | | | | (void)| | | | TYPE | | | | | (int) | | | | BODY | | | | | N_STMT_COMPOUND ((nil),0) | | | | | | N_STMT_LIST ((nil),0) | | | | | | | N_STMT_RETURN ((nil),0) | | | | | | | | N_EXP_IDENT ((nil),0) | | | | | | | | | (ID="x") TYPE:0x55e1ef9462a0 KIND:PARM SPEC=NULL LEV=0 VAL=0 ADDR=0 | | | | | | | N_STMT_LIST_NIL ((nil),0) 재현 C, 전역 변수와 함수 매개변수의 정상적인 가림도 거부됨: int x; int f(int x) { return x; } [ERROR] Semantic error: Redeclaration of parameter 'int f(int x); int f(int x) { return x; }' | .\parser.exe 'int f(int x); int main(void) { return x; }' | .\parser.exe 'int x; int f(int x) { return x; }' | .\parser.exe
해결 / 상태	매개변수 스코프를 분리하고 종료 시 검색 상태를 복원해야 하는 오류입니다.

19. AST 검증기 6건: 순서와 스코프 가정이 오래됨

문제 / 원인	검증기가 이전 선언 순서와 스코프 규칙을 사용하는 오류입니다.
문제 코드 실행 출력	전역 선언 목록 재현: int a; int b; int c; Parsing success [ERROR] Semantic error: Incorrect global declaration order 지역 변수 관련 재현: AST_TEST_ADDRESS_OF / 15_address_of.c, AST_TEST_PRE_INC / 18_pre_increment.c, AST_TEST_PRE_DEC / 19_pre_decrement.c, AST_TEST_POST_INC / 20_post_increment.c, AST_TEST_POST_DEC / 21_post_decrement.c. Parsing success [ERROR] Semantic error: Identifier mismatch Get-ChildItem Env:AST_TEST* | Remove-Item $env:AST_TEST_ADDRESS_OF="1" .\parser.cmd tests/ast/15_address_of.c Remove-Item Env:AST_TEST_ADDRESS_OF $env:AST_TEST_PROGRAM="1" 'int a; int b; int c;' | .\parser.exe Remove-Item Env:AST_TEST_PROGRAM
해결 / 상태	전역 목록은 **link** 순회, 지역 변수는 함수 본문의 선언 목록으로 검사하는 수정안입니다.

20. void 출력 뒤 줄바꿈 누락

문제 / 원인	(void) 뒤에 줄바꿈이 없어 다음 출력이 붙는 서식 오류입니다.
문제 코드 실행 출력	void f(void) { return; } int main(void) { f(); return 0; } 실제 출력 발췌: | | | | | | | (void)| | | | TYPE | | | | | (void)| | | | BODY | | | | | | | (void)| | | | TYPE
해결 / 상태	개행을 추가하는 수정안입니다. 제공된 print.c 보존을 위해 미적용입니다.
수정안	`printf("(void)\n");`

21. typedef 가림과 미지원 문법의 별도 분류

문제 / 원인	내부 스코프의 **typedef** 가림과 미지원 문법 처리에 관한 항목입니다.
문제 코드 실행 출력	typedef int I; int main(void) { typedef float I; I y; return 0; } [ERROR] Semantic error: Duplicate type specifier 01_declarations, 06_storage_qualifiers, 07_arrays_and_calls: Initializers are not supported. 04_struct_enum: Enumerator initializers are not supported. 05_operators: Left shift is not supported. 09_bitwise: Bitwise AND is not supported. 13_variadic_function: Variadic parameters are not supported. 15_comma_and_conditional: Comma expressions are not supported.
해결 / 상태	typedef 가림은 지원 범위 확인 대상입니다. 미지원 문법 실패는 별도 분류 대상입니다.

사진 원본과 w4-1 구현의 차이

비교 대상: w4-1/img의 부록 사진과 현재 w4-1 소스

결론: 현재 코드는 사진의 구문 분석기 구조를 따르지만, 과제 범위에 맞춰 일부 기능을 제거하고 이식성·빌드 안정성을 보완한 버전이다. 따라서 사진을 그대로 옮긴 1:1 복사본은 아니다.

1. 차이 요약

1-1. main.c

사진 원본

부록 A9의 main.c는 a.asm 파일을 열고 initialize(), yyparse(), semantic_analysis(root), semantic_err 확인, code_generation(root)을 차례로 수행합니다.

현재 구현

C 구문분석기만 구현하는 과제 범위에 맞춰 의미분석·코드생성·어셈블리 파일 생성 부분을 제거했습니다. root와 print_ast() 선언을 추가했고, yyparse() 실패 또는 syntax_err 발생 시 오류 종료합니다. 성공 시 print_ast(root)를 호출해 신택스 트리를 출력합니다.

현재 코드

```
extern A_NODE *root;
void print_ast(A_NODE *);


int main() {
  initialize();


  if (yyparse() || syntax_err)
    return (1);


  print_ast(root);
  return (0);
}
```

변경 이유

현재 소스에는 semantic_analysis(), semantic_err, code_generation() 구현이 포함되어 있지 않으며, 구문분석기 범위에도 필요하지 않습니다.

검증 상태

Makefile을 사용하지 않고 bison, flex, gcc를 직접 실행해 컴파일과 링크에 성공했습니다. 정상 C 입력은 신택스 트리를 출력하고 종료 코드 0을 반환했습니다. 문법 오류 입력은 AST를 출력하지 않고 오류 메시지와 종료 코드 1을 반환했습니다. 이번 변경에서는 main.c만 수정했습니다.

1-2. YYSTYPE 64비트 대응

사진 원본

c.y는 Yacc의 기본 semantic value 형식을 사용하고, c.l은 yylval을 int로 외부 선언합니다. 64비트 UNIX 환경에서는 4바이트 int에 8바이트 포인터를 저장할 수 없습니다.

현재 구현

c.y와 c.l의 선언부 최상단에 다음 정의를 추가하고, c.l의 extern int yylval; 선언을 삭제했습니다.

추가 코드

```
#define YYSTYPE_IS_DECLARED 1

typedef long YYSTYPE;
```

변경 이유

64비트 UNIX의 LP64 환경에서 long은 8바이트이므로 Yacc의 semantic value와 yylval이 8바이트 포인터 값을 보존할 수 있도록 하기 위한 첫 번째 필수 수정입니다. 기존 문법과 함수 구현은 변경하지 않았습니다.

검증 상태

Bison과 Flex의 코드 생성에 성공했고, 현재 환경에서 LONG_BIT=64임을 확인했습니다. 생성된 y.tab.c에는 YYSTYPE yylval이 반영됐습니다. 이 단계는 전체 컴파일 성공을 의미하지 않습니다.

1-3. 함수 프로토타입 선언

사진 원본

support.c 상단에는 support 함수들의 프로토타입이 있지만 c.y는 이 선언을 참조하지 않습니다. 따라서 포인터를 반환하는 함수가 선언 없이 호출되면 반환값이 int로 해석될 수 있습니다.

현재 구현

support.h를 생성하고 support.c에 있던 프로토타입을 그대로 옮겼습니다. support.h는 type.h를 포함하며, c.y와 support.c는 support.h를 포함합니다. support.c의 중복 프로토타입은 제거했고 Makefile의 HEADERS에 support.h를 추가했습니다.

주요 변경

```
#include "support.h"

HEADERS := type.h support.h
```

변경 이유

포인터를 반환하는 support 함수가 int 반환 함수로 잘못 해석되는 문제를 방지하고 과제 조건 (3)을 충족하기 위한 수정입니다. 함수 이름·반환 타입·구현, 문법 규칙과 자료구조는 변경하지 않았습니다.

검증 상태

Bison과 Flex 코드 생성 후 C11 직접 컴파일을 수행했습니다. makeNode 등 support 함수의 암시적 int 반환 오류는 제거됐습니다. 전체 빌드는 아직 성공하지 않으며, 다음 단계에서 yylex 선언, YYSTYPE과 포인터 간 명시적 변환, 표준 라이브러리 선언 등 남은 오류를 수정해야 합니다.

1-4. Makefile 없는 직접 컴파일

목표 및 결과

Makefile을 사용하거나 수정하지 않고 bison, flex, gcc 명령만으로 w4-1 구문분석기를 생성하도록 수정했습니다. 세 명령 모두 성공했으며 parser 실행 파일이 생성됩니다.

컴파일 명령

```
bison -d -o y.tab.c c.y
```

```
flex -o lex.yy.c c.l
```

```
gcc -o parser y.tab.c lex.yy.c support.c print.c main.c
```

추가 수정

c.y에는 stdio.h와 yylex·yyerror 선언 및 64비트 LP64 변환 처리를 추가했습니다. c.l에는 표준 헤더, 정확한 로컬 함수 프로토타입과 noyywrap 옵션을 추가했습니다. support.c에는 string.h와 누락된 isPointerOrArrayType() 구현을 추가했습니다. 기존 문법 규칙, 자료구조와 기존 함수 동작은 변경하지 않았습니다.

실행 방법

```
./parser < 입력파일.c
```

검증 결과

생성된 parser는 ARM64 64비트 실행 파일입니다. 정상 C 입력은 종료 코드 0, 문법 오류 입력은 오류 메시지와 종료 코드 1을 반환했습니다. dangling-else에 따른 shift/reduce conflict 1개와 교수님 원본 구조에서 비롯된 비치명적 경고는 남아 있습니다.

1-5. 구문분석과 무관한 코드 정리 (2026.11.08)
대상: w4-1/c.l, w4-1/support.c
수정 전: c.l의 checkIdentifier()에 사용하지 않는 지역 변수 char *t가 선언되어 있었습니다. support.c의 setDefaultSpecifier()에도 사용하지 않는 지역 변수 A_TYPE *t가 선언되어 있었고, support.c는 실제로 참조하지 않는 생성 헤더 y.tab.h를 포함하고 있었습니다.
수정 후: 위 두 지역 변수 선언과 support.c의 #include "y.tab.h"만 제거했습니다.
제거 근거: 두 지역 변수는 함수 본문에서 참조되지 않으며, support.c에 필요한 자료형과 함수 선언은 support.h와 type.h에서 제공합니다. 구문분석 함수, 심볼/타입 테이블 구성, 스코프 및 오류 검사, 신택스 트리 생성 로직은 유지했습니다.
원본 보존: 교수님 제공 type.h(A1)와 print.c(A5)는 수정하지 않았습니다. c.y의 문법 규칙, main.c와 다른 구현도 이번 커밋에서는 변경하지 않았습니다.
검증 상태: GitHub 커밋의 변경 파일과 diff를 확인했습니다. 이번 정리 이후의 직접 컴파일 및 실행 검증은 아직 수행하지 않았습니다.
커밋: 49e607d9388f508aea471031f0929cd3b5c2e6bc (Remove unused parser-local declarations in w4-1)